

xv6: نظام تشغيل تعليمي بسيط يشبه Unix

الترجمة العربية الأكاديمية المعتمدة وفق معايير الترجمة البرمجية والتحليل النحوي

تأليف: روس كوكس ، فرانس كاشوك ، روبيرت موريس
قسم الهندسة الكهربائية وعلم الحاسوب - معهد ماساتشوستس للتكنولوجيا (MIT)

جدول المحتويات

1	التصدير والشكر والتقدير	5
2	واجهات نظام التشغيل	5
2.1	العمليات والذاكرة	6
2.2	الإدخال/الإخراج وواصفات الملفات	9
2.3	الأنابيب	11
2.4	نظام الملفات	12
2.5	العالم الحقيقي	13
2.6	تمارين	13
3	تنظيم نظام التشغيل	14
3.1	تجريد الموارد الفيزيائية	14
3.2	نمط المستخدم، نمط المشرف، واستدعاءات النظام	15
3.3	تنظيم النواة	15
3.4	الكود البرمجي: تنظيم xv6	18
3.5	نظرة عامة على العملية	18
3.6	الكود البرمجي: بدء تشغيل xv6 والعملية الأولى	20
3.7	نموذج الأمان	21
3.8	العالم الحقيقي	21
3.9	تمارين	21
4	جداول الصفحات	22
4.1	عتاد الصفحات	22
4.2	مساحة عناوين النواة	26
4.3	الكود البرمجي: إنشاء مساحة العناوين	27
4.4	تخصيص الذاكرة الفيزيائية	28
4.5	الكود البرمجي: مخصص الذاكرة الفيزيائية	28
4.6	مساحة عناوين العملية	29
4.7	الكود البرمجي: exec	30
4.8	العالم الحقيقي	32
4.9	تمارين	32
5	المصادر واستدعاءات النظام	32
5.1	آليات المصادر في معالج RISC-V	33
5.2	المصادر القادمة من مساحة المستخدم	35
5.3	الكود البرمجي: استدعاء استدعاءات النظام	36
5.4	الكود البرمجي: معاملات استدعاءات النظام	37
5.5	المصادر القادمة من مساحة النواة	37
5.6	العالم الحقيقي	38
5.7	تمارين	38
6	أخطاء الصفحات	38
6.1	التخصيص الكسول للذاكرة	39
6.2	الكود البرمجي: التخصيص الكسول	39
6.3	العالم الحقيقي: النسخ عند الكتابة (COW Fork)	40
6.4	العالم الحقيقي: استدعاء الصفحات عند الطلب (Demand Paging)	40
6.5	العالم الحقيقي: الملفات المربوطة بالذاكرة (Memory-Mapped Files)	41
6.6	تمارين	41
7	المقاطعات وبرامج تشغيل الأجهزة	41

42	7.1 الكود البرمجي: مدخلات لوحة التحكم
43	7.2 الكود البرمجي: مخرجات لوحة التحكم
43	7.3 التزامن في برامج تشغيل الأجهزة
43	7.4 مقاطعات الميقاتي
44	7.5 العالم الحقيقي
44	7.6 تمارين
45	8 الأقفال
45	8.1 حالات التنافس
48	8.2 الكود البرمجي: الأقفال
49	8.3 الكود البرمجي: استخدام الأقفال
50	8.4 الانسداد المتبادل وترتيب الأقفال
51	8.5 الأقفال والمقاطعات
52	8.6 ترتيب التعليمات والذاكرة
53	8.7 البرمجة بالعمليات الذرية
53	8.8 أقفال النوم
53	8.9 العالم الحقيقي
54	8.10 تمارين
54	9 الجدولة
55	9.1 المضاعفة
55	9.2 نظرة عامة على تبديل السياق
55	9.3 الكود البرمجي: تبديل السياق
56	9.4 الكود البرمجي: الجدولة
57	9.5 الكود البرمجي: mycpu و myproc
58	9.6 العالم الحقيقي
58	9.7 تمارين
58	10 النوم والاستيقاظ
58	10.1 نظرة عامة
60	10.2 الكود البرمجي: النوم والاستيقاظ
60	10.3 الكود البرمجي: الأنابيب
61	10.4 الكود البرمجي: الانتظار، الخروج، والقتل
62	10.5 قفل العملية
62	10.6 العالم الحقيقي
63	10.7 تمارين
63	11 نظام الملفات
64	11.1 نظرة عامة
65	11.2 طبقة المخزن المؤقت للكتل
66	11.3 الكود البرمجي: المخزن المؤقت للكتل
66	11.4 الكود البرمجي: مخصص الكتل
67	11.5 طبقة العقد الفهرسية
68	11.6 الكود البرمجي: العقد الفهرسية
69	11.7 الكود البرمجي: محتوى العقد الفهرسية
70	11.8 الكود البرمجي: طبقة الدلائل
71	11.9 الكود البرمجي: أسماء المسارات
71	11.10 طبقة واصفات الملفات
72	11.11 الكود البرمجي: استدعاءات النظام

73	11.12 العالم الحقيقي
73	11.13 تمارين
74	12 تسجيل المعاملات
74	12.1 تصميم السجل
75	12.2 الكود البرمجي: التسجيل
76	12.3 العالم الحقيقي
76	12.4 تمارين
77	13 العودة لموضوع التزامن
77	13.1 أنماط القفل
77	13.2 أنماط شبيهة بالأقفال
78	13.3 بدون أقفال على الإطلاق
78	13.4 التوازي
79	13.5 تمارين
79	14 الخاتمة
79	15 ملحق الإعراب النحوي والتحليل اللغوي
79	15.1 أولاً: إعراب الجمل المفتاحية في واجهات ونظام التشغيل
79	15.1.1 جملة: «تَصْنَعُ النَوَاةَ عَمَلِيَّةً جَدِيدَةً»
80	15.1.2 جملة: «تُتَقَدُّ العملية إِسْتِدْعَاءَ النَّطَامِ»
80	15.1.3 جملة: «يُحَقِّقُ القفل الإِسْتِيعَادَ الْمُتَبَادَلَ»
80	15.1.4 جملة: «تُحَزَّنُ النَوَاةُ العُنْوَانُ الإِفْتِرَاضِيَّ فِي جدول الصفحات»
80	15.2 ثانياً: إعراب المصطلحات الفنية الموحدة (TermBase)
80	15.2.1 مصطلح: «مخزن الترجمة المؤقت» (Translation Lookaside Buffer - TLB)
80	15.2.2 مصطلح: «العقدة الفهرسية» (Inode)
80	15.2.3 مصطلح: «واصف الملف» (File Descriptor)
80	15.3 ثالثاً: قواعد ضبط أواخر الكلمات والجماليات النحوية في الترجمة

1 التصدير والشكر والتقدير

هذا النص عبارة عن مسودة مخصصة لمقرر دراسي في نظم التشغيل. وهو يشرح المفاهيم الأساسية لنظم التشغيل من خلال دراسة نواة تطبيقية نموذجية تُسمى xv6. ضُممت xv6 استناداً إلى الإصدار السادس من نظام تشغيل (v6) Unix المطور من قبل دينيس ريتشي وكن ثومبسون. وتتبع xv6 البنية والأسلوب العام للإصدار السادس بمرونة، إلا أنها مُنفذة بلغة ANSI C ومخصصة لمعالجات RISC-V متعددة النوى.

يُوصى بقراءة هذا النص جنباً إلى جنب مع الشفرة المصدرية لنظام xv6؛ وهو منهج مستوحى من تعليقات جون لاينز على الإصدار السادس من UNIX؛ ويحتوي النص على روابط فائقة للشفرة المصدرية عبر <https://github.com/mit-pdos/xv6-riscv>. انظر <https://pdos.csail.mit.edu/6.828/> للحصول على إشارات إضافية للموارد المتاحة على الشبكة لكل من v6 و xv6، بما في ذلك العديد من الواجبات العملية التي تستخدم xv6.

لقد استخدمنا هذا النص في المقررين 6.828 و 6.810، وهما مقررا نظم التشغيل بمعهد ماساتشوستس للتكنولوجيا (MIT). ونتوجه بالشكر لأعضاء هيئة التدريس، والمساعدات التعليميين، والطلاب في تلك المقررات الذين ساهموا جميعاً بشكل مباشر أو غير مباشر في تطوير xv6. وبشكل خاص، نود أن نشكر كلاً من أدام بيلاي، وأوستن كليمنتس، ونيكولاي زيلدوفيتش. وأخيراً، نود شكر كل من تواصل معنا عبر البريد الإلكتروني للإبلاغ عن أخطاء في النص أو تقديم اقتراحات للتحسين: Abutalib Aghayev, Sebastian Boehm, brandb97, Anton Burtsev, Raphael Carvalho, Tej Chajed, Brendan Davidson, Rasit Eskicioglu, Color Fuzzy, Wojciech Gac, Giuseppe, Tao Guo, Haibo Hao, Naoki Hayama, Chris Henderson, Robert Hilderan, Eden Hochbaum, Wolfgang Keller, Paweł Kraszewski, Henry Lai, Jin Li, Austin Liew, lyazj@github.com, Pavan Maddamsetti, Jacek Masiulaniec, Michael McConville, m3hm00d, Mes0903, miguelgvieira, Mark Morrissey, Muhammed Mourad, Harry Pan, Harry Porter, pr3pony, Siyuan Qian, Zhefeng Qiao, Askar Safin, Salman Shah, Huang Sha, Vikram Shenoy, Adeodato Simó, Ruslan Savchenko, Paweł Szczurko, Warren Toomey, tyfkda, tzerbib, unicornx, Vanush Vaswani, Chen Wang, Xi Wang, and Zou Chang Wei, Sam Whitlock, Qionsgi Wu, LucyShawYang, ykf1114@gmail.com, and Meng Zhou.

إذا لاحظت أي أخطاء أو كانت لديك اقتراحات للتحسين، يُرجى إرسال بريد إلكتروني إلى فرانس كاشوك وروبيرت موريس عبر (kaashoek, rtm@csail.mit.edu).

2 واجهات نظام التشغيل

وظيفة نظام التشغيل هي مشاركة حاسوب بين برامج متعددة وتوفير طقم نافع من الخدمات يفوق ما يدعمه العتاد بمفرده. يدير نظام التشغيل العتاد منخفض المستوى ويجرده، لكي لا يحتاج، على سبيل المثال، معالج نصوص للانشغال بأي نوع من عتاد القرص المعتمد. يشارك نظام التشغيل العتاد بين برامج متعددة لكي تنفذ (أو تبدو أنها تنفذ) في الوقت نفسه. أخيراً، توفر نظم التشغيل طرقاً خاضعة للتحكم للبرامج للتفاعل، لكي تتمكن من مشاركة البيانات أو العمل معاً.

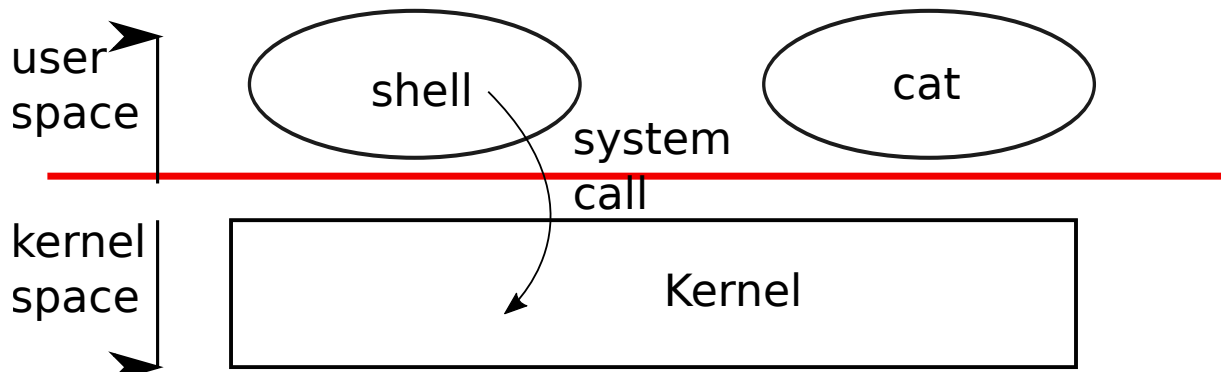
يوفر نظام التشغيل الخدمات لبرامج المستخدم عبر واجهة. تصميم واجهة جيدة يتبين أنه أمر بالغ الصعوبة. فمن جهة، نود أن تكون الواجهة بسيطة ومحدودة النطاق لأن ذلك يجعل من الأسهل جعل التنفيذ صحيحاً. ومن جهة أخرى، قد نميل إلى تقديم العديد من الميزات المعقدة للتطبيقات. الحيلة في حل هذا التوتر هي تصميم واجهات تعتمد على بضع آليات يمكن دمجها لتوفير عمومية كبيرة.

يستخدم هذا الكتاب نظام تشغيل واحداً كمثال ملموس لتوضيح مفاهيم نظام التشغيل. ذلك النظام، xv6، يوفر الواجهات الأساسية المقدمة بواسطة نظام تشغيل Unix الكائن لـ Ken Thompson و Dennis Ritchie، فضلاً عن محاكاة التصميم الداخلي لـ Unix. يوفر Unix واجهة ضيقة تندمج آلياتها جيداً، مقدمة درجة مذهلة من العمومية. كانت هذه الواجهة ناجحة جداً لدرجة أن نظم التشغيل الحديثة—BSD، Linux، macOS، Solaris، وحتى بدرجة أقل، Microsoft Windows—تحوز واجهات شبيهة بـ Unix. فهم xv6 بداية جيدة نحو فهم أي من هذه النظم والنظم الأخرى الكثيرة.

كما يوضح الشكل الشكل 1، يأخذ xv6 الشكل التقليدي لـ النواة (Kernel)، وهو برنامج خاص يوفر الخدمات للبرامج المنفذة. كل برنامج منفذ، يُسمى عملية (Process)، يحوز ذاكرة تحوي التعليمات، البيانات، والمكدس. تنفذ التعليمات حسابات البرنامج. البيانات هي المتغيرات التي تعمل عليها الحسابات. ينظم المكدس استدعاءات إجراءات البرنامج. حاسوب محدد تحوز عادة عمليات كثيرة ولكن نواة واحدة فقط.

عندما تحتاج عملية لاستدعاء خدمة من النواة، تدعو استدعاء نظام (System call)، واحداً من الاستدعاءات في واجهة نظام التشغيل. يدخل استدعاء النظام النواة؛ تنفذ النواة الخدمة وترجع. وهكذا تتناوب العملية بين التنفيذ في مساحة المستخدم (User space) و مساحة النواة (Kernel space).

كما هو موصف بالتفصيل في الفصول التالية، تستخدم النواة آليات الحماية العتادية الموفرة بواسطة المعالج لضمان أن كل عملية تنفذ في مساحة المستخدم يمكنها الوصول فقط لذاكرتها الخاصة. تنفذ النواة بالامتيازات العتادية المطلوبة لتنفيذ هذه الحماية؛ تنفذ برامج المستخدم بدون تلك الامتيازات. عندما يدعو برنامج مستخدم استدعاء نظام، يرفع العتاد مستوى الامتياز ويبدأ تنفيذ دالة مرتبة مسبقاً في النواة.



شكل 1: نواة وعمليتا مستخدم.

مجموعة استدعاءات النظام التي توفرها النواة هي الواجهة التي تشاهدها برامج المستخدم. توفر نواة xv6 طقماً فرعياً من الخدمات واستدعاءات النظام التي تقدمها نوى Unix تقليدياً. يعرض الجدول جدول 1 كافة استدعاءات النظام لـ xv6.

يرسم بقية هذا الفصل خدمات xv6—العمليات، الذاكرة، واصفات الملفات، الأنابيب، ونظام الملفات—وبوضحها بمقاطع شفرة ومناقشات لكيفية استخدام موجه الأوامر (Shell)، واجهة مستخدم سطر الأوامر بـ Unix، لها. استخدام موجه الأوامر لاستدعاءات النظام يوضح مدى الحذر الذي صُممت به.

موجه الأوامر برنامج عادي يقرأ الأوامر من المستخدم وينفذها. حقيقة كون موجه الأوامر برنامج مستخدم، وليس جزءاً من النواة، توضح قوة واجهة استدعاءات النظام: لا يوجد شيء خاص بخصوص موجه الأوامر. كما يعني هذا أن موجه الأوامر سهل الاستبدال؛ ونتيجة لذلك فإن نظم Unix الحديثة تحوز تنوعاً من موجهات الأوامر للاختيار من بينها، كل منها بواجهة مستخدمها وميزات برمجتها النصية. موجه أوامر xv6 تنفيذ بسيط لجوهر موجه أوامر Bourne بـ Unix.

يمكن العثور على تنفيذ موجه أوامر xv6 في user/sh.c. ممارسة جيدة هي محاولة قراءة الشفرة المصدرية أولاً بمفردك ثم العودة لهذا الكتاب. بحلول نهاية هذا الكتاب ينبغي أن تكون قادراً على فهم كل سطر من الشفرة المصدرية لـ xv6 دون الحاجة لاستشارة هذا الكتاب.

2.1 العمليات والذاكرة

تتكون عملية xv6 من ذاكرة مساحة مستخدم (تعليمات، بيانات، ومكدس) وحالة خاصة بكل عملية حصرية بالنواة. تتقاسم xv6 العمليات بالمشاركات الزمنية: تبدل شفافياً المعالجات المتاحة بين طقم العمليات المنتظرة للتنفيذ. عندما لا تكون العملية تنفذ، تحفظ xv6 سجلات المعالج للعملية، مستعادة إياها عندما تنفذ العملية تالياً. تربط النواة معرف عملية، أو PID، مع كل عملية.

الوصف	استدعاء النظام
إنشاء عملية، إرجاع PID الابن.	<code>int fork()</code>
إنهاء العملية الحالية؛ إبلاغ الحالات لـ <code>wait()</code> . لا رجوع.	<code>int exit(int status)</code>
الانتظار لخروج ابن؛ حالة الخروج بـ <code>*status</code> ؛ إرجاع PID الابن.	<code>int wait(int *status)</code>
إنهاء العملية PID. إرجاع 0، أو -1 للخطأ.	<code>int kill(int pid)</code>
إرجاع PID العملية الحالية.	<code>int getpid()</code>
إيقاف مؤقت لـ <code>n</code> دقة ساعة.	<code>int pause(int n)</code>
تحميل ملف وتنفيذه بمعاملات؛ يرجع فقط عند الخطأ.	<code>int exec(char *file, char *argv[])</code>
تنمية ذاكرة العملية بـ <code>n</code> بايت صفري. إرجاع بداية الذاكرة الجديدة.	<code>char *sbrk(int n)</code>
فتح ملف؛ العلامات تشير للقراءة/الكتابة؛ إرجاع واصف ملف (<code>fd</code>).	<code>int open(char *file, int flags)</code>
كتابة <code>n</code> بايت من <code>buf</code> لوصف الملف <code>fd</code> ؛ إرجاع <code>n</code> .	<code>int write(int fd, char *buf, int n)</code>
قراءة <code>n</code> بايت في <code>buf</code> ؛ إرجاع عدد المقروء؛ أو 0 لنهاية الملف.	<code>int read(int fd, char *buf, int n)</code>
تحرير الملف المفتوح <code>fd</code> .	<code>int close(int fd)</code>
إرجاع واصف ملف جديد يشير لنفس الملف كـ <code>fd</code> .	<code>int dup(int fd)</code>
إنشاء أنبوب، وضع واصفات ملفات القراءة/الكتابة بـ <code>p[0]</code> و <code>p[1]</code> .	<code>int pipe(int p[])</code>
تغيير المجلد الحالي.	<code>int chdir(char *dir)</code>
إنشاء مجلد جديد.	<code>int mkdir(char *dir)</code>
إنشاء ملف جهاز.	<code>int mknod(char *file, int, int)</code>
وضع معلومات عن ملف مفتوح في <code>*st</code> .	<code>int fstat(int fd, struct stat *st)</code>
إنشاء اسم آخر (<code>file2</code>) للملف <code>file1</code> .	<code>int link(char *file1, char *file2)</code>
إزالة ملف.	<code>int unlink(char *file)</code>

جدول 1: استدعاءات نظام xv6. ما لم يُذكر غير ذلك، ترجع 0 لعدم وجود خطأ، و -1 في حالة الخطأ.

قد تنشئ عملية ما عملية جديدة باستخدام استدعاء النظام `fork`. تعطي العملية الجديدة نسخة متطابقة من ذاكرة العملية المستدعية: تنسخ `fork` التعليمات، البيانات، والمكدس للعملية المستدعية في ذاكرة العملية الجديدة. ترجع `fork` في كل من العمليتين الأصلية والجديدة. في العملية الأصلية ترجع PID `fork` العملية الجديدة. في العملية الجديدة ترجع `fork` صفراً. تُسمى العمليتان الأصلية والجديدة غالباً بـ الأب و الابن.

على سبيل المثال، راجع المقطع البرمجي التالي المكتوب بلغة البرمجة C:

```
int pid = fork();
if(pid > 0)
    printf("parent: child=%d\n", pid);
pid = wait((int *) 0);
printf("child %d is done\n", pid);
else if(pid == 0) {
    printf("child: exiting\n");
    exit(0);
} else {
    printf("fork error\n");
}
```

يتسبب استدعاء النظام exit في إيقاف العملية المستدعية عن التنفيذ وتحرير الموارد مثل الذاكرة والملفات المفتوحة. يأخذ Exit معامل حالة كعدد صحيح، تقليدياً 0 للإشارة للنجاح و 1 للإشارة للفشل. يرجع استدعاء النظام PID wait ابن خارج (أو مقتول) للعملية الحالية وينسخ حالة خروج الابن للعنوان الممرر ل wait؛ إذا لم يكن أي من أبناء المستدعي قد خرج، تنتظر wait واحداً ليفعل ذلك. إذا لم يحز المستدعي أبناء، ترجع wait فوراً -1. إذا كان الأب لا يهتم بحالة خروج ابن، يمكنه تمرير عنوان 0 ل wait.

في المثال، أسطر المخرجات:

```
parent: child=1234
child: exiting
```

قد تخرج في أي من الترتيبين (أو حتى مختلطة)، اعتماداً على ما إذا كان الأب أو الابن يصل لاستدعائه ل printf أولاً. بعد خروج الابن، يرجع wait الأب، متسبباً في جعل الأب يطبع:

```
parent: child 1234 is done
```

على الرغم من أن الابن يبدأ بنسخة من ذاكرة الأب، إلا أن الأب والابن ينفذان بذاكرة منفصلة وسجلات منفصلة: تغيير متغير في أحدهما لا يؤثر على الآخر. على سبيل المثال، عندما تُخزن القيمة المرجوعة ل wait في pid في عملية الأب، فإن ذلك لا يغير المتغير pid في الابن. قيمة pid في الابن ستظل صفراً.

استدعاء النظام exec يستبدل ذاكرة العملية المستدعية بصورة ذاكرة جديدة ممررة من ملف مخزن في نظام الملفات. يجب أن يحوز الملف طرازاً محدداً، يحدد أي جزء من الملف يحوي التعليمات، أي جزء هو البيانات، عند أية تعليمة يبدأ، إلخ. تستخدم xv6 طراز ELF، والذي يناقشه الفصل الفصل 4 بتفصيل أكثر. عادة ما يكون الملف نتيجة ترجمة شفرة مصدرة لبرنامج. عندما تنجح exec فإنها لا ترجع للبرنامج المستدعي؛ بدلاً من ذلك تبدأ التعليمات المحملة من الملف التنفيذ عند نقطة الدخول المعلنة في رأس ELF. تأخذ exec معاملين: اسم الملف الحاوي للملف التنفيذي ومصفوفة من المعاملات النصية. على سبيل المثال:

```
;char *argv[3]
```

```
;argv[0] = "echo"
;argv[1] = "hello"
;argv[2] = 0
;exec("/bin/echo", argv)
;printf("exec error\n")
```

هذا المقطع يستبدل البرنامج المستدعي بحالة من البرنامج bin/echo/ ينفذ مع قائمة المعاملات echo hello. معظم البرامج تتجاهل العنصر الأول من مصفوفة المعاملات، والذي يكون تقليدياً اسم البرنامج.

يستخدم موجه أوامر xv6 الاستدعاءات أعلاه لتشغيل البرامج نيابة عن المستخدمين. التركيب الرئيسي لموجه الأوامر بسيط؛ انظر main في user/sh.c. تقرأ الحلقة التكرارية الرئيسية سطرًا من المدخلات من المستخدم بـ getcmd. ثم تدعو fork التي تنشئ نسخة من عملية موجه الأوامر. يدعو الأب wait بينما ينفذ الابن الأمر. على سبيل المثال لو كتب المستخدم echo hello لموجه الأوامر، لكانت runcmd قد دُعيت مع echo hello ك معاملة. تنفذ runcmd الأمر الحقيقي. ل echo hello لكانت قد دعت exec. إذا نجحت exec فإن الابن سينفذ التعليمات من echo بدلاً من runcmd. عند نقطة ما استدعو echo الدالة exit والتي ستتسبب في جعل الأب يرجع من wait في main.

قد تتساءل لماذا لا يندمج fork و exec عند استدعاء واحد؛ سنرى لاحقاً أن موجه الأوامر يستغل الفصل في تنفيذه لإعادة توجيه الإدخال/الإخراج. لتجنب هدر إنشاء عملية مكررة ثم استبدالها فوراً (بـ exec)، تحسن نوى التشغيل تنفيذ fork لهذه حالة الاستخدام باستخدام تقنيات الذاكرة الافتراضية مثل النسخ عند الكتابة (انظر الفصل 6).

تخصص xv6 معظم ذاكرة مساحة المستخدم ضمناً؛ تخصص fork الذاكرة المطلوبة لنسخة الابن من ذاكرة الأب، وتخصص exec ذاكرة كافية لاحتجاز الملف التنفيذي. العملية التي تحتاج ذاكرة أكثر في وقت التنفيذ (ربما ل malloc) يمكنها دعوة sbrk(n) لتنمية ذاكرة بياناتها بمقدار n بايت صفري؛ ترجع sbrk موقع الذاكرة الجديدة.

2.2 الإدخال/الإخراج وواصفات الملفات

واصف الملف (File descriptor) هو عدد صحيح صغير يمثل كائناً تدار بواسطة النواة يمكن للعملية القراءة منه أو الكتابة إليه. قد تحصل العملية على واصلف ملف عبر فتح ملف، مجلد، أو جهاز، أو عبر إنشاء أنبوب، أو عبر مضاعفة واصلف قائم. للتبسيط سنشير غالباً للكائن الذي يشير إليه واصلف الملف كـ «ملف»؛ واجهة واصلف الملف تجرد الفروق بين الملفات، الأنابيب، والأجهزة، جاعلة إياها جميعاً تبدو كتدفقات من البايتات. سنشير للإدخال والمخرجات كـ I/O.

داخلياً تستخدم نواة xv6 واصلف الملف كـ فهرس في جدول خاص بكل عملية، لكي تحوز كل عملية مساحة خاصة من واصفات الملفات تبدأ عند الصفر. تقليدياً تقرأ العملية من واصلف الملف 0 (المدخل المعياري)، وتكتب المخرجات لواصلف الملف 1 (المخرج المعياري)، وتكتب رسائل الأخطاء لواصلف الملف 2 (الخطأ المعياري). كما سنرى، يستغل موجه الأوامر هذا العرف لتنفيذ إعادة توجيه الإدخال/الإخراج والأنابيب. يضمن موجه الأوامر حوزته دائماً لثلاثة واصفات ملفات مفتوحة، والتي تكون افتراضياً واصفات ملفات للوحة التحكم.

يقرأ استدعاء النظام read و write بايتات من ويكتبان بايتات إلى ملفات مفتوحة مسماة بواسطة واصفات الملفات. الاستدعاء read(fd, buf, n) يقرأ بحد أقصى n بايت من واصلف الملف fd وينسخها في buf ويرجع عدد البايتات المقروءة. كل واصلف ملف يشير لملف يحوز إزاحة مرتبطة به. تقرأ read البيانات من إزاحة الملف الحالية ثم تقدم تلك الإزاحة بعدد البايتات المقروءة: قراءة لاحقة ترجع البايتات التالية لتلك المرجوعة بواسطة القراءة الأولى. عندما لا تتوفر بايتات أخرى للقراءة، ترجع read صفراً للإشارة لنهاية الملف.

الاستدعاء write(fd, buf, n) يكتب n بايت من buf لواصلف الملف fd ويرجع عدد البايتات المكتوبة. يُكتب أقل من n بايت فقط عند وقوع خطأ. مثل read تكتب write البيانات عند إزاحة الملف الحالية ثم تقدم تلك الإزاحة بعدد البايتات المكتوبة: كل write تبدأ من حيث انتهت السابقة.

المقطع البرمجي التالي (والذي يشكل جوهر البرنامج cat) ينسخ البيانات من مدخله المعياري لمخرجه المعياري. إذا وقع خطأ يكتب رسالة للخطأ المعياري.

```
char buf[512];
int n;

for(;;){
    n = read(0, buf, sizeof buf);
    if(n == 0)
        break;
    if(n < 0)
        fprintf(2, "read error\n");
        exit(1);
    {
        if(write(1, buf, n) != n)
            fprintf(2, "write error\n");
            exit(1);
    }
}
```

الشيء الهام للملاحظة في المقطع البرمجي هو أن cat لا تعلم ما إذا كانت تقرأ من ملف، لوحة تحكم، أو أنبوب. بالمثل لا تعلم cat ما إذا كانت تطبع للوحة تحكم، ملف، أو أي شيء. استخدام واصفات الملفات والعرف بأن واصلف الملف 0 هو المدخل وواصلف الملف 1 هو المخرج يتيح تنفيذاً بسيطاً لـ cat.

استدعاء النظام close يحرق واصلف ملف، جاعلاً إياه حراً لإعادة الاستخدام بواسطة استدعاء نظام open أو pipe أو dup مستقبلي. واصلف الملف المخصص حديثاً يكون دائماً هو الواصلف غير المستخدم الأقل رقماً للعملية الحالية.

واصفات الملفات و fork يتفاعلان لجعل إعادة توجيه الإدخال/الإخراج سهلة التنفيذ. تنسخ fork جدول واصفات ملفات الأب جنباً إلى جنب مع ذاكرته، لكي يبدأ الابن بنفس الملفات المفتوحة بالضبط كالأب. استدعاء النظام exec يستبدل ذاكرة العملية المستدعية لكنه يحتفظ بجدول ملفاتها. هذا السلوك يتيح لموجه الأوامر تنفيذ إعادة توجيه الإدخال/الإخراج (I/O redirection) عبر الاستدعاء لـ fork، إغلاق وإعادة فتح واصفات ملفات مختارة في

الابن، ثم دعوة exec لتشغيل البرنامج الجديد. فيما يلي نسخة مبسطة من الشفرة التي ينفذها موجه الأوامر للأمر

```
cat < input.txt
;char *argv[2]
```

```
;argv[0] = "cat"
;argv[1] = 0
} if(fork() == 0)
;close(0)
;open("input.txt", O_RDONLY)
;exec("cat", argv)
{
```

بعد أن يغلق الابن واصف الملف 0، تضمن open استخدام ذلك الوصف للملف المفتوح حديثاً 0 input.txt سيكون الوصف المتاح الأصغر. تنفذ cat عندئذ مع واصف الملف 0 (المدخل المعياري) مشيراً لـ input.txt. واصفات ملفات عملية الأب لا تتغير بهذا التسلسل، نظراً لأنه يعدل فقط واصفات الابن.

شفرة إعادة توجيه الإدخال/الإخراج في موجه أوامر xv6 تعمل بنفس هذه الطريقة بالضبط. تذكر أنه في هذه النقطة في الشفرة يكون موجه الأوامر قد استدعى بالفعل fork لالابن وأن runcmd ستدعو exec لتحميل البرنامج الجديد. المعامل الثاني لـ open يتكون من طقم من العلامات، معبر عنها كبتات، للتحكم في ما تفعله open. القيم الممكنة معرفة في رأس fcntl: O_RDONLY, O_WRONLY, O_RDWR, O_CREATE, O_TRUNC والتي توجه open لفتح الملف للقراءة، أو الكتابة، أو للقراءة والكتابة، إنشاء الملف إذا لم يكن موجوداً، وقطع الملف لـ صفر طول.

الآن ينبغي أن يكون واضحاً لماذا يُعد نافعاً أن تكون fork و exec استدعاءين منفصلين: بينهما يحوز موجه الأوامر فرصة لإعادة توجيه إدخال/إخراج الابن دون اضطراب إعداد إدخال/إخراج موجه الأوامر الرئيسي. يمكن للمرء بدلاً من ذلك تخيل استدعاء نظام افتراضي مدمج forkexec لكن خيارات القيام بإعادة توجيه الإدخال/الإخراج مع مثل هذا الاستدعاء تبدو غير ملائمة.

على الرغم من أن fork تنسخ جدول واصفات الملفات، فإن كل إزاحة ملف تحتية تكون مشتركة بين الأب والابن. راجع هذا المثال:

```
} if(fork() == 0)
;write(1, "hello ", 6)
;exit(0)
} else {
;wait(0)
;write(1, "world\n", 6)
}
```

في نهاية هذا المقطع، فإن الملف المرتبط بوصف الملف 1 سيحتوي البيانات hello world. الـ write في الأب (والتي، بفضل wait تنفذ فقط بعد انتهائه) تبدأ من حيث انتهت write الابن. هذا السلوك يساعد في إنتاج مخرجات متتالية من تسلسلات أوامر موجه الأوامر، مثل (echo hello; echo world) output.txt.

استدعاء النظام dup يضاعف واصف ملف قائم، مرجعاً واصفاً جديداً يشير لنفس كائن الإدخال/الإخراج التحي. كلا واصفي الملفات يتشاركان الإزاحة، تماماً كما تفعل واصفات الملفات المضاعفة بواسطة fork. هذه طريقة أخرى لكتابة hello world في ملف:

```
;fd = dup(1)
;write(1, "hello ", 6)
;write(fd, "world\n", 6)
```

واصفا ملفات يتشاركان الإزاحة إذا كانا مشتقين من نفس واصف الملف الأصلي بتسلسل من استدعاءات fork و dup. وإلا فإن واصفات الملفات لا تتشارك الإزاحات، حتى وإن نتجت عن استدعاءات open لنفس الملف. تتيح dup لموجهات الأوامر تنفيذ أوامر مثل هذا: 1>2 tmp1 > existing-file non-existing-file ls الـ 2<1 تخبر موجه الأوامر بإعطاء الأمر واصف ملف 2 يكون مضاعفاً للواصف 1. كلا اسم الملف القائم ورسالة الخطأ للملف

غير القائم سيظهران في الملف tmp1. موجه أوامر xv6 لا يدعم إعادة توجيه الإدخال/الإخراج لوصف ملف الخطأ، لكنك تعلم الآن كيفية تنفيذه.

واصفات الملفات تجريد قوي، لأنها تخفي تفاصيل ما هي متصلة به: عملية تكتب لوصف الملف 1 قد تكون تكتب لملف، لجهاز مثل لوحة التحكم، أو لأنبوب.

2.3 الأنابيب

الأنبوب (Pipe) هو حاوي نواة صغير مخرج للعمليات كزوج من واصفات الملفات، واحد للقراءة وواحد للكتابة. كتابة البيانات لأحد طرفي الأنبوب تجعل تلك البيانات متاحة للقراءة من الطرف الآخر للأنبوب. توفر الأنابيب طريقة للعمليات للتواصل.

مثال الشفرة التالي ينفذ البرنامج wc مع إدخال معياري متصل بطرف القراءة للأنبوب.

```
int p[2];
char *argv[2];

argv[0] = "wc";
argv[1] = 0;

pipe(p);
if(fork() == 0)
    close(0);
    dup(p[0]);
    close(p[0]);
    close(p[1]);
    exec("/bin/wc", argv);
else {
    close(p[0]);
    write(p[1], "hello world\n", 12);
    close(p[1]);
}
```

يدعو البرنامج pipe التي تنشئ أنبوباً جديداً وتسجل واصفي ملفات القراءة والكتابة في المصفوفة p. بعد fork يحوز كل من الأب والابن واصفات ملفات تشير للأنبوب. يدعو الابن close و dup لجعل واصف الملف صفر يشير لطرف القراءة للأنبوب، ويغلق واصفات الملفات بـ p ويدعو exec لتشغيل wc. عندما تقرأ wc من مدخلها المعياري، فإنها تقرأ من الأنبوب. يغلق الأب جانب القراءة للأنبوب، ويكتب للأنبوب، ثم يغلق جانب الكتابة.

إذا لم تتوفر بيانات، فإن read على أنبوب تنتظر إما كتابة بيانات أو إغلاق كافة واصفات الملفات المشاركة لطرف الكتابة؛ في الحالة الأخيرة ترجع read صفراً، تماماً كما لو أن نهاية ملف بيانات قد وصلت. حقيقة أن read تحظر حتى يكون من المستحيل وصول بيانات جديدة هي أحد الأسباب في كون الهام للابن إغلاق طرف الكتابة للأنبوب قبل تنفيذ wc أعلاه: لو كان أحد واصفات ملفات wc يشير لطرف الكتابة للأنبوب فلن تشاهد wc نهاية الملف مطلقاً.

ينفذ موجه أوامر xv6 الأنابيب مثل wc -l sh.c | grep fork بطريقة شبيهة بالشفرة أعلاه. تنشئ عملية الابن أنبوباً لربط الطرف الأيسر من الأنبوب بالطرف الأيمن. ثم تدعو fork و runcmd للطرف الأيسر و fork و runcmd للطرف الأيمن، وتنتظر كلاهما لينها. قد يكون الطرف الأيمن أمراً يتضمن هو نفسه أنبوباً (مثلاً a | b | c)، والذي يحوز بدوره استدعائي fork لعمليتي ابن جديدتين (واحدة لـ b وواحدة لـ c). وهكذا قد ينشئ موجه الأوامر شجرة من العمليات. أوراق هذه الشجرة هي أوامر والعقد الداخلية هي عمليات تنتظر حتى يكتمل الأبناء الأيسر والأيمن.

قد تبدو الأنابيب ليست أكثر قوة من الملفات المؤقتة: فالأنبوب echo hello world | wc كـ echo hello world >/tmp/xyz; wc </tmp/xyz في هذه الحالة. أولاً تنظف الأنابيب نفسها تلقائياً؛ مع إعادة توجيه الملف يتعين على موجه الأوامر الحذر لإزالة /tmp/xyz عند الانتهاء. ثانياً يمكن للأنابيب تمرير تدفقات بيانات طويلة عشوائياً، بينما تتطلب إعادة توجيه الملف مساحة حرة كافية على القرص لتخزين كافة البيانات. ثالثاً تتيح الأنابيب التنفيذ المتوازي لمراحل الأنبوب، بينما يتطلب نهج الملف أن ينهي البرنامج الأول قبل بدء الثاني.

2.4 نظام الملفات

يوفر نظام ملفات xv6 ملفات البيانات، التي تحوي مصفوفات بايتات غير مفسرة، والدلائل، التي تحوي مراجع مسماة لملفات بيانات ودلائل أخرى. تشكل الدلائل شجرة تبدأ عند مجلد خاص يُسمى الجذر (Root). مسار مثل /a/b/c يشير للملف أو المجلد المسمى c في المجلد المسمى b في المجلد المسمى a في المجلد الرئيسي /. المسارات التي لا تبدأ ب / تُقيم بالنسبة لـ المجلد الحالي للعملية المستدعية، والذي يمكن تغييره باستدعاء النظام chdir. كلا مقطعي الشفرة هذين يفتحان نفس الملف (بافتراض أن كافة الدلائل المتضمنة موجودة):

```
;chdir("/")
;chdir("b")
;open("c", O_RDONLY)

;open("/a/b/c", O_RDONLY)
```

المقطع الأول يغير المجلد الحالي للعملية لـ a/b/؛ الثاني لا يشير ولا يغير المجلد الحالي للعملية.

توجد استدعاءات نظام لإنشاء ملفات ودلائل جديدة: mkdir تنشئ مجلدًا جديدًا، open مع العلامة O_CREATE تنشئ ملف بيانات جديدًا، و mknod تنشئ ملف جهاز جديدًا. هذا المثال يوضح الثلاثة جميعًا:

```
;mkdir("/dir")
;fd = open("/dir/file", O_CREATE|O_WRONLY)
;close(fd)
;mknod("/console", 1, 1)
```

تنشئ mknod ملفًا خاصًا يشير لجهاز. المرتبط بملف الجهاز هما رقما الجهاز الرئيسي والفرعي (المعاملان لـ mknod)، اللذان يحددان بفرادة جهاز النواة. عندما تفتح عملية لاحقًا ملف جهاز، تحول النواة استدعاءات النظام read و write لتنفيذ جهاز النواة بدلاً من تمريرها لنظام الملفات.

اسم الملف متميز عن الملف نفسه؛ نفس الملف التحتي، المسمى العقدة الفهرسية (Inode)، يمكن أن يحوز أسماء متعددة، تُسمى الروابط (Links). كل رابط يتكون من مدخل في مجلد؛ يحوي المدخل اسم ملف ومرجعاً لعقدة فهرسية. تحتفظ العقدة الفهرسية بـ بيانات وصفية (Metadata) عن ملف، يتضمن نوعه (ملف أو مجلد أو جهاز)، طوله، موقع محتوى الملف على القرص، وعدد الروابط للملف.

استدعاء النظام fstat يستعيد معلومات من العقدة الفهرسية التي يشير إليها واصف ملف. يملأ هيكل struct stat المعروف بـ stat.h كـ:

```
define T_DIR      1 // Directory#
define T_FILE     2 // File#
define T_DEVICE   3 // Device#

} struct stat
int dev; // File system's disk device
uint ino; // Inode number
short type; // Type of file
short nlink; // Number of links to file
uint64 size; // Size of file in bytes
;{
```

استدعاء النظام link ينشئ اسم نظام ملفات آخر يشير لنفس العقدة الفهرسية لملف قائم. هذا المقطع ينشئ ملفاً جديداً مسماً كلاً من a و b.

```
;open("a", O_CREATE|O_WRONLY)
;link("a", "b")
```

القراءة من أو الكتابة لـ a هي نفسها القراءة من أو الكتابة لـ b. كل عقدة فهرسية تحدد بـ رقم عقدة فهرسية فريد. بعد تسلسل الشفرة أعلاه، يكون من الممكن القرار بكون a و b يشيران لنفس المحتويات التحتية عبر فحص نتيجة fstat: كلاهما سيرجعان نفس رقم العقدة الفهرسية (ino) وعداد nlink مضبوطاً لـ 2.

استدعاء النظام unlink يزيل اسماً من نظام الملفات. العقدة الفهرسية للملف والمساحة القرصية الحائزة لمحتواه تُحرران فقط عندما يكون عداد روابط الملف صفراً ولا توجد واصفات ملفات تشير إليه. وهكذا فإن إضافة `unlink("a")`; لتسلسل الشفرة الأخير تترك العقدة الفهرسية ومحتوى الملف قابلي للوصول ك `b`. علاوة على ذلك:

```
;fd = open("/tmp/xyz", O_CREATE|O_RDWR);
unlink("/tmp/xyz")
```

طريقة اصطلاحية لإنشاء عقدة فهرسية مؤقتة دون اسم سُنظف عندما تغلق العملية `fd` أو تخرج.

توفر Unix أدوات ملفات قابلة للدعوة من موجه الأوامر كبرامج مستوى مستخدم، على سبيل المثال `ln` و `mkdir` و `rm`. يتيح هذا التصميم لأي شخص توسيع واجهة سطر الأوامر عبر إضافة برامج مستوى مستخدم جديدة. بإعادة النظر يبدو هذا المخطط واضحاً، لكن النظم الأخرى المصممة في وقت Unix كانت تبني غالباً مثل هذه الأوامر في موجه الأوامر (وتبني موجه الأوامر في النواة).

استثناء واحد هو `cd` المدمجة في موجه الأوامر. يجب على `cd` تغيير المجلد الحالي لموجه الأوامر نفسه. لو نُفذت `cd` كأمر عادي، لكان موجه الأوامر قد استدعى `fork` لعملية ابن، ولنُفذت عملية الابن `cd` ولغيرت `cd` المجلد الحالي لـ الابن. المجلد الحالي للأب (أي موجه الأوامر) لن يتغير.

2.5 العالم الحقيقي

دمج Unix لواصفات الملفات القياسية، الأنابيب، وبنية موجه الأوامر المريحة للعمليات عليها كان تقدماً رئيسياً في كتابة برامج عامة الغرض قابلة لإعادة الاستخدام. أشعلت الفكرة ثقافة «أدوات البرمجيات» التي كانت مسؤولة عن معظم قوة وشعبية Unix، وكان موجه الأوامر أول ما يُسمى بـ «لغات البرمجة النصية». واجهة استدعاءات نظام Unix تستمر اليوم في نظم مثل BSD و Linux و macOS.

جرت معايير واجهة استدعاءات نظام Unix عبر معيار `xv6`. POSIX ليست متوافقة مع POSIX: تفتقد استدعاءات نظام كثيرة (تتضمن الأساسية مثل `lseek`)، والعديد من استدعاءات النظام التي توفرها تختلف عن المعيار. أهدافنا الرئيسية لـ `xv6` هي البساطة والوضوح مع توفير واجهة استدعاءات نظام بسيطة تشبه بـ UNIX. قام العديد من الأشخاص بتوسيع `xv6` بوضع استدعاءات نظام إضافية ومكتبة C بسيطة لتشغيل برامج Unix الأساسية. النوى الحديثة توفر استدعاءات نظام أكثر بكثير، وأنواعاً أكثر من خدمات النواة مقارنة بـ `xv6`. على سبيل المثال تدعم الشبكات، نظم النوافذ، خيوط مستوى المستخدم، برامج تشغيل لأجهزة كثيرة، وما إلى ذلك. تتطور النوى الحديثة باستمرار وسرعة، وتوفر ميزات كثيرة يتجاوز POSIX.

وحدت Unix الوصول لأنواع متعددة من الموارد (الملفات، الدلائل، والأجهزة) بطقم واحد من واجهات أسماء الملفات وواصفات الملفات. يمكن توسيع هذه الفكرة لأنواع أكثر من الموارد؛ مثال جيد هو `Plan 9` الذي طبق مفهوم «الموارد ملفات» على الشبكات، الرسوميات، وأكثر. ومع ذلك فإن معظم نظم التشغيل المشتقة من Unix لم تتبع هذا المسار.

كان نظام الملفات وواصفات الملفات تجريدات قوية. ورغم ذلك توجد نماذج أخرى لواجهات نظم التشغيل. Multics، أسلاف Unix، جردت تخزين الملفات بطريقة جعلته يبدو كالذاكرة، منتجة طابعاً مختلفاً جداً للواجهة. تعقيد تصميم Multics كان له أثر مباشر على مصممي Unix الذين هدفت بناؤهم لشيء أبسط.

لا توفر `xv6` مفهوماً للمستخدمين أو حماية مستخدم من آخر؛ بالترتيب لـ Unix تنفذ كافة عمليات `xv6` ك `root`.

يفحص هذا الكتاب كيف تنفذ `xv6` واجهتها الشبيهة بـ Unix، لكن الأفكار والمفاهيم تنطبق على ما هو أكثر من مجرد Unix. أي نظام تشغيل يجب عليه تقاسم العمليات بالمشاركات الزمنية على العتاد التحتي، عزل العمليات عن بعضها البعض، وتوفير آليات للتواصل الخاضع للتحكم بين العمليات. بعد دراسة `xv6` ينبغي لك القدرة على النظر في نظم تشغيل أخرى أكثر تعقيداً ومشاهدة المفاهيم التحتية لـ `xv6` في تلك النظم أيضاً.

2.6 تمارين

1.

اكتب برنامجاً يستخدم استدعاءات نظام UNIX لتبادل بايت بين عمليتين عبر زوج من الأنابيب (ping-pong)، واحد لكل اتجاه. قس أداء البرنامج في التبادلات في الثانية.

3 تنظيم نظام التشغيل

المطلب الرئيسي لنظام التشغيل هو دعم عدة أنشطة في الوقت الواحد. على سبيل المثال، يمكن استخدام استدعاءات النظام `fork` و `exec` من الفصل 2 لبدء مترجم ومحرر نصوص كعمليات. يجب على نظام التشغيل تقاسم الموارد بالمشاركات الزمنية (Time-share) مثل المعالجات والذاكرة بين هذه العمليات. يجب على نظام التشغيل أيضاً الترتيب لـ العزل (Isolation) بين العمليات. إذا احتوت إحدى العمليات على خطأ برمجي وتعطلت، فلا ينبغي أن تؤثر على العمليات غير المرتبطة بها. ومع ذلك، فإن العزل الكامل قوي جداً، نظراً لأنه ينبغي أن يكون بمقدور العمليات التفاعل قصداً؛ والأنابيب مثال على ذلك. وهكذا يجب على نظام التشغيل تلبية ثلاثة متطلبات: المضاعفة، العزل، والتفاعل.

يقدم هذا الفصل نظرة عامة عن كيفية تنظيم نظم التشغيل لتحقيق هذه المتطلبات الثلاثة. يتبين وجود طرق عديدة القيام بذلك، لكن هذا النص يركز على التصميم السائدة التي تتمركز حول النواة أحادية القالب (Monolithic Kernel)، والمستخدم بواسطة العديد من نظم تشغيل Unix. يقدم هذا الفصل أيضاً نظرة عامة عن عملية xv6، وهي وحدة العزل بـ xv6.

تنفذ xv6 على معالج RISC-V دقيق متعدد النوى (تُعنى بـ «متعدد النوى» في هذا النص المعالجات الفيزيائية المتعددة التي تتشارك الذاكرة ولكن تنفذ بالتوازي، لكل منها طقم سجلاته الخاص. يستخدم هذا النص أحياناً مصطلح متعدد المعالجات كمترادف لمتعدد النوى، على الرغم من أن متعدد المعالجات قد يشير تحديداً لحاسوب بشرائح معالجات متعددة متميزة). كثير من وظائفها منخفضة المستوى (على سبيل المثال، تنفيذ العملية) محدد بـ RISC-V. RISC-V هو معالج 64-بت، و xv6 مكتوبة بلغة C ذات طراز «LP64»، مما يعني أن `long` و المؤشرات (P) في لغة البرمجة C تكون 64-بت، ولكن `int` يكون 32-بت. يفترض هذا الكتاب أن القارئ قام ببعض البرمجة على مستوى الآلة في بنية ما، وسيقدم الأفكار المحددة بـ RISC-V عند ظهورها. الوثائق الخاصة بـ ISA لمستوى المستخدم والبنية المتميزة هي الوثائق الكاملة. يمكنك أيضاً الرجوع لكتاب «The RISC-V Reader: An Open Architecture Atlas».

المعالج في الحاسوب الكامل محاط بعناد مساند، معظمه في شكل واجهات إدخال/إخراج. xv6 مكتوبة للعتاد المساند المحاكى بواسطة خيار `qemu` المسمى «`machine virt`». يتضمن هذا ذاكرة الوصول العشوائي RAM، وذاكرة ROM حواية شفرة الإقلاع، واتصال تسلسلي للوحة مفاتيح/شاشة المستخدم، وقرصاً للتخزين.

3.1 تجريد الموارد الفيزيائية

السؤال الأول الذي قد يطرحه المرء عند مصادفة نظام تشغيل هو لماذا نحوزه أساساً؟ أي أنه يمكن تنفيذ استدعاءات النظام في الشكل القياسي كمكتبة، ترتبط بها التطبيقات. في هذه الخطة، يمكن لكل تطبيق حوز مكتبته المخصصة لاحتياجاته. يمكن للتطبيقات التفاعل مباشرة مع الموارد العتادية واستخدام تلك الموارد بأفضل طريقة للتطبيق (مثلاً، لتحقيق أداء عالٍ أو متوقع). بعض نظم التشغيل للأجهزة المدمجة أو النظم لحظية الاستجابة تُنظم بهذه الطريقة.

الجانب السلبي لنهج المكتبة هذا هو أنه إذا كان هناك أكثر من تطبيق ينفذ، فيجب أن تكون التطبيقات حسنة السلوك. على سبيل المثال، يجب على كل تطبيق التخلي دورياً عن المعالج لكي تتيح للتطبيقات الأخرى التنفيذ. مخطط المشاركة الزمنية التعاوني هذا قد يكون مقبولاً إذا كانت كافة التطبيقات تثق ببعضها البعض ولا تحوي أخطاء برمجية. المعهود أكثر هو ألا تثق التطبيقات ببعضها البعض، وتحوي أخطاء برمجية، لذا يحتاج المرء لعزل أقوى مما يوفره المخطط التعاوني.

لتحقيق عزل قوي يُعد من النافع منع التطبيقات من الوصول المباشر للموارد العتادية الحساسة، وبدلاً من ذلك تجريد الموارد في خدمات. على سبيل المثال، تتفاعل تطبيقات Unix مع التخزين فقط عبر استدعاءات نظام الملفات `open` و `read` و `write` و `close` بدلاً من قراءة وكتابة القرص مباشرة. يوفر هذا للتطبيق راحة أسماء المسارات، ويسمح لنظام التشغيل (الذي يوفر الواجهة) بإدارة القرص. حتى إن لم يكن العزل هماً، فإن البرامج التي تتفاعل قصداً (أو ترغب مجرداً في الابتعاد عن طريق بعضها البعض) يُرجح أن تجد نظام الملفات تجريداً أكثر راحة من الاستخدام المباشر للقرص.

بالمثل، تبدل Unix شفافياً المعالجات العتادية بين العمليات، حافزة ومستعادة حالة السجلات حسب الضرورة، لكي لا تحتاج التطبيقات لتكون مدركة للمشاركة الزمنية. تتيح هذه الشفافية لنظام التشغيل مشاركة المعالجات حتى وإن كانت بعض التطبيقات في حلقات تكرارية نهائية.

كمثال آخر، تستخدم عمليات Unix الاستدعاء exec لبناء صورة ذاكرتها، بدلاً من التفاعل المباشر مع الذاكرة الفيزيائية. يتيح هذا لنظام التشغيل القرار بخصوص مكان وضع العملية في الذاكرة؛ وإذا كانت الذاكرة ضيقة، فقد يخزن نظام التشغيل بعض بيانات العملية على القرص. يوفر exec أيضاً للمستخدمين راحة نظام الملفات لتخزين صور البرامج التنفيذية.

عديد أشكال التفاعل بين عمليات Unix تقع عبر واصفات الملفات. ليس فقط تجرد واصفات الملفات العديد من التفاصيل (مثلاً، أين تُخزن البيانات في الأنبوب أو الملف)، بل إنها معرفة بطريقة تبسط التفاعل. على سبيل المثال، إذا خرج تطبيق في أنبوب أو فشل، فإن النواة تنشئ تلقائياً إشارة نهاية-الملف للعملية التالية في الأنبوب.

واجهة استدعاء النظام مصممة بحذر لتوفير راحة المبرمج واحتمالية العزل القوي في الوقت نفسه. واجهة Unix ليست الطريقة الوحيدة لتجريد الموارد، لكنها أثبتت كونها طريقة جيدة.

3.2 نمط المستخدم، نمط المشرف، واستدعاءات النظام

يتطلب العزل القوي حداً صلباً بين التطبيقات ونظام التشغيل. لا ينبغي السماح للتطبيقات باضطراب تشغيل نظام التشغيل أو البرامج الأخرى، حتى وإن احتوى التطبيق على خطأ برمجي أو كان خبيثاً. لتحقيق عزل قوي، يجب على نظام التشغيل الترتيب بحيث لا تتمكن التطبيقات من تعديل (أو حتى قراءة) هياكل بيانات نظام التشغيل وتعليماته ولا تتمكن التطبيقات من الوصول لذاكرة العمليات الأخرى.

توفر المعالجات دعماً عتادياً للعزل القوي. على سبيل المثال، يمتلك RISC-V ثلاثة مستويات امتياز تقيد ما يمكن للشفرة فعله: نمط الآلة (Machine mode)، نمط المشرف (Supervisor mode)، و نمط المستخدم (User mode). التعليمات المنفذة في نمط الآلة تحوز الامتياز الكامل؛ يبدأ المعالج في نمط الآلة. نمط الآلة مخصص أساساً لإعداد الحاسوب أثناء الإقلاع. تنفذ xv6 موجزاً في نمط الآلة ثم تتغير إلى نمط المشرف.

في نمط المشرف يُسمح للمعالج بتنفيذ التعليمات المتميزة (Privileged instructions): على سبيل المثال، تفعيل وتعطيل المقاطعات، قراءة وكتابة المسجل الحائز عنوان جدول الصفحات، إلخ. إذا حاول تطبيق في نمط المستخدم تنفيذ تعليمة متميزة، فإن المعالج لا ينفذ التعليمة، بل يقع في مصيدة للشفرة الخاصة في نمط المشرف التي يمكنها إنهاء التطبيق. التطبيق يمكنه تنفيذ تعليمات نمط المستخدم فقط (مثلاً، جمع الأرقام، إلخ) ويُقال إنه ينفذ في مساحة المستخدم (User space)، بينما البرمجيات في نمط المشرف يمكنها أيضاً تنفيذ التعليمات المتميزة ويُقال إنها تنفذ في مساحة النواة (Kernel space). البرمجيات التي تنفذ في مساحة النواة (أو في نمط المشرف) تُسمى النواة (Kernel).

تتفاعل التطبيقات مع النواة عبر استدعاءات النظام مثل read. لا يُسمح للتطبيقات بالاستدعاء المباشر لدوال النواة أو الوصول لذاكرة النواة. يوفر RISC-V تعليمة ecall لاستدعاءات النظام؛ تبذل المعالج من نمط المستخدم إلى المشرف وتقفز لنقطة دخول محدودة بالنواة. بمجرد تبديل المعالج لنمط المشرف، يمكن للنواة التحقق من معاملات استدعاء النظام (مثلاً، فحص ما إذا كان العنوان الممرر لاستدعاء النظام جزءاً من ذاكرة التطبيق)، والقرار بخصوص ما إذا كان التطبيق مسموحاً له بالقيام بالعملية المطلوبة، ثم رفضها أو تنفيذها. من المهم أن تتحكم النواة بنقطة الدخول للانتقالات لنمط المشرف؛ فلو استطاع التطبيق القرار بنقطة دخول النواة، لتمكن تطبيق خبيث، على سبيل المثال، من دخول النواة عند نقطة حيث يُتجاوز الفحص للمعاملات.

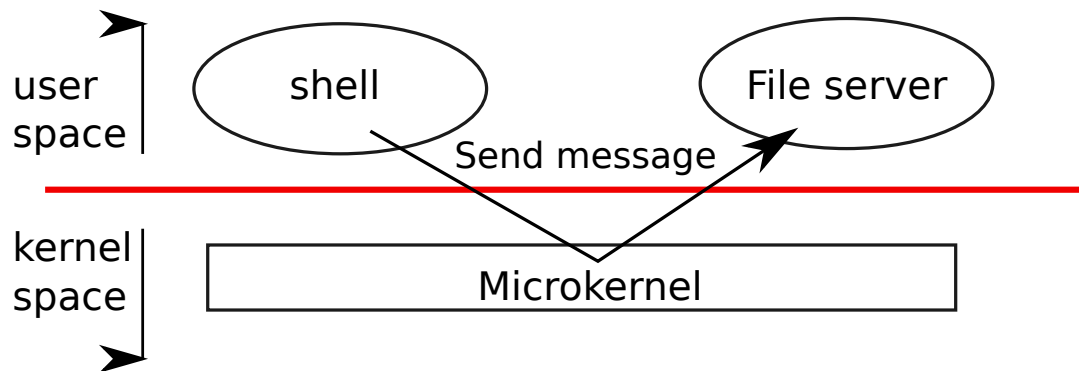
3.3 تنظيم النواة

السؤال التصميمي الرئيسي هو ما هي الأجزاء من نظام التشغيل التي ينبغي أن تنفذ في نمط المشرف. أحد الاحتمالات هو أن يستقر كامل نظام التشغيل في النواة، لكي تنفذ كافة تنفيذات استدعاءات النظام في نمط المشرف. هذا التنظيم يُسمى النواة أحادية القالب (Monolithic kernel).

في التنظيم أحادي القالب، يتكون كامل نظام التشغيل من برنامج واحد ينفذ في نمط المشرف. أحد الأسباب لجعل هذا التنظيم مريحاً هو أن مصمم نظام التشغيل لا يتعين عليه تقسيم الشفرة إلى أجزاء تتطلب وأجزاء لا تتطلب امتيازات المشرف. علاوة على ذلك، يسهل للأجزاء المختلفة من نظام التشغيل التعاون، نظراً لأنها أجزاء من برنامج واحد. على سبيل المثال، النواة أحادية القالب قد تشارك مخزن كتل القرص المؤقت مع نظام الملفات ونظام الذاكرة الافتراضية.

الجانب السلبي هو أن النوى أحادية القالب تميل لأن تنمو ضخمة ومعقدة، لكي لا يفهم أي مطور بمفرده كافة التفاعلات بين الأجزاء المختلفة من الشفرة؛ هذه وصفة للأخطاء البرمجية. الخطأ البرمجي في النواة مزعج بصفة خاصة لأنه قد يتسبب في انهيار كامل الحاسوب، أو يتسبب في تعطل العديد من التطبيقات، أو يجعل كامل الحاسوب عرضة للهجمات الأمنية.

تهدف النواة الدقيقة (Microkernel) إلى تقليل وقوع الأخطاء البرمجية في النواة. الفكرة هي وضع حد أدنى مطلق من الوظائف في النواة نفسها، لكي تنفذ شفرة قليلة في نمط المشرف، ولكي تكون النواة سهلة الفهم والتحليل للصحة. معظم نظام التشغيل ينفذ كخوادم عمليات في مستوى المستخدم. على سبيل المثال، شفرة نظام الملفات ستنفذ كخادم عملية، في نمط المستخدم بدلاً من نمط المشرف.



شكل 2: نواة دقيقة مع خادم نظام ملفات.

يوضح الشكل شكل 2 تصميم النواة الدقيقة هذا. في الشكل ينفذ نظام الملفات كخادم عملية في مستوى المستخدم. للسماح للتطبيقات بالتفاعل مع خادم الملفات، توفر النواة آلية تواصل بين العمليات لإرسال الرسائل

من عملية في نمط المستخدم لأخرى. على سبيل المثال، إذا أراد تطبيق مثل موجه الأوامر قراءة أو كتابة ملف، فإنه يرسل رسالة ل خادم الملفات وينتظر الاستجابة.

في النواة الدقيقة، تتكون واجهة النواة من بضع دوال منخفضة المستوى لبدء التطبيقات، إرسال الرسائل، الوصول لعتاد الأجهزة، إلخ. يتيح هذا التنظيم للنواة أن تكون بسيطة نسبياً، نظراً لأن معظم نظام التشغيل يستقر في خوادم مستوى المستخدم.

في العالم الحقيقي، النوى أحادية القالب والنوى الدقيقة كلاهما شائع. العديد من نوى Unix أحادية القالب. على سبيل المثال يحوز Linux نواة أحادية القالب، على الرغم من أن بعض وظائف نظام التشغيل تنفذ كخوادم في مستوى المستخدم (مثل نظام النوافذ). يوفر Linux أداءً عالياً للتطبيقات المكثفة لنظام التشغيل، جزئياً لأن النظم الفرعية للنواة يمكن ربطها بكثافة.

نظم التشغيل مثل Minix و L4 و QNX تُنظم ك نواة دقيقة مع خوادم، وشهدت انتشاراً واسعاً في الإعدادات المدمجة. متغير من L4، المسمى sel4، صغير بما يكفي لدرجة أنه جرى التحقق منه لأمان الذاكرة والخصائص الأمنية الأخرى.

هناك نقاش كبير بين مطوري نظم التشغيل بخصوص أيهما أفضل، لكن لا يوجد دليل قاطع في أحد الاتجاهين. علاوة على ذلك، فإنه يعتمد كثيراً على ما يعنيه «أفضل»: أداء أسرع، حجم شفرة أصغر، موثوقية النواة، موثوقية كامل نظام التشغيل (بما في ذلك خدمات مستوى المستخدم)، إلخ.

توجد أيضاً اعتبارات عملية قد تكون أكثر أهمية من سؤال أيهما أنسب. بعض نظم التشغيل تحوز نواة دقيقة لكنها تنفذ بعض خدمات مستوى المستخدم في مساحة النواة لأسباب تتعلق بالأداء. بعض نظم التشغيل تحوز نوى أحادية القالب لأن تلك هي الطريقة التي بدأت بها ويوجد دافع قليل للانتقال لتنظيم نواة دقيقة خالص، لأن الميزات الجديدة قد تكون أكثر أهمية من إعادة كتابة نظام التشغيل القائم ليلام تصميم النواة الدقيقة.

من منظور هذا الكتاب، تشارك النوى الدقيقة والنوى أحادية القالب العديد من الأفكار الرئيسية. تنفذ استدعاءات النظام، وتستخدم جداول الصفحات، وتتعامل مع المقاطعات، وتدعم العمليات، وتستخدم الأقفال للتحكم في التزامن، وتنفذ نظام الملفات، إلخ. يركز هذا الكتاب على هذه الأفكار الجوهرية.

تُنقذ xv6 ك نواة أحادية القالب، مثل معظم نظم تشغيل Unix. وهكذا فإن واجهة نواة xv6 تناظر واجهة نظام التشغيل، وتنفذ النواة كامل نظام التشغيل. نظراً لأن xv6 لا توفر خدمات كثيرة، فإن نواتها أصغر من بعض النوى الدقيقة، ولكن من الناحية المفهومية يُعد xv6 أحادي القالب.

3.4 الكود البرمجي: تنظيم xv6

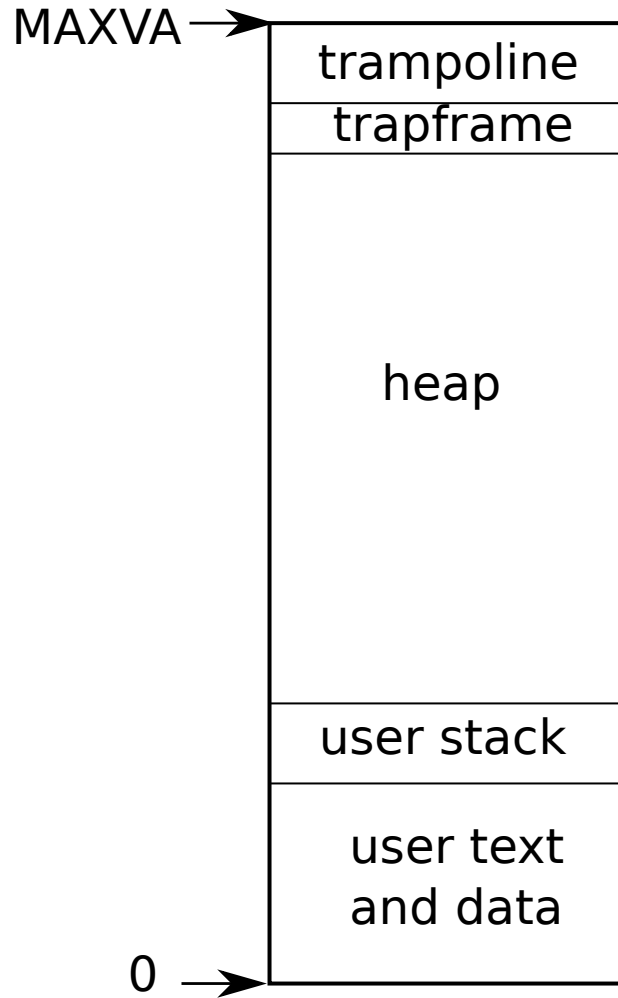
تستقر شفرة المصدر لنواة xv6 في المجلد الفرعي kernel/. توزع الملفات مقسمة في المجالات الرئيسية لمسؤولية النواة: بدء النظام (الإقلاع)، إنشاء والتحكم في العمليات، معالجة المصائد (المقاطعات واستدعاءات النظام)، تخصيص الذاكرة وتكوين العناوين الافتراضية، التحكم في الأجهزة، وإدارة نظام الملفات.

3.5 نظرة عامة على العملية

وحدة العزل بـ xv6 (كما في نظم تشغيل Unix الأخرى) هي العملية (Process). تمنع مجرد العملية عملية ما من تخريب أو التجسس على ذاكرة، معالج، واصفات ملفات، إلخ، لعملية أخرى. كما تمنع العملية من تخريب النواة نفسها، لكي لا تتمكن عملية من تقويض آليات العزل للنواة. يجب على النواة تنفيذ مجرد العملية بحذر لأن تطبيقاً يحوي خطأ برمجي أو خبيثاً قد يخدع النواة أو العتاد للقيام بشيء سيئ (مثل الالتفاف على العزل). الآليات المستخدمة بواسطة النواة لتنفيذ العمليات تتضمن علامة نمط المستخدم/المشرف، مساحات العناوين، والتجزئة الزمنية للخيوط.

للمساعدة في فرض العزل، يوفر مجرد العملية الوهم للبرنامج بأن له آليته الخاصة. توفر العملية للبرنامج ما يبدو أنه نظام ذاكرة خاص، أو مساحة عناوين (Address space)، لا يمكن للعمليات الأخرى قراءته أو كتابته. كما توفر العملية للبرنامج ما يبدو أنه معالجه الخاص لتنفيذ تعليمات البرنامج.

تستخدم xv6 جداول الصفحات (المنفذة بواسطة العتاد) لإعطاء كل عملية مساحة عناوينها الخاصة. يترجم جدول صفحات RISC-V (أو «يرسم») العنوان الافتراضي (Virtual address) (العنوان الذي تتعامل معه تعليمية RISC-V) إلى عنوان فيزيائي (Physical address) (عنوان يرسله المعالج للذاكرة الرئيسية).



شكل 3: تخطيط الذاكرة الافتراضية للعملية.

تحتفظ xv6 بجدول صفحات مستقل لكل عملية يحدد مساحة عناوين تلك العملية. كما يوضح الشكل شكل 3، تتضمن مساحة العناوين ذاكرة المستخدم للعملية بدءاً من العنوان الافتراضي صفر. تأتي التعليمات أولاً، تليها المتغيرات العامة، ثم المكديس، وأخيراً منطقة «الهيبي» (malloc) التي يمكن للعملية توسيعها حسب الحاجة. توجد عوامل تحد من الحد الأقصى لحجم مساحة عناوين العملية: المؤشرات بـ RISC-V بعرض 64 بت؛ العتاد يستخدم فقط الـ 39 بت الأدنى عند البحث في العناوين الافتراضية في جداول الصفحات؛ وتستخدم xv6 فقط 38 من تلك الـ 39 بت. وهكذا فإن العنوان الأقصى هو $2^{38} - 1$ «0x3fffffff» وهو الثابت MAXVA. في أعلى مساحة العناوين تضع xv6 صفحة المنطاد trampoline (4096 بايت) و صفحة trapframe. تستخدم xv6 هاتين صفحتي للانتقال في النواة والعودة؛ تحوي صفحة المنطاد الشفرة للانتقال من وإلى النواة، وتكون صفحة trapframe حيث تحفظ النواة سجلات المستخدم للعملية، كما يشرح الفصل الفصل 5.

تحتفظ نواة xv6 بأجزاء حالة عديدة لكل عملية، والتي تجمعها في struct proc. أهم أجزاء حالة النواة للعملية هي جدول صفحاتها، مكديس نواتها، وحالة تشغيلها. سنستخدم الترميز p->xxx للإشارة لعناصر هيكل proc؛ على سبيل المثال p->pagetable مؤشر لجدول صفحات العملية.

عند هذه النقطة يُرجى قراءة kernel/proc.h والذي يحدد struct proc. شفرة xv6 أكثر أهمية لفهمك من هذا الكتاب؛ ينبغي لك إعطاء الأولوية للشفرة واستشارة هذا الكتاب حسب الحاجة لتوضيح الشفرة. غرض بعض الشفرة قد لا يكون ظاهراً في البداية، لكن القراءة والاستكشاف في الشفرة سيساعدان. لا تتردد في استكشاف وتعديل الشفرة.

لكل عملية خيط تنفيذ (أو خيط Thread اختصاراً) يحتفظ بالحالة المطلوبة لتنفيذ العملية. في أي وقت محدد، قد يكون الخيط ينفذ على معالج، أو معلقاً (لا ينفذ، ولكن قادر على استئناف التنفيذ في المستقبل). لتبديل معالج بين العمليات، تعطل النواة الخيط المنفذ حالياً على ذلك المعالج وتحفظ حالته، وتستعيد حالة خيط عملية أخرى معلق

سابقاً. كثير من حالة الخيط (المتغيرات المحلية، عناوين العودة لدوال الاستدعاء) تُخزن على مكدسات الخيط. لكل عملية مكدسان: مكدس مستخدم ومكدس نواة (p->kstack). عندما تنفذ العملية تعليمات مستخدم، يكون مكدس مستخدمها فقط قيد الاستخدام، ومكدس نواتها فارغاً. عندما تدخل العملية النواة (لاستدعاء نظام أو مقاطعة)، تنفذ شفرة النواة على مكدس نواة العملية؛ بينما تكون العملية في النواة فإن مكدس مستخدمها يظل يحوي بيانات محفوظة، لكنه ليس قيد الاستخدام الفعال. يتناوب خيط العملية بين الاستخدام الفعال لمكدس مستخدمه ومكدس نواته. مكدس النواة منفصل (ومحمي من شفرة المستخدم) لكي تتمكن النواة من التنفيذ حتى وإن دمرت العملية مكدس مستخدمها.

يمكن لشفرة المستخدم للعملية تنفيذ استدعاء نظام عبر تنفيذ تعليمة `ecall` لـ RISC-V. تبدل هذه التعليمة إلى نمط المشرف وتغير مؤشر البرنامج لنقطة دخول محدودة بالنواة. تبدل الشفرة عند نقطة الدخول لمكدس نواة العملية وتنفذ تعليمات النواة التي تنفذ استدعاء النظام. عندما يكتمل استدعاء النظام، ترجع النواة لمساحة المستخدم عبر تنفيذ تعليمة `sret` التي تبدل لنمط المستخدم وتستأنف تنفيذ تعليمات المستخدم بعد تعليمة استدعاء النظام مباشرة. يمكن لخيط العملية أن يحظر في النواة للانتظار في الإدخال/الإخراج، ويستأنف من حيث توقف عندما يكتمل الإدخال/الإخراج.

يشير `p->state` إلى ما إذا كانت العملية مخصصة، جاهزة للتشغيل، تنفذ حالياً على معالج، تنتظر الإدخال/الإخراج، أو تخرج.

يحتوي `p->pagetable` جدول صفحات العملية، في الشكل الذي يتوقعه عتاد RISC-V. تتسبب `xv6` في جعل عتاد الصفحات يستخدم `p->pagetable` للعملية عند تنفيذ تلك العملية في مساحة المستخدم. يعمل جدول صفحات العملية أيضاً كمسجل لعناوين الصفحات الفيزيائية المخصصة لتخزين ذاكرة العملية.

باختصار، تجمع العملية فكرتي تصميم: مساحة عناوين لإعطاء العملية الوهم بذاكرتها الخاصة، وخيطاً لإعطاء العملية الوهم بمعالجها الخاص. بـ `xv6` تتكون العملية من مساحة عناوين واحدة وخيط واحد. في نظم التشغيل الحقيقية قد تحوز العملية أكثر من خيط للاستفادة من المعالجات المتعددة.

3.6 الكود البرمجي: بدء تشغيل xv6 والعملية الأولى

لجعل `xv6` أكثر ملموسية، سنخطط كيف تبدأ النواة وتنفذ العملية الأولى. ستصف الفصول التالية الآليات التي تظهر في هذه النظرة العامة بالتفصيل. يُرجى قراءة `kernel/entry.S` و `kernel/start.c` و `kernel/main.c` و `user/init.c`.

عندما يعمل حاسوب RISC-V، يهيئ نفسه وينفذ برنامج الإقلاع المخزن في ذاكرة القراءة فقط ROM. ينسخ برنامج الإقلاع نواة `xv6` في الذاكرة عند العنوان الفيزيائي `0x80000000`. السبب في وضعه للنواة عند `0x80000000` بدلاً من `0x0` هو أن النطاق `0x0:0x80000000` يحوي أجهزة الإدخال/الإخراج.

ثم يقفز برنامج الإقلاع إلى `xv6` بدءاً من `entry_`. يبدأ RISC-V مع تعطيل عتاد الصفحات: العناوين الافتراضية ترسم مباشرة لعناوين فيزيائية. تعدل التعليمات بـ `entry_` مكدساً لكي تتمكن `xv6` من تنفيذ شفرة `C`. تعلن `xv6` مساحة لهذا المكدس، `stack0` في الملف `start.c`. تحمل الشفرة بـ `entry_` مسجل مؤشر المكدس `sp` بالعنوان `stack0+4096` وهو أعلى المكدس، لأن المكدس بـ RISC-V ينظر للأسفل. الآن بعد أن حازت النواة مكدساً، تدعو `entry_` شفرة `C` بـ `start_`.

تنفذ الدالة `start` بعض الإعدادات التي يتيحها المعالج فقط في نمط الآلة، وأهمها برمجة شريحة الساعة لتوليد مقاطعات الميقاتي. ثم تستخدم `start` تعليمة `mret` لـ RISC-V للتبديل لنمط المشرف والقفز إلى `main`. تتطلب بعض الإعدادات: تضبط `start` النمط السابق للمشرف في المسجل `mstatus`، وتضبط عنوان الوجهة لـ `main` عبر كتابة عنوان `main` في المسجل `mepc`، وتعطل ترجمة العناوين الافتراضية في نمط المشرف عبر كتابة 0 في مسجل جدول الصفحات `satp`، وتفوض كافة المقاطعات والاستثناءات لنمط المشرف.

بعد أن تهيئ `main` عدة أجهزة ونظم فرعية، تنشئ العملية الأولى عبر دعوة `userinit`. كافة العمليات المنشأة حديثاً تبدأ التنفيذ في النواة في `forkret`. كحالة خاصة للعملية الأولى تدعو الدالة `kexec` لتحميل برنامج المستخدم `init/`.

بعد دعوة kexec ترجع forkret لمساحة المستخدم في العملية /init. تنشئ init ملف جهاز لوحة تحكم جديد إذا لزم الأمر ثم تفتحه كواصفات ملفات 0 و 1 و 2. ثم تبدأ موجه الأوامر على لوحة التحكم. النظام يعمل الآن.

3.7 نموذج الأمان

قد تتساءل كيف يتعامل نظام التشغيل مع الشفرة التي تحوي أخطاء برمجية أو الخبيثة. نظراً لأن مواجهة الخبث أصعب من التعامل مع الأخطاء غير المقصودة، فمن المنطقي التركيز أساساً على توفير الأمان ضد الخبث. فيما يلي نظرة رفيعة المستوى للافتراضات والأهداف الأمنية المعهودة في تصميم نظام التشغيل.

يجب على نظام التشغيل افتراض أن شفرة مستوى المستخدم للعملية ستبذل قصارى جهدها لتخريب النواة أو العمليات الأخرى. قد تحاول شفرة المستخدم فك إشارة مؤشرات خارج مساحة عناوينها المسموحة؛ وقد تحاول تنفيذ تعليمات ليست مخصصة لشفرة المستخدم؛ وقد تحاول قراءة وكتابة سجلات تحكم RISC-V؛ وقد تحاول الوصول لعتاد الأجهزة؛ وقد تمرر قيماً خبيثة لاستدعاءات النظام في محاولة لخداع النواة للانهيال أو القيام بشيء غبي.

هدف النواة هو تقييد كل عملية مستخدم لكي تتمكن فقط من الوصول لذاكرة مستخدمها الخاصة، استخدام الـ 32 مسجلاً عام الاستخدام بـ RISC-V، والتأثير على النواة والعمليات الأخرى بالطرق التي صُممت استدعاءات النظام للسماح بها. يجب على النواة منع أية أفعال أخرى. هذه متطلبات مطلقة عادة في تصميم النواة.

التوقعات لشفرة النواة نفسها مختلفة. يُفترض أن شفرة النواة مكتوبة بواسطة مبرمجين حسني النية وحذرين، خالية من الأخطاء البرمجية، ولا تحوي أي شيء خبيث. هذا الافتراض يؤثر على كيفية تحليلنا لشفرة النواة. على سبيل المثال، توجد دوال داخلية كثيرة في النواة (مثل أقفال الدوران) قد تتسبب في مشاكل خطيرة إذا استخدمتها شفرة النواة بأسلوب خاطئ. نفترض، مع ذلك، أن النواة تستخدم دوالها الخاصة بأسلوب صحيح. على مستوى العتاد يُفترض أن معالج RAM، RISC-V، القرص، إلخ يعملون كما مُعلن في الوثائق، بدون أخطاء عتادية.

الحياة الحقيقية ليست بسيطة. من الصعب منع برامج المستخدم المساواة من دعوة استدعاءات النظام بأسلوب يجعل النظام غير قابل للاستخدام عبر استهلاك الموارد المحمية بالنواة: المساحة القرصية، وقت المعالج، خانات جدول العمليات، إلخ. من المستحيل عادة كتابة شفرة نواة 100% خالية من الأخطاء أو تصميم عتاد خال من الأخطاء؛ وإذا كان كتاب الشفرات الخبيثة مدركين لأخطاء النواة أو العتاد، فسيستغلونها. حتى في نوى ناضجة وشائعة الاستخدام مثل Linux يكتشف الناس غالباً ثغرات غير معروفة سابقاً. أخيراً، التمييز بين شفرة المستخدم والنواة يكون غير واضح أحياناً؛ بعض عمليات مستوى المستخدم المتميزة قد توفر خدمات أساسية وتكون فعلياً جزءاً من نظام التشغيل، وفي بعض نظم التشغيل يمكن لشفرة المستخدم المتميزة إدخال شفرة جديدة في النواة (كما مع وحدات النواة القابلة للتحميل بـ Linux و eBPF).

كدفاع جزئي ضد أخطاء النواة البرمجية، تتضمن شفرة xv6 فحوصات لعدم الاتساق والأخطاء غير القابلة للتعافي، وستنفذ الهلع «panic» استجابة لذلك، عبر دعوة panic(). تطبع هذه الدالة رسالة خطأ وتوقف النظام. الهلع ليس مرغوباً، ولكنه أفضل من الاستمرار في التنفيذ. ينتج الهلع عادة عن خطأ برمجي في النواة يتسبب في جعل بيانات النواة غير صحيحة أو يجعل النواة تنفذ فعلاً غير صالح مثل الإشارة لذاكرة غير موجودة؛ في مثل هذه الحالة يكون من الآمن إيقاف التنفيذ بـ panic() بدلاً من محاولة الاستمرار في حالة غير متنسقة. مطور النواة سيتفاعل مع الهلع عبر العمل للتعرف على الخطأ البرمجي التعتي وإصلاحه.

3.8 العالم الحقيقي

اعتمدت معظم نظم التشغيل مفهوم العملية، ومعظم العمليات تبدو شبيهة بـ xv6. نظم التشغيل الحديثة تدعم خيوطاً متعددة في العملية الواحدة لتتيح لعملية واحدة الاستفادة من المعالجات المتعددة. دعم خيوط متعددة في العملية يتضمن آليات كثيرة لا تحوزها xv6 تتضمن غالباً تغييرات في الواجهة (مثل clone بـ Linux وهي متغير من fork) للتحكم في أية جوانب من العملية تشاركها الخيوط.

3.9 تمارين

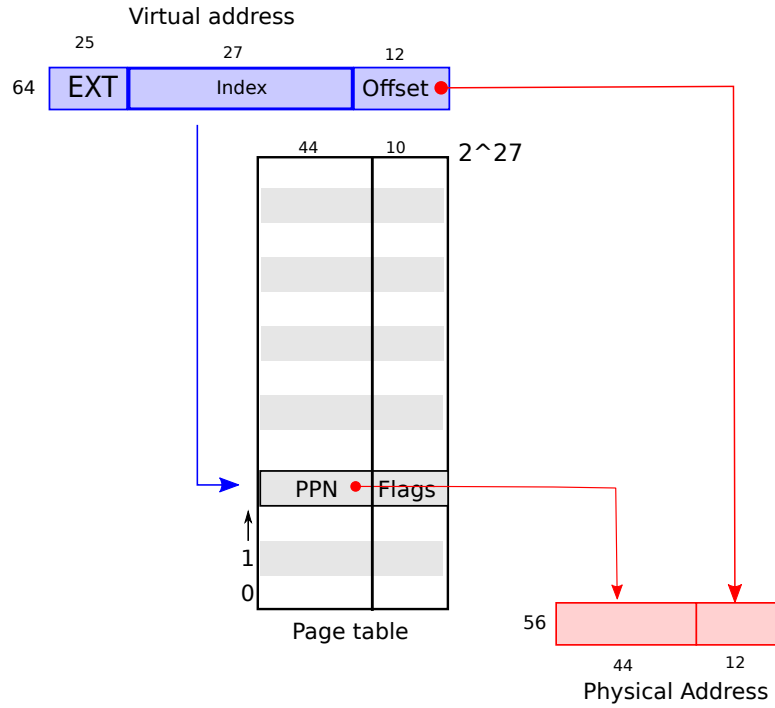
1. أضف استدعاء نظام لـ xv6 يرجع مقدار الذاكرة الحرة المتاحة.

4 جداول الصفحات

جداول الصفحات هي الآلية الأكثر شعبية التي يتوفر عبرها نظام التشغيل لكل عملية مساحة عناوينها الخاصة والذاكرة. تحدد جداول الصفحات ما تعنيه عناوين الذاكرة، وأية أجزاء من الذاكرة الفيزيائية يمكن الوصول إليها. تتيح لـ xv6 عزل مساحات عناوين العمليات المختلفة ومضاعفة تقاسمها على ذاكرة فيزيائية واحدة. توفر جداول الصفحات مستوى من عدم المباشرة يتيح لنظم التشغيل تنفيذ العديد من الحيل النافعة. تنفذ xv6 بضعة حيل: رسم نفس الذاكرة (صفحة المنطاد) في عدة مساحات عناوين، حماية مكدسات النواة والمستخدم بصفحة حراسة غير مرسومة، وتخصيص ذاكرة هيب المستخدم كسولاً. يشرح بقية هذا الفصل جداول الصفحات التي يوفرها عتاد RISC-V وكيف تستخدمها xv6.

4.1 عتاد الصفحات

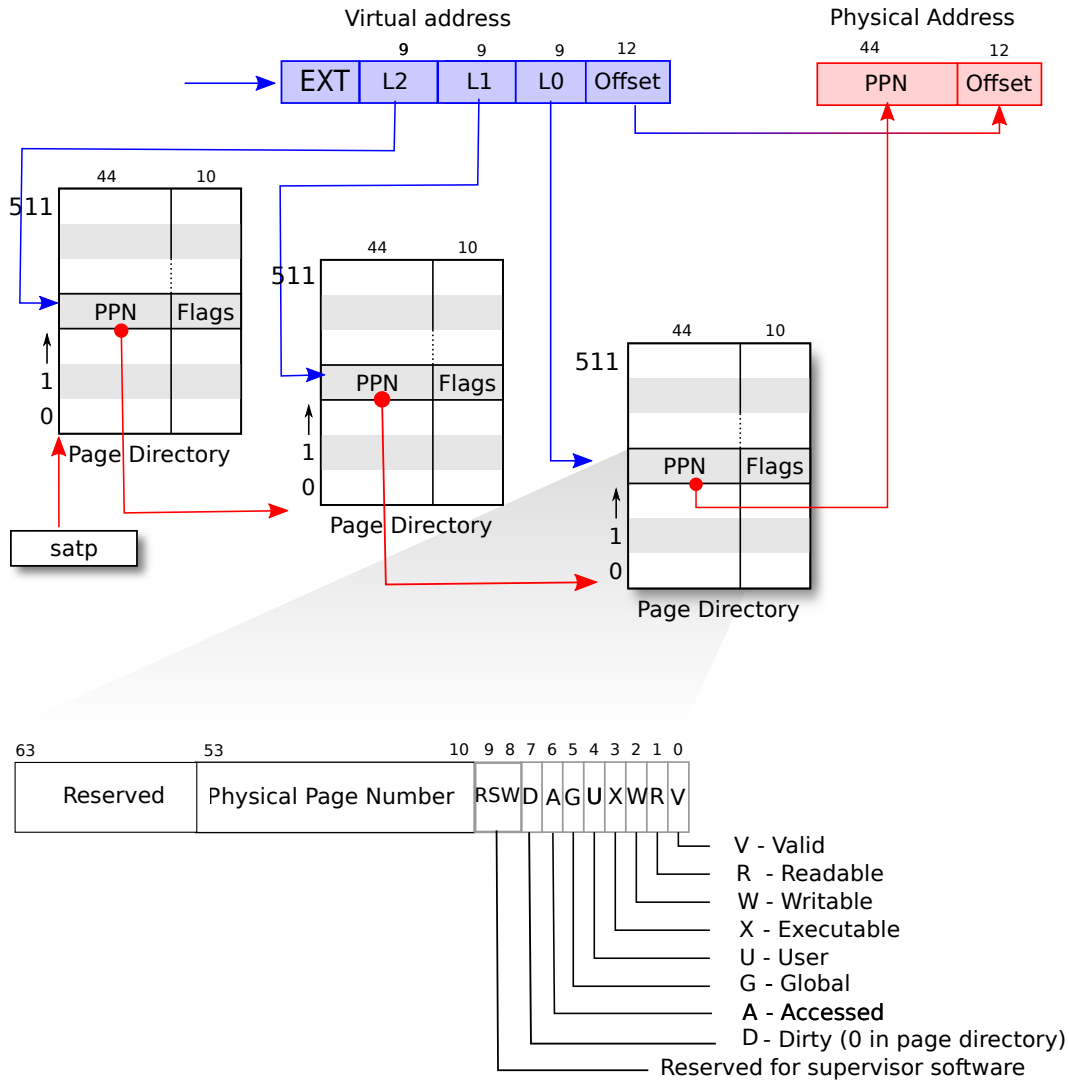
لتذكيرك، تتعامل تعليمات RISC-V (سواء للمستخدم أو النواة) مع العناوين الافتراضية. ذاكرة الذاكرة العشوائية للآلة RAM، أو الذاكرة الفيزيائية، تُفهرس بعناوين فيزيائية. يتصل عتاد جداول الصفحات بـ RISC-V بين هذين النوعين من العناوين عبر رسم كل عنوان افتراضي لعنوان فيزيائي.



شكل 4: منظور تجريدي لجداول صفحات مسطح يترجم العناوين الافتراضية إلى عناوين فيزيائية.

تستخدم xv6 نمط Sv39 لـ RISC-V، مما يعني أن البتات الـ 38 الأدنى فقط من العنوان الافتراضي الـ 64-بت تُستخدم؛ والبتات الـ 25 الأعلى لا تُستخدم. في تكوين Sv39، يكون جدول صفحات RISC-V منطقياً مصفوفة من 2^{27} (134,217,728) مدخل جدول صفحات (PTE). يحوي كل مدخل PTE رقم صفحة فيزيائية (PPN) بعرض 44 بت وبعض العلامات. يترجم عناود الصفحات الافتراضي باستخدام البتات الـ 27 الأعلى من الـ 39 بت للفهرسة في جدول الصفحات للعثور على PTE، وصنع عنوان فيزيائي بعرض 56 بت تأتي بتاته الـ 44 الأعلى من PPN في PTE وتُنسخ بتاته الـ 12 الأدنى من العنوان الافتراضي الأصلي. يوضح الشكل شكل 4 هذه العملية مع

منظور منطقي لجدول الصفحات كمصفوفة بسيطة من PTEs (جدول صفحات RISC-V في الواقع هو شجرة؛ انظر الشكل شكل 5 لقصة أوفى). يعطي جدول الصفحات نظام التشغيل التحكم بتراجم العناوين الافتراضية-إلى-فيزيائية عند درجة محاذاة كتل بحجم 4096 (2^{12}) بايت. تُسمى مثل هذه الكتلة صفحة (Page).



شكل 5: تفاصيل ترجمة العناوين في RISC-V.

يتيح تصميم RISC-V مساحة لتوسعة كل من العناوين الافتراضية والفيزيائية. إذا لزمتم مساحة عناوين افتراضية أكثر، تدعم RISC-V النمط Sv48 بعناوين افتراضية بعرض 48 بت. العناوين الفيزيائية تحوز أيضاً مساحة للنمو:

توجد مساحة في طراز PTE لنمو رقم الصفحة الفيزيائية بمقدار 10 بتات أخرى. اختار مصمم RISC-V أحجام العناوين بناءً على التنبؤات التقنية. ^{48} 2 بايت هي 262,144 غيغابايت، مساحة عناوين افتراضية للمستخدم أكبر بكثير مما يُرجح أن يستخدمه أي تطبيق اليوم. ^{56} 2 بايت من مساحة العناوين الفيزيائية هي 65,536 تيرابايت، ذاكرة RAM أكثر بكثير مما يمكن لأي حاسوب حيازتها حالياً.

كما يوضح الشكل الشكل 5، يُخزن جدول صفحات المعالج بـ RISC-V في الذاكرة الفيزيائية ك شجرة ثلاثية المستويات. جذر الشجرة هو صفحة جدول صفحات بحجم 4096 بايت تحوي 512 PTE، والتي تحوي العناوين الفيزيائية لصفحات جداول الصفحات في المستوى التالي من الشجرة. كل واحدة من تلك الصفحات تحوي 512 PTE للمستوى النهائي في الشجرة. يستخدم عتاد الصفحات البتات الـ 9 الأعلى من الـ 27 بت لاختيار PTE في صفحة جدول صفحات الجذر، والبتات الـ 9 الوسطى لاختيار PTE في صفحة جدول صفحات في المستوى التالي من الشجرة، والبتات الـ 9 الأدنى لاختيار الـ PTE النهائي. (بـ RISC-V Sv48 يحوز جدول الصفحات أربعة مستويات، والبتات من 39 حتى 47 من العنوان الافتراضي تفهرس في المستوى الأعلى).

إذا كان أي من الـ PTEs الثلاثة المطلوبة لترجمة عنوان ما غير موجود، يولد عتاد الصفحات استثناء خطأ صفحة (Page-fault exception)، تاركاً معالجة خطأ الصفحة للنواة (انظر الفصلين الفصل 5 و الفصل 6).

التركيب ثلاثي المستويات بالشكل الشكل 5 يتيح طريقة كفوءة ذاكراً لتسجيل الـ PTEs، مقارنة بالتصميم أحادي المستوى بالشكل الشكل 4. في الحالة المعهودة التي تكون فيها نطاقات واسعة من العناوين الافتراضية لا تحوز رسومات، يمكن للتركيب ثلاثي المستويات الحذف لأدلة صفحات كاملة. على سبيل المثال لو كان تطبيق ما يستخدم فقط بضعة صفحات تبدأ عند العنوان صفر، فإن المدخلات من 1 حتى 511 في دليل صفحات المستوى الأعلى تكون غير صالحة، ولا يتعين على النواة تخصيص صفحات لأدلة الصفحات الوسيطة الـ 511 تلك. علاوة على ذلك لا يتعين على النواة أيضاً تخصيص صفحات لأدلة صفحات المستوى الأدنى لأدلة الصفحات الوسيطة الـ 511 تلك. لذا في هذا المثال يفر التصميم ثلاثي المستويات 511 صفحة لأدلة الصفحات الوسيطة و 512 × 511 صفحة لأدلة صفحات المستوى الأدنى.

على الرغم من أن المعالج يقطع التركيب ثلاثي المستويات في العتاد كجزء من تنفيذ تعليمة تحميل أو تخزين، إلا أن جانباً سلبياً محتملاً لثلاثة مستويات هو أنه يتعين على المعالج تحميل ثلاثة PTEs من الذاكرة لتنفيذ ترجمة العنوان الافتراضي في تعليمة التحميل/التخزين لعنوان فيزيائي. لتجنب كلفة تحميل الـ PTEs من الذاكرة الفيزيائية، يخزن المعالج بـ RISC-V مدخلات جدول الصفحات مؤقتاً في مخزن الترجمة المؤقت (Translation Lookaside Buffer, TLB).

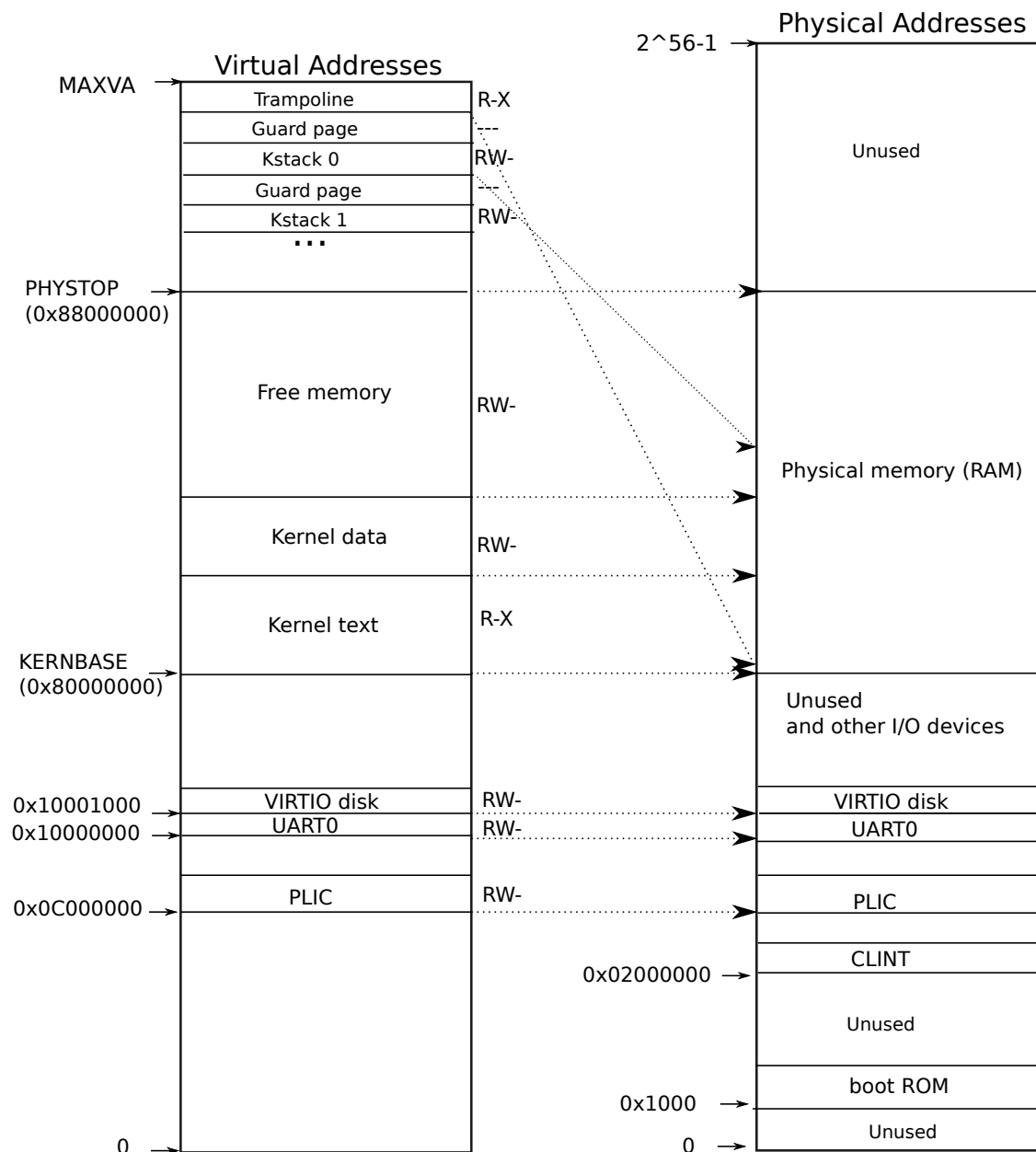
تحتوي كل PTE بتات علامات تخبر عتاد الصفحات بكيفية السماح باستخدام العنوان الافتراضي المرتبط. PTE_V تشير لما إذا كانت الـ PTE موجودة؛ إذا لم تكن مضبوطة، فإن المرجع للصفحة يتسبب في خطأ صفحة (أي غير مسموح به). PTE_R يتحكم بكون التعليمات مسموحاً لها بالقراءة للصفحة. PTE_W يتحكم بكون التعليمات مسموحاً لها بالكتابة للصفحة. PTE_X يتحكم بكون المعالج قد يفسر محتوى الصفحة كتعليمات وينفذها. PTE_U يتحكم بكون التعليمات في نمط المستخدم مسموحاً لها بالوصول للصفحة؛ إذا لم تكن PTE_U مضبوطة، يمكن استخدام الـ PTE فقط في نمط المشرف. يوضح الشكل الشكل 5 أين تجلس بتات العلامات في PTE. العلامات وكافة الهياكل العنانية الأخرى المتعلقة بالصفحات معرفة في kernel/riscv.h.

لإخبار المعالج باستخدام جدول صفحات، يجب على النواة كتابة العنوان الفيزيائي لصفحة جدول صفحات الجذر في المسجل satp. سيترجم المعالج كافة العناوين المولدة بواسطة التعليمات اللاحقة باستخدام جدول الصفحات المشار إليه بـ satp الخاص به. لكل معالج satp الخاص به لكي تتمكن المعالجات المختلفة من تشغيل عمليات مختلفة، كل منها بمساحة عناوين خاصة موصفة بواسطة جدول صفحاتها الخاص.

من منظور النواة، يُعد جدول الصفحات بيانات مخزنة في الذاكرة، وتنشئ النواة وتعديل جداول الصفحات باستخدام شفرة شبيهة كثيراً بما قد تشاهده لأي هيكل بيانات شجري.

بضع ملاحظات عن المصطلحات المستخدمة في هذا الكتاب. الذاكرة الفيزيائية تشير لخلايا التخزين في RAM. بايت الذاكرة الفيزيائية يحوز عنواناً يُسمى عنواناً فيزيائياً. التعليمات التي تفك إشارات العناوين (مثل التحميل، التخزين، القفزات، واستدعاءات الدوال) تستخدم فقط عناوين افتراضية، والتي يترجمها عتاد الصفحات لعناوين فيزيائية، ثم يرسلها لعتاد RAM لقراءة أو كتابة التخزين. مساحة العناوين هي طقم العناوين الافتراضية الصالحة

في جدول صفحات محدد؛ كل عملية بـ xv6 تحوز مساحة عناوين مستخدم منفصلة، ونواة xv6 تحوز مساحة عناوينها الخاصة أيضاً. ذاكرة المستخدم تشير لمساحة عناوين المستخدم للعملية بالإضافة لذاكرة الفيزيائية التي يتيح جدول الصفحات للعملية الوصول إليها. الذاكرة الافتراضية تشير للأفكار والتقنيات المرتبطة بإدارة جداول الصفحات واستخدامها لتحقيق أهداف مثل العزل.



شكل 6: على اليسار، مساحة عناوين النواة الافتراضية بـ xv6. على اليمين، مساحة عناوين RISC-V الفيزيائية التي تتوقع xv6 مشاهدتها.

4.2 مساحة عناوين النواة

عندما تبدأ، تنشئ xv6 جدول صفحات واحداً يصف مساحة عناوين النواة. تضبط النواة تخطيط مساحة عناوينها لإعطاء نفسها وصولاً لذاكرة الفيزيائية وموارد عتادية متنوعة عند عناوين افتراضية متوقعة. يوضح الشكل شكل 6 كيف يرسم هذا التخطيط عناوين النواة الافتراضية لعناوين فيزيائية. يعلن الملف `kernel/memlayout.h` الثوابت لتخطيط ذاكرة النواة بـ `xv6`.

يحاكي QEMU حاسوباً يتضمن RAM (ذاكرة فيزيائية) تبدأ عند العنوان الفيزيائي 0x80000000 وتستمر حتى 0x88000000 على الأقل، والتي تسميها xv6 بـ PHYSTOP. تتضمن محاكاة QEMU أيضاً أجهزة إدخال/إخراج مثل واجهة

القرص. تكشف QEMU واجهات الأجهزة للبرمجيات ك سجلات تحكم مربوطة بالذاكرة تجلس أسفل 0x80000000 في مساحة العناوين الفيزيائية. يمكن للنواة التفاعل مع الأجهزة عبر قراءة/كتابة هذه العناوين الفيزيائية الخاصة؛ وتتواصل مثل هذه القراءات والكتابات مع عتاد الجهاز بدلاً من RAM. يشرح الفصل الفصل 5 كيف تتفاعل xv6 مع الأجهزة.

ترسم النواة كافة RAM الفيزيائية وسجلات الأجهزة عند عناوين افتراضية مساوية للعناوين الفيزيائية. هذا يُسمى الرسم المباشر (Direct mapping)، ويتيح للنواة قراءة أو كتابة العنوان الفيزيائي x بمجرد التحميل أو التخزين للعنوان الافتراضي x . شفرة النواة نفسها تستقر عند $\text{KERNBASE} = 0x80000000$ في كل من مساحة العناوين الافتراضية والذاكرة الفيزيائية. عندما تخصص kfork ذاكرة مستخدم للعملية الابن، يرجع المخصص العنوان الفيزيائي لتلك الذاكرة؛ وتستخدم kfork ذلك العنوان مباشرة ك عنوان افتراضي عندما تنسخ ذاكرة مستخدم الأب للابن.

توجد بضعة عناوين افتراضية للنواة ليست مرسومة مباشرة:

- صفحة المنطاد. تُرسم في أعلى مساحة العناوين الافتراضية؛ جداول صفحات المستخدم تحوز نفس هذا الرسم. يناقش الفصل الفصل 5 دور صفحة المنطاد، لكننا نرى هنا حالة استخدام ممتعة لجداول الصفحات؛ صفحة فيزيائية (تحتفظ بشفرة المنطاد) تُرسم مرتين في مساحة العناوين الافتراضية للنواة: مرة في أعلى مساحة العناوين الافتراضية ومرة مع رسم مباشر.
- صفحات مكدهس النواة. لكل عملية مكدهس نواتها الخاص، والذي يُرسم عند عنوان افتراضي عال في النواة لكي تترك xv6 أسفله صفحة حراسة (Guard page) غير مرسومة. الـ PTE لصفحة الحراسة غير صالح (أي PTE_V ليس مضبوطاً)، لكي يتسبب طغح مكدهس النواة في خطأ صفحة وتنفيذ النواة الهلع panic . بدون صفحة حراسة كان مكدهس طافح سيكتب فوق ذاكرة نواة أخرى، منتجاً تشغيلاً غير صحيح. انهيار الهلع أفضل.

بينما تستخدم النواة مكدهساتها عبر رسومات الذاكرة العالية، فإن كل منها قابلة للوصول أيضاً للنواة عبر عنوان مرسوم مباشرة. تصميم بديل كان قد يحوز مجرد الرسم المباشر، ويستخدم المكدهسات عند العنوان المباشر الرسم. في ذلك الترتيب، مع ذلك، فإن توفير صفحات حراسة كان يتضمن إلغاء رسم عناوين افتراضية كانت ستشير لولا ذلك لذاكرة فيزيائية، والتي كان يصعب استخدامها عندئذ.

ترسم النواة صفحات المنطاد ونص النواة مع الأدونات PTE_R و PTE_X ولكن ليس PTE_W . ترسم النواة الصفحات الأخرى مع الأدونات PTE_R و PTE_W ولكن ليس PTE_X . الرسومات لصفحات الحراسة تكون غير صالحة. الغرض من هذه الأدونات المقيدة هو المساعدة في التقاط أخطاء النواة البرمجية التي تصل للصفحات بطرق غير متوقعة، على سبيل المثال لو حاولت شفرة النواة بالصدفة الكتابة فوق تعليمات النواة.

تنشئ النواة جدول صفحات نواة واحداً، تُستخدم بواسطة كافة المعالجات عندما تنفذ في النواة. لا تعدل xv6 جدول صفحات النواة بعد إنشائه أولاً.

4.3 الكود البرمجي: إنشاء مساحة العناوين

يُرجى قراءة `kernel/vm.c` حتى نهاية `mappages` () قبل المتابعة.

معظم شفرة xv6 للتعامل مع مساحات العناوين وجداول الصفحات تستقر في `vm.c`. هيكل البيانات المحوري هو `pagetable_t` والذي هو بفعالية مؤشر لصفحة جدول صفحات جذر بـ RISC-V؛ و `pagetable_t` قد يكون إما جدول صفحات النواة، أو أحد جداول الصفحات الخاصة بكل عملية. الدوال المحورية هي `walk` والتي تجد الـ PTE لعنوان افتراضي، و `mappages` التي تثبت PTES لرسومات جديدة. الدوال البائدة بـ `kvm` تتعامل مع جدول صفحات النواة؛ الدوال البائدة بـ `uvm` تتعامل مع جدول صفحات المستخدم؛ الدوال الأخرى تُستخدم لكيهما. تنسخ `copyin` و `copyout` البيانات من وإلى عناوين المستخدم الافتراضية الموفرة ك معاملات استدعاءات نظام؛ وهي في `vm.c` لأنها تحتاج للترجمة الصريحة لتلك العناوين من أجل العثور على الذاكرة الفيزيائية المناظرة.

ميكراً في تسلسل الإقلاع، تدعو `main` الدالة `kvm_init` لإنشاء جدول صفحات النواة باستخدام `kvm_make`. يقع هذا الاستدعاء قبل أن تفعل xv6 تحويل الصفحات على RISC-V، لذا العناوين تشير مباشرة لذاكرة فيزيائية. تخصص `kvm_make` أولاً صفحة ذاكرة فيزيائية لاحتجاز صفحة جدول صفحات الجذر. ثم تدعو `kvmmap` لتثبيت التراجع التي تحتاجها النواة. تتضمن التراجع تعليمات النواة وبياناتها، الذاكرة الفيزيائية حتى `PHYSTOP`، ونطاقات الذاكرة التي

تكون أجهزة بفعالية. تخصص `proc_mapstacks` مكدس نواة لكل عملية. تدعو `kvmmap` لرسم كل مكدس عند العنوان الافتراضي المولد بـ `KSTACK` والذي يترك مساحة لصفحات حراسة المكس غير الصالحة.

تدعو `kvmmap` الدالة `mappages` والتي تثبت رسومات في جدول صفحات لنطاق من العناوين الافتراضية لنطاق مناظر من العناوين الفيزيائية. تفعل هذا بصفة مستقلة لكل عنوان افتراضي في النطاق، عند فترات صفحات. لكل عنوان افتراضي سيُرسَم تدعو `mappages` الدالة `walk` للعثور على عنوان الـ PTE لذلك العنوان. ثم تهيئ الـ PTE لاحتجاز رقم الصفحة الفيزيائية المناظر، الأدونات المرغوبة (`PTE_X`, `PTE_W`, و/أو `PTE_R`)، و `PTE_V` لتعليم الـ PTE ك صالح. تحاكي `walk` عتاد الصفحات بـ RISC-V بينما تبحث عن الـ PTE لعنوان افتراضي (انظر الشكل شكل 5). تنزل `walk` شجرة جدول الصفحات مستوياً في الوقت الواحد، مستخدمة الـ 9 بتات للعنوان الافتراضي لكل مستوى للفهرسة في صفحة دليل الصفحات المناظرة. في كل مستوى تجد إما الـ PTE لصفحة دليل صفحات المستوى التالي، أو الـ PTE للصفحة النهائية. إذا كان PTE في صفحة دليل صفحات مستوى أول أو ثان غير صالح، فإن صفحة الدليل المطلوبة لم تُخصص بعد؛ إذا كان المعامل `alloc` مضبوطاً، تخصص `walk` صفحة جدول صفحات جديدة وتضع عنوانها الفيزيائي في الـ PTE. ترجع عنوان الـ PTE في الطبقة الأدنى في الشجرة.

تعتمد الشفرة أعلاه على كون الذاكرة الفيزيائية مرسومة مباشرة في مساحة عناوين النواة الافتراضية. على سبيل المثال بينما تنزل `walk` مستويات جدول الصفحات، تسحب العنوان (الفيزيائي) لجدول صفحات المستوى الأدنى التالي من PTE، ثم تستخدم ذلك العنوان ك عنوان افتراضي لجلب الـ PTE في المستوى الأدنى التالي.

على كل معالج، تدعو `main` الدالة `kvmminithart` لتثبيت جدول صفحات النواة، واضعة العنوان الفيزيائي لصفحة جدول صفحات الجذر في المسجل `satp` للمعالج. بعد هذا يترجم المعالج العناوين باستخدام جدول صفحات النواة. تكتمل النواة التنفيذ بأسلوب صحيح لأن جدول صفحات النواة مرسوم مباشرة، لكي تشير العناوين لنفس المواقع في RAM قبل وبعد هذا التغيير.

يخزن كل معالج بـ RISC-V مدخلات جدول الصفحات مؤقتاً في مخزن الترجمة المؤقت (Translation Lookaside Buffer، TLB)، وعندما تغير xv6 جدول صفحات يجب أن تخبر المعالج بإلغاء صلاحية مدخلات TLB المخزنة مؤقتاً المناظرة. لو لم تفعل، لكان عند نقطة ما لاحقاً قد يستخدم TLB رسماً مخزناً مؤقتاً قديماً، إشارياً لصفحة فيزيائية حُصصت في هذه الأثناء لعملية أخرى، ونتيجة لذلك قد تتمكن عملية من الكتابة العشوائية على ذاكرة عملية أخرى. تحوز RISC-V تعليمة `sfence.vma` لتفريغ الـ TLB للمعالج الحالي. تنفذ xv6 التعليمة `sfence.vma` في `kvmminithart` بعد إعادة تحميل المسجل `satp` وفي شفرة المظلة لـ `uservec` و `userret`.

من الضروري أيضاً إصدار `sfence.vma` قبل تغيير `satp` من أجل الانتظار لإتمام كافة التحميلات والتخزينات الجارية. هذا الانتظار يضمن أن التحديثات السابقة لجدول الصفحات قد اكتملت، ويضمن أن التحميلات والتخزينات السابقة تستخدم جدول الصفحات القديم وليس الجديد.

4.4 تخصيص الذاكرة الفيزيائية

يجب على النواة تخصيص وتفرغ الذاكرة الفيزيائية في وقت التنفيذ لجدول الصفحات، ذاكرة المستخدم، مكدسات النواة، وحواشي الأنابيب.

تستخدم xv6 الذاكرة الفيزيائية بين نهاية النواة و `PHYSTOP` للتخصيص في وقت التنفيذ. تخصص وتفرغ صفحات كاملة بحجم 4096 بايت في الوقت الواحد. تتبع أية الصفحات حرة عبر لضم قائمة ترابطية في الصفحات نفسها. يتكون التخصيص من إزالة صفحة من القائمة الترابطية؛ ويتكون التفرغ من إضافة الصفحة المحررة للقائمة.

يُرجى قراءة `kernel/kalloc.c`.

4.5 الكود البرمجي: مخصص الذاكرة الفيزيائية

يستقر المخصص في `kalloc.c`. هيكل بيانات المخصص هو قائمة حرة (Free list) لصفحات الذاكرة الفيزيائية المتاحة للتخصيص. مؤشر «التالي» لكل صفحة حرة يستقر في `struct run`. يخزن المخصص هيكل `run` لكل صفحة حرة في الصفحة الحرة نفسها، نظراً لأنه لا يوجد شيء آخر مخزن هناك بينما الصفحة حرة. القائمة الحرة محمية بقفل دوران. تُغلف القائمة والقفل في هيكل لجعل الأمور واضحة بأن القفل يحمي الحقول في الهيكل. حالياً تجاهل القفل والاستدعاءات لـ `acquire` و `release`؛ سيفحص الفصل الفصل 8 القفل بتفصيل.

تدعو الدالة main الدالة kinit لهيئة المخصص. تهيئ kinit القائمة الحرة لاحتجاز كل صفحة من RAM الفيزيائية بين نهاية النواة و PHYSTOP. ينبغي لـ xv6 القرار بخصوص مقدار الذاكرة الفيزيائية المتاحة عبر تحليل معلومات التكوين الموفرة بواسطة العتاد. بدلاً من ذلك تفترض xv6 أن الآلة تحوز 128 ميغابايت من RAM. تدعو kinit الدالة freerange لإضافة الذاكرة للقائمة الحرة عبر استدعاءات لكل صفحة لـ kfree. يمكن لـ PTE الإشارة فقط لعنوان فيزيائي محاذ على حد 4096 بايت (مضاعف لـ 4096)، لذا تستخدم freerange الثابت PGROUNDUP لضمان أنها تفرغ فقط عناوين فيزيائية محاذة. يبدأ المخصص بغير ذاكرة؛ وهذه الاستدعاءات لـ kfree تعطيه بعض الذاكرة لإدارتها. يعامل المخصص أحياناً العناوين ك أعداد صحيحة لتنفيذ الحساب عليها (مثلاً، تصفح كافة الصفحات بـ freerange)، ويستخدم أحياناً العناوين ك مؤشرات لقراءة وكتابة الذاكرة (مثلاً، التعامل مع هيكل run المخزن في كل صفحة)؛ هذا الاستخدام المزدوج للعناوين هو السبب الرئيسي بكون شفرة المخصص ملأى بتحويلات الأنواع بـ C.

تبدأ الدالة kfree بضبط كل بايت في الذاكرة الجاري تحريرها للقيمة 1. سيتسبب هذا في جعل الشفرة التي تستخدم الذاكرة بعد تحريرها (تستخدم «مراجع معلقة») تقرأ قمامة بدلاً من المحتويات الصالحة القديمة؛ أملاً في جعل مثل هذه الشفرة تتعطل أسرع. ثم تلصق kfree الصفحة في مقدمة القائمة الحرة: تحول pa لمؤشر لـ struct run وتسجل البداية القديمة للقائمة الحرة في r->next وتضبط القائمة الحرة مساوية لـ r. تزيل kalloc وترجع العنصر الأول في القائمة الحرة.

4.6 مساحة عناوين العملية

لكل عملية جدول صفحاتها الخاص، وعندما تبدل xv6 بين العمليات، فإنها تغير أيضاً جداول الصفحات. يوضح الشكل شكل 7 مساحة عناوين عملية بتفصيل أكثر من الشكل شكل 3. تبدأ مساحة عناوين مستخدم العملية عند الصفر وتنتهي ميدئياً عند MAXVA (0x400000000000)، على الرغم من أنه في الممارسة فإن جزءاً صغيراً فقط من هذا يكون مرسومة لذاكرة فيزيائية.

تتكون مساحة عناوين العملية من صفحات تحوي نص البرنامج (والتي ترسمها xv6 بالأذونات PTE_R, PTE_X, و PTE_U)، صفحات تحوي بيانات البرنامج المهيأة أولاً، صفحة للمكدس، وصفحات للهيبي. ترسم xv6 البيانات، المكدس، والهيبي بالأذونات PTE_R, PTE_W, و PTE_U.

استخدام الأذونات في مساحة عناوين مستخدم تقنية شائعة لتقوية عملية مستخدم. لو كان النص مرسوماً بـ PTE_W لتمكنت عملية بالصدفة من تعديل برنامجها الخاص؛ على سبيل المثال خطأ برمجي قد يتسبب في كتابة البرنامج لمؤشر مفرغ، معدلاً التعليمات عند العنوان 0، ثم يستمر في التشغيل متنسبباً في فوضى أكثر. لالتقاط مثل هذه الأخطاء فورياً ترسم xv6 النص بدون PTE_W؛ إذا حاول برنامج بالصدفة التخزين للعنوان 0 يرفض العتاد تنفيذ التخزين وبولد خطأ صفحة (انظر الفصل الفصل 5). تقتل النواة عندئذ العملية وتطبع رسالة إعلامية لكي يتمكن المطور من تتبع المشكلة.

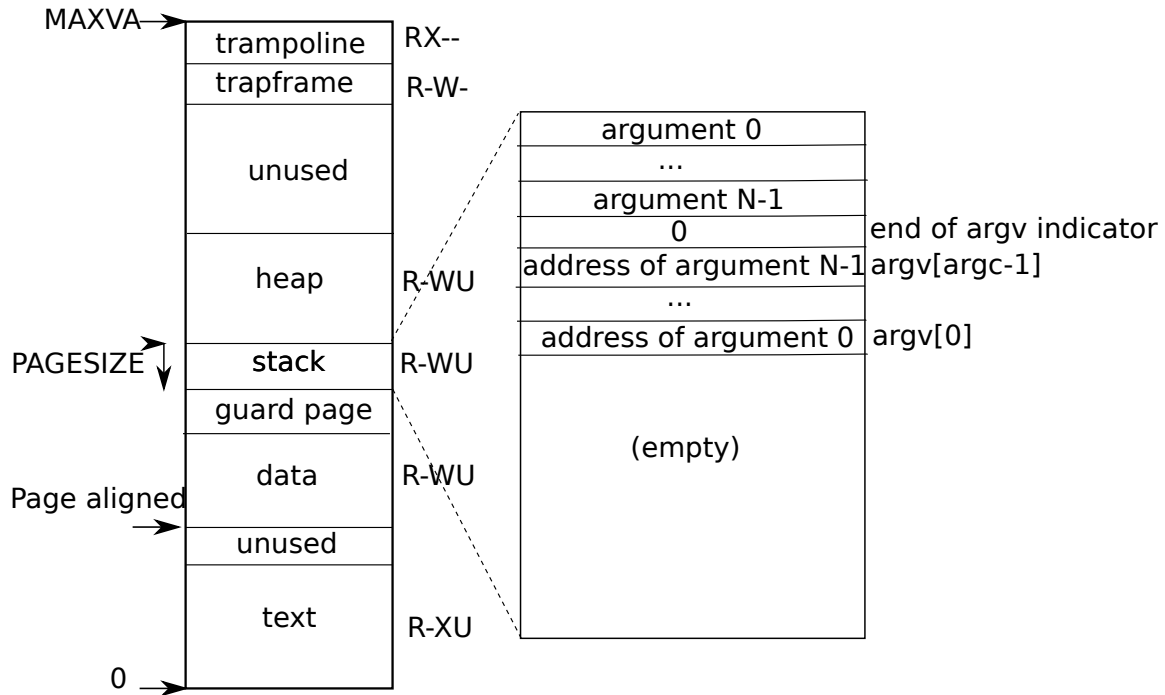
بالمثل، عبر رسم البيانات بدون PTE_X لا يمكن لبرنامج مستخدم القفز بالصدفة لعنوان في بيانات البرنامج وبدء التنفيذ عند ذلك العنوان.

في العالم الحقيقي، تقوية عملية عبر ضبط الأذونات بحذر يساعد أيضاً في الدفاع ضد الهجمات الأمنية. خصم ما قد يغذي مدخلات مصممة بحذر لبرنامج (مثلاً، خادم ويب) تطلق خطأ في البرنامج في أمل تحويل ذلك الخطأ لاستغلال أمني. ضبط الأذونات بحذر وتقنيات أخرى، مثل تعشيت تخطيط مساحة عناوين المستخدم، تجعل مثل هذه الهجمات أصعب.

المكدس صفحة واحدة، وموضح بالمحتويات الأولية كما أنشئت باستدعاء النظام exec. النصوص الحاوية لمعاملات سطر الأوامر، بالإضافة لمصفوفة مؤشرات إليها، تتواجد في أقصى أعلى المكدس. أسفل ذلك مباشرة توجد قيم تتيح لبرنامج البدء عند main كما لو أن الدالة main(argc, argv) قد دُعيت توأ.

لالتقاط طفح مكدس مستخدم يتجاوز الذاكرة المخصصة للمكدس، تضع xv6 صفحة حراسة غير قابلة للوصول أسفل المكدس مباشرة عبر تصفير العلامة PTE_U. إذا طفح مكدس المستخدم وحاولت العملية استخدام عنوان أسفل المكدس يولد العتاد استثناء خطأ صفحة لأن صفحة الحراسة غير قابلة للوصول لبرنامج ينفذ في نمط المستخدم. نظام تشغيل في العالم الحقيقي قد يخصص بدلاً من ذلك تلقائياً ذاكرة أكثر لمكدس المستخدم عندما يطفح.

نرى هنا بضع أمثلة جميلة لاستخدام جداول الصفحات. أولاً، جداول صفحات العمليات المختلفة تترجم عناوين المستخدم لصفحات مختلفة من الذاكرة الفيزيائية، لكي تحوز كل عملية ذاكرة مستخدم خاصة. ثانياً، كل عملية تشاهد ذاكرتها كحائز عناوين افتراضية متصلة تبدأ عند الصفر، بينما ذاكرة العملية الفيزيائية قد تكون غير متصلة. ثالثاً، ترسم النواة صفحة بشفرة المنطاد في أعلى مساحة عناوين المستخدم (بدون PTE_U)، وبذلك صفحة واحدة من الذاكرة الفيزيائية تظهر في كافة مساحات العناوين، ولكن يمكن استخدامها فقط بواسطة النواة.



شكل 7: مساحة عناوين مستخدم العملية مع مكدها الأولي.

4.7 الكود البرمجي: exec

يُرجى قراءة kernel/exec.c و kernel/vm.c بدءاً من uvmcreate().

exec استدعاء نظام يستبدل مساحة عناوين مستخدم بعملية بيانات مقروءة من ملف، يُسمى ملفاً ثنائياً أو تنفيذاً. الملف الثنائي يكون عادة مخرج المترجم والمربط، ويحتفظ بتعليمات آلة وبيانات برنامج. تنفيذ النواة الداخلي لـ exec المسمى kexec يفتح الملف الثنائي المسمى path باستخدام namei الموضحة في الفصل 11. ثم يقرأ رأس ELF. الملفات الثنائية لـ xv6 مصوغة بـ طراز ELF واسع الاستخدام والمعرف في kernel/elf.h. يتكون الملف الثنائي لـ ELF من رأس struct elfhdr، متبوعاً بمتتالية من رؤوس أقسام البرنامج struct proghdr. كل proghdr يصف جزءاً من التطبيق يجب تحميله في الذاكرة؛ برامج xv6 تحوز رأسي قسم برنامج: واحد للتعليمات وواحد للبيانات.

الخطوة الأولى هي فحص سريع يكون الملف يرجح احتوائه ملفاً ثنائياً لـ ELF. يبدأ الملف الثنائي لـ ELF بالـ «رقم السحري» المكون من أربعة بايتات 'F', 'L', 'E', '0x7F' أو الثابت ELF_MAGIC. إذا حاز رأس ELF الرقم السحري الصحيح تفترض kexec أن الملف الثنائي جيد الطراز.

تخصص kexec جدول صفحات جديد بدون رسومات مستخدم بـ proc_pagetable، وتخصص ذاكرة لكل قطعة ELF بـ uvmalloc، وتحمل كل قطعة في الذاكرة بـ loadseg. تستخدم loadseg الدالة walkaddr للعثور على العنوان الفيزيائي للذاكرة المخصصة التي سٌكتب عندها كل صفحة من قطعة ELF، و readi للقراءة من الملف.

رأس قسم البرنامج لـ init/ وهي أول برنامج مستخدم ينشأ بـ kexec يبدو هكذا:

```
objdump -p user/_init #

user/_init:      file format elf64-little

:Program Header
0x70000003 off    0x00000000000006bb0 vaddr 0x0000000000000000
paddr 0x0000000000000000 align 2**0
--filez 0x000000000000004a memsz 0x0000000000000000 flags r
LOAD off    0x00000000000001000 vaddr 0x0000000000000000
paddr 0x0000000000000000 align 2**12
filez 0x00000000000001000 memsz 0x00000000000001000 flags r-x
LOAD off    0x00000000000002000 vaddr 0x00000000000001000
paddr 0x00000000000001000 align 2**12
--filez 0x0000000000000010 memsz 0x0000000000000030 flags rw
STACK off    0x0000000000000000 vaddr 0x0000000000000000
paddr 0x0000000000000000 align 2**4
--filez 0x0000000000000000 memsz 0x0000000000000000 flags rw
```

نشاهد أن قطعة النص ينبغي تحميلها عند العنوان الافتراضي 0x0 في الذاكرة (بدون أذونات كتابة) من محتوى عند الإزاحة 0x1000 في الملف. كما نشاهد أن البيانات ينبغي تحميلها عند العنوان 0x1000 وهو حد صفحة، وبدون أذونات تنفيذ.

حجم الملف filez لرأس قسم برنامج قد يكون أقل من حجم الذاكرة memsz مما يشير لأن الفجوة بينهما ينبغي ملؤها بأصفار (لمتغيرات C العامة) بدلاً من القراءة من الملف. لـ init/ فإن filez للبيانات هو 0x10 بايت و memsz هو 0x30 بايت، وبذلك تخصص uvmalloc ذاكرة فيزيائية كافية لاحتجاز 0x30 بايت ولكن تقرأ فقط 0x10 بايت من الملف init/.

الآن تخصص kexec وتهيئ مكسد المستخدم. تخصص صفحة مكسد واحدة مجرداً. تنسخ kexec نصوص المعاملات لأعلى المكسد واحدة في الوقت الواحد، مسجلة المؤشرات إليها في ustack. تضع مؤشراً مفرغاً في نهاية ما سيكون قائمة argv الممررة لـ main. القيم لـ argc و argv تُمرر لـ main عبر مسار العودة من استدعاء النظام: argc يُمرر عبر القيمة المرجوعة لاستدعاء النظام بـ a0 و argv يُمرر عبر المدخل a1 لـ trapframe العملية.

تضع kexec صفحة غير قابلة للوصول أسفل صفحة المكسد مباشرة، لكي ترجع البرامج التي تحاول استخدام أكثر من صفحة خطأ. هذه الصفحة غير القابلة للوصول تتيح لـ kexec أيضاً التعامل مع معاملات ضخمة جداً؛ في ذلك الموقف فإن الدالة copyout التي تستخدمها kexec لنسخ المعاملات للمكسد ستلاحظ أن صفحة الوجهة غير قابلة للوصول وترجع -1.

أثناء إعداد صورة الذاكرة الجديدة لو اكتشفت kexec خطأ مثل قسم برنامج غير صالح فإنها تقفز للعلامة bad وتفرغ الصورة الجديدة وترجع -1. يجب على kexec الانتظار لتفريغ الصورة القديمة حتى تتأكد من أن استدعاء النظام سينجح: لو زالت الصورة القديمة فلن يتمكن استدعاء النظام من إرجاع -1 إليها. حالات الخطأ الوحيدة بـ kexec تقع أثناء إنشاء الصورة. بمجرد اكتمال الصورة يمكن لـ kexec التثبيت لجداول الصفحات الجديد وتفرغ القديم.

استدعاء النظام exec يحمل بايتات من ملف ELF في الذاكرة عند عناوين موصفة بواسطة ملف ELF. يمكن للمستخدمين أو العمليات وضع أية عناوين يريدونها في ملف ELF. وهكذا فإن exec محفوفة بالمخاطر، لأن العناوين في ملف ELF قد تشير للنواة بالصدفة أو قصداً. العواقب لنواة غير حذرة قد تتراوح من انهيار لاستغلال خبيث لآليات العزل للنواة. تنفذ xv6 عدداً من الفحوصات لتجنب هذه المخاطر. على سبيل المثال if(ph.vaddr + ph.memsz < ph.vaddr) تفحص ما إذا كان المجموع يفيض عن عدد صحيح بعرض 64 بت. الخطر هو أن مستخدماً قد يبنى ملف ثنائي ELF مع ph.vaddr يشير لعنوان يختاره المستخدم و ph.memsz ضخم بما يكفي لكي يفيض المجموع لـ 0x1000 والذي سيبدو كقيمة صالحة. في نسخة أقدم من xv6 كانت فيها مساحة عناوين المستخدم تحوي أيضاً النواة كان يمكن للمستخدم اختيار عنوان يناظر ذاكرة النواة وبالتالي ينسخ بيانات من الملف الثنائي لـ ELF في النواة. في نسخة RISC-V من xv6 لا يمكن حدوث هذا لأن النواة تحوز جدول صفحاتها المستقل؛ loadseg تحمل في جدول صفحات العملية وليس جدول صفحات النواة.

من السهل لمطور النواة إغفال فحص حرج والنوى في العالم الحقيقي تحوز تاريخاً طويلاً من فحصات مفقودة تمكنت برامج المستخدمين من استغلالها للحصول على امتيازات النواة. من المرجح أن xv6 لا تنفذ عملاً كاملاً في التحقق من صحة بيانات مستوى المستخدم الموفرة للنواة، والتي قد يتمكن برنامج مستخدم خبيث من استغلالها للالتفاف على عزل xv6.

4.8 العالم الحقيقي

مثل معظم نظم التشغيل تستخدم xv6 عتاد الصفحات لحماية الذاكرة والرسومات. معظم نظم التشغيل تستخدم تحويل الصفحات بأسلوب أكثر تعقيداً من xv6 عبر الدمج بين تحويل الصفحات واستثناءات أخطاء الصفحات، وهو ما ناقشه في الفصل 6.

تُبسّط xv6 عبر استخدام النواة لرسم مباشر بين العناوين الافتراضية والفيزيائية، وعبر افتراضها وجود RAM فيزيائية عند العنوان 0x80000000 حيث تتوقع النواة أن تُحمل. هذا يعمل مع QEMU لكنه على العتاد الحقيقي يتبين أنه فكرة سيئة؛ العتاد الحقيقي يضع RAM والأجهزة عند عناوين فيزيائية غير متوقعة لكي لا توجد RAM عند 0x80000000 حيث تتوقع xv6 القدرة على تخزين النواة. تصاميم النوى الأكثر جدية تستغل جدول الصفحات لتحويل تخطيطات الذاكرة الفيزيائية العشوائية للعتاد لتخطيطات عناوين افتراضية متوقعة للنواة.

تدعم RISC-V الحماية على مستوى العناوين الفيزيائية، لكن xv6 لا تستخدم تلك الميزة.

على الآلات بحوزة ذاكرة كثيرة قد يكون من المنطقي استخدام دعم RISC-V لـ «الصفحات الفائقة». الصفحات الصغيرة تكون منطقية عندما تكون الذاكرة الفيزيائية صغيرة، للسماح بالتخصيص والتحويل للقرص بدرجة دقيقة. على سبيل المثال لو كان برنامج يستخدم فقط 8 كيلوبايت من الذاكرة فإن إعطائه صفحة فائقة كاملة بحجم 4 ميغابايت من الذاكرة الفيزيائية أمر هادر. الصفحات الأكبر تكون منطقية على الآلات بكثرة من RAM، وقد تقلل النفقات للتعامل مع جدول الصفحات.

لتجنب التحرير لتفريغ الـ TLB الكامل عند تغيير جداول الصفحات، قد تدعم معالجات RISC-V معرفات مساحات العناوين (ASIDs). تستطيع النواة عندئذ تفريغ مجرد مدخلات TLB لمساحة عناوين محددة. لا تستخدم xv6 هذه الميزة.

افتار نواة xv6 لمخصص شبيه بـ malloc يمكنه توفير ذاكرة لكائنات صغيرة يمنع النواة من استخدام هياكل بيانات معقدة كانت تتطلب تخصيصاً ديناميكياً. نواة أكثر تفصيلاً كانت يرجح أن تخصص أحجاماً مختلفة عديدة من الكتل الصغيرة بدلاً من مجرد كتل بحجم 4096 بايت كـ xv6؛ مخصص النواة الحقيقي سيتعين عليه التعامل مع التخصيصات الصغيرة والكبيرة.

تخصيص الذاكرة موضوع ساخن دائم، المشاكل الأساسية هي الاستخدام الكفء للذاكرة المحدودة والتجهز لطلبات مستقبلية غير معروفة. اليوم يهتم الناس بالسرعة أكثر من كفاءة المساحة.

4.9 تمارين

1. حلل شجرة أجهزة RISC-V للعثور على مقدار الذاكرة الفيزيائية التي يحوزها الحاسوب.
2. الدالتان copyinstr و copyinstr تقطعان جدول صفحات المستخدم بالبرمجيات. اضبط جدول صفحات النواة لكي تحوز النواة برنامج المستخدم مرسوماً وتتمكن copyinstr و copyinstr من استخدام memcpy لنسخ معاملات استدعاءات النظام في مساحة النواة معتمدة على العتاد للقيام بقطع جدول الصفحات.
3. عدل xv6 لاستخدام الصفحات الفائقة للنواة.
4. تنفيذات Unix لـ exec تتضمن تقليدياً معالجة خاصة لبرامج موجه الأوامر النصية. إذا كان الملف للتنفيذ يبدأ بالنص #! فإن السطر الأول يؤخذ كبرنامج للتنفيذ لتفسير الملف. على سبيل المثال لو دُعيت exec لتنفيذ myprog arg1 وكان السطر الأول لـ myprog هو #!/interp فإن exec تشغل /interp مع سطر الأوامر /interp myprog arg1. نفذ الدعم لهذا العرف بـ xv6.
5. نفذ تعشية تخطيط مساحة العناوين (ASLR) للنواة.

5 المصايد واستدعاءات النظام

هناك ثلاثة أنواع من الأحداث التي تجبر المعالج CPU على تعليق التنفيذ العادي للتعليمات وتحويل التحكم قسراً إلى شفرة خاصة في النواة تتعامل مع الحدث. الحالة الأولى هي استدعاء النظام (System call)، وذلك عندما

ينفذ برنامج مستخدم تعليمة `ecall` لطلب خدمة معينة من النواة. الحالة الثانية هي الاستثناء (Exception): عندما تنفذ تعليمة (في نمط المستخدم أو المشرف) خطأً غير صالح، مثل التحميل من عنوان افتراضي غير صالح. الحالة الثالثة هي مقاطعة الجهاز (Device interrupt): عندما يشير جهاز إدخال/إخراج إلى أنه يطلب اهتماماً، مثل إتمام عتاد القرص الصلب لطلب قراءة أو كتابة.

يستخدم هذا الكتاب مصطلح المصيدة (Trap) كعنوان عام وشامل لهذه الحالات الثلاث. عادةً ما تكون الشفرة التي كانت تنفذ وقت حدوث المصيدة هي شفرة ستحتاج لاحقاً لاستئناف عملها، ولا ينبغي أن تحتاج للمعرفة بوقوع أي حدث خاص. أي أننا نريد غالباً أن تكون المصايد شفافة تماماً؛ ويُعد هذا الأمر هاماً بصفة خاصة لمقاطعات الأجهزة التي لا تتوقع الشفرة المنقطعة وقوعها عادةً. تجبر المصيدة تحويل التحكم إلى النواة؛ حيث تحفظ النواة السجلات وحالات التنفيذ الأخرى لكي يستأنف التنفيذ لاحقاً؛ وتنفذ النواة الشفرة المعالجة المناسبة (مثل تنفيذ استدعاء النظام أو برنامج تشغيل الجهاز)؛ ثم تستعيد النواة الحالة المحفوظة وترجع من المصيدة؛ وتستأنف الشفرة الأصلية عملها من حيث توقفت.

يتعامل xv6 مع كافة المصايد في النواة؛ ولا تُسلم المصايد مباشرة لشفرة المستخدم. تعتبر معالجة المصايد في النواة أمراً طبيعياً بالنسبة لاستدعاءات النظام. كما أنها منطقية تماماً للمقاطعات لأن العزل الصارم يتطلب السماح للنواة وحدها باستخدام الأجهزة، ولأن النواة قادرة على مشاركة الأجهزة بين عمليات متعددة. كذلك تُعد منطقية للاستثناءات لأن النواة قد تكون قادرة على معالجة الاستثناء القادم من مساحة المستخدم (انظر الفصل 6 كمنال) أو الاستجابة بقتل البرنامج المخالف.

تمر معالجة المصيدة في xv6 بأربع مراحل متتالية: إجراءات عتادية يتولاها معالج RISC-V، وبعض تعليمات التجميع التي تمهد الطريق لشفرة C في النواة، ودالة C في النواة تقرر كيفية التعامل مع المصيدة، وروتين خدمة استدعاء النظام أو برنامج تشغيل الجهاز. وعلى الرغم من أن الخصائص المشتركة بين الأنواع الثلاثة للمصايد توحى بإمكانية أن تعالج النواة كافة المصايد عبر مسار برلمجي موحد، إلا أنه يتبين أن من المريح تخصيص شفرة مستقلة لحالتين متميزتين: المصايد القادمة من مساحة المستخدم، والمصايد القادمة من مساحة النواة. تُسمى شفرة النواة (بلغة التجميع أو C) التي تعالج المصيدة بـ المعالج (Handler)؛ وتُكتب تعليمات المعالج الأولى بلغة التجميع عادةً (بدلاً من C) وتُسمى أحياناً بـ المتجه (Vector).

قبل المتابعة، يُرجى قراءة الملفات: `kernel/trampoline.S` و `kernel/trap.c`.

5.1 آليات المصايد في معالج RISC-V

يضم كل معالج RISC-V مجموعة من سجلات التحكم والحالة الفيزيائية (Control and Status Registers, CSRs) التي تكتبها النواة لإخبار المعالج بكيفية معالجة المصايد، والتي تقرأها النواة للتعرف على المصيدة التي وقعت. تحتوي وثائق RISC-V على التفاصيل الكاملة. ويحتوي الملف `kernel/riscv.h` على التعاريف التي يستخدمها xv6. فيما يلي مخطط لأهم هذه السجلات:

• `stvec` (Supervisor Trap Vector Base Address Register)

تكتب النواة في هذا المسجل عنوان شفرة معالج المصيدة الخاص بها؛ حيث يقفز معالج RISC-V إلى العنوان المكتوب في `stvec` لمعالجة المصيدة.

• `sepc` (Supervisor Exception Program Counter)

عند وقوع المصيدة، يحفظ معالج RISC-V مؤشر التعليمات `pc` هنا (نظراً لأن `pc` سيتم كتابته بالقيمة الموجودة في `stvec`). تقوم تعليمة العودة من المصيدة `sret` بنسخ القيمة المخزنة في `sepc` إلى `pc`. ويمكن للنواة كتابة `sepc` للتحكم بجهة العودة عند تنفيذ `sret`.

• `scause` (Supervisor Cause Register)

يضع معالج RISC-V في هذا المسجل رقماً يصف السبب الدقيق لوقوع المصيدة.

• `sscratch` (Supervisor Scratch Register)

تستخدم شفرة معالج المصيدة في النواة المسجل `sscratch` لمساعدتها على تجنب الكتابة فوق سجلات المستخدم قبل حفظها.

• `sstatus` (Supervisor Status Register)

يتحكم البت SIE في sstatus بتفعيل أو تعطيل مقاطعات الأجهزة. إذا قامت النواة بتصفير البت SIE، سيعمل معالج RISC-V على إرجاء مقاطعات الأجهزة حتى تقوم النواة بتفعيل البت SIE. وبشير البت SPP إلى ما إذا كانت المصيدة قد جاءت من نمط المستخدم أم نمط المشرف، كما يتحكم بالنمط الذي تعود إليه تعليمة sret.

لا يمكن الوصول للسجلات أعلاه إلا في نمط المشرف (Supervisor mode) فقط (أي بواسطة النواة)؛ وتمنع وحدة المعالجة المركزية شفرة المستخدم من قراءتها أو كتابتها.

يحتفظ كل معالج فيزيائي على الشريحة متعددة النوى بطقم كامل ومستقل من هذه السجلات، وقد يعالج أكثر من معالج واحد مصيدة في الوقت نفسه.

عندما يجبر العتاد المعالج على معالجة مصيدة، ينفذ عتاد RISC-V الخطوات التالية:

1.

إذا كانت المصيدة مقاطعة جهاز وكان البت SIE في sstatus صفراً، لا تنفذ أيّاً من الخطوات التالية.

2.

تعطيل المقاطعات عبر تصفير البت SIE في sstatus.

3.

نسخ قيمة مؤشر التعليمات pc إلى المسجل sepc.

4.

حفظ النمط الحالي (مستخدم أم مشرف) في البت SPP في sstatus.

5.

ضبط مسجل scause برقم يوضح سبب المصيدة.

6.

تحويل نمط التنفيذ إلى نمط المشرف (Supervisor mode).

7.

نسخ العنوان المخزن في stvec إلى مؤشر التعليمات pc.

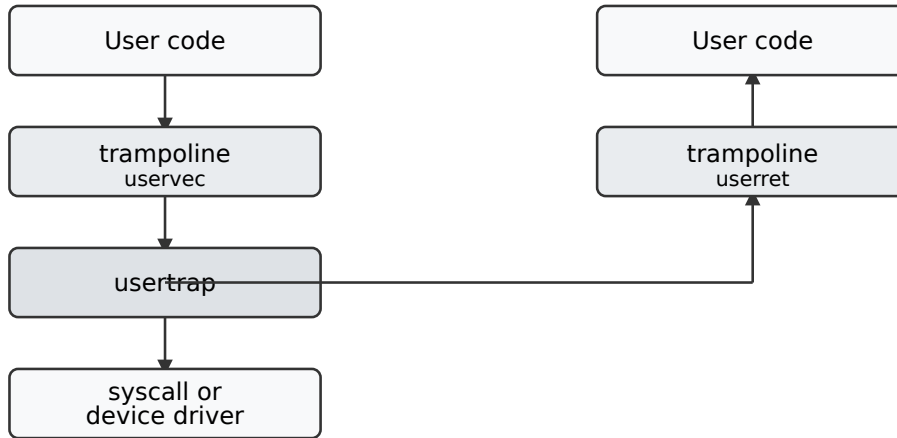
8.

بدء تنفيذ التعليمات من العنوان الجديد المخزن في pc.

لاحظ أن المعالج لا يبدل إلى جدول صفحات النواة، ولا يبدل إلى مكدر في النواة، ولا يحفظ أية سجلات أخرى غير pc. يجب على برمجيات النواة تنفيذ هذه المهام بنفسها. إن أحد أسباب قيام المعالج بحد أدنى من العمل أثناء المصيدة هو توفير المرونة الكاملة للبرمجيات؛ على سبيل المثال، تتجاوز بعض نظم التشغيل عملية تبديل جدول الصفحات في بعض الحالات لزيادة أداء المصايد.

من المفيد التفكير فيما إذا كان بالإمكان الاستغناء عن أي من الخطوات المذكورة أعلاه في سبيل تحقيق مزايا أسرع. وعلى الرغم من وجود حالات يمكن أن يعمل فيها تسلسل أبسط، إلا أن الاستغناء عن العديد من الخطوات سيكون خطيراً بشكل عام. على سبيل المثال، افترض أن المعالج لم يقم بتبديل مؤشر التعليمات؛ عندها يمكن لمصيدة قادمة من مساحة المستخدم أن تبدل إلى نمط المشرف بينما لا تزال تنفذ تعليمات المستخدم. يمكن لتلك التعليمات التابعة للمستخدم أن تكسر العزل بين المستخدم والنواة، على سبيل المثال عبر تعديل المسجل satp ليشير إلى جدول صفحات يتيح الوصول لكامل الذاكرة الفيزيائية. لذا يُعد من الضروري أن يبدل المعالج إلى عنوان تعليمات محدد من قبل النواة، وهو العنوان المخزن في stvec.

5.2 المصايد القادمة من مساحة المستخدم



شكل 8: مخطط عام لكيفية معالجة المصيدة القادمة من شفرة المستخدم.

يتعامل xv6 مع المصايد بأسلوب مختلف اعتماداً على ما إذا كانت المصيدة قد وقعت أثناء التنفيذ في النواة أو في شفرة المستخدم. فيما يلي القصة الخاصة بالمصايد القادمة من شفرة المستخدم؛ بينما يصف القيد التالي المصايد القادمة من شفرة النواة.

قد تقع المصيدة أثناء التنفيذ في مساحة المستخدم إذا ينفذ برنامج المستخدم استدعاء نظام (تعليمية `ecall`)، أو قام بفعل غير صالح، أو إذا أصدر جهاز مقاطعة. كما يتضح في الشكل شكل 8، فإن المسار رفيع المستوى للمصيدة القادمة من مساحة المستخدم يبدأ بـ `uservec` في `kernel/trampoline.S`، ثم `usertrap` في `kernel/trap.c`؛ وعندما تجهز النواة للعودة، ترجع `usertrap` إلى `userret` في `kernel/trampoline.S` والتي تنفذ تعليمية `sret` للعودة إلى مساحة المستخدم.

إن القيد الرئيسي في تصميم معالجة المصايد بـ xv6 هو حقيقة أن عتاد RISC-V لا يبدل جداول الصفحات عندما يجبر وقوع مصيدة. هذا يعني أن عنوان معالج المصيدة المخزن في `stvec` يجب أن يحظى برسم صالح في جدول صفحات المستخدم، لأنه جدول الصفحات الساري وقت بدء تنفيذ شفرة معالجة المصيدة. علاوة على ذلك، تحتاج شفرة معالجة المصيدة بـ xv6 إلى التبديل لجدول صفحات النواة؛ ولكي تتمكن من الاستمرار في التنفيذ بعد هذا التبديل، يجب أن يحتوي جدول صفحات النواة أيضاً على رسم لمعالج المصيدة المشار إليه بـ `stvec`.

يحقق xv6 هذه المتطلبات باستخدام صفحة المنطاد (Trampoline). تحتوي هذه الصفحة على `uservec`، وهي شفرة معالجة المصايد بـ xv6 التي يشير إليها المسجل `stvec`. تُرسم صفحة المنطاد في جدول صفحات كل عملية عند العنوان الافتراضي `0x3fffffff000` (المسمى TRAMPOLINE)، وهي الصفحة الأخيرة في مساحة العناوين الافتراضية لكي تستقر أعلى الذاكرة التي تستخدمها البرامج لنفسها. تُرسم صفحة المنطاد عند نفس العنوان الافتراضي في جدول صفحات النواة. (انظر الشكلين شكل 3 و شكل 6). ونظراً لأن صفحة المنطاد مرسومة في جدول صفحات المستخدم، يمكن للمصايد بدء التنفيذ هناك في نمط المشرف. ونظراً لأن صفحة المنطاد مرسومة عند نفس العنوان في مساحة عناوين النواة، يمكن لمعالج المصيدة الاستمرار في التنفيذ بعد التبديل لجدول صفحات النواة.

تستقر الشفرة البرمجية لمعالج المصيدة `uservec` في `kernel/trampoline.S`. عندما تبدأ `uservec` التنفيذ، تحتوي كافة السجلات الـ 32 على قيم مملوكة لشفرة المستخدم المنقطعة. تجب حماية هذه القيم الـ 32 وحفظها في مكان ما في الذاكرة، لكي تتمكن النواة لاحقاً من استعادتها قبل العودة لمساحة المستخدم. يتطلب التخزين في الذاكرة استخدام سجل لحفظ عنوان وجهة التخزين، ولكن في هذه النقطة لا تتوفر أية سجلات عامة الاستخدام متاحة! لحسن الحظ يوفر عتاد RISC-V يد العون عبر المسجل `sscratch`. تقوم تعليمية `csrw` في بداية `uservec` بحفظ قيمة السجل `a0` في `sscratch`. الآن أصبح لـ `uservec` سجل واحد (`a0`) متاح للعمل به!

تتمثل المهمة التالية لـ `uservec` في حفظ سجلات المستخدم الـ 32. تخصص النواة لكل عملية صفحة ذاكرة لهيكل `trapframe` الذي يحتوي (من بين أمور أخرى) على مساحة لحفظ سجلات المستخدم الـ 32. ونظراً لأن `satp` لا يزال يشير إلى جدول صفحات المستخدم، تحتاج `uservec` لرسم هيكل `trapframe` في مساحة عناوين المستخدم. يرسم `xv6` هيكل `trapframe` الخاص بكل عملية عند العنوان الافتراضي `(0x3fffffe000) TRAPFRAME` في جدول صفحات المستخدم لتلك العملية؛ أي صفحة واحدة أسفل `TRAMPOLINE`. يحتوي الحقل `p->trapframe` الخاص بكل عملية على عنوان افتراضي في النواة لـ `trapframe` الخاص بالعملية.

تضبط `uservec` السجل `a0` ليشير إلى العنوان `TRAPFRAME` وتحفظ كافة سجلات المستخدم هناك. ثم تستعيد قيمة `a0` الخاصة بالمستخدم من المسجل `sscratch` وتحفظها في `trapframe`.

كانت النواة قد هياأت هيكل `trapframe` مسبقاً ليحتوي على بعض القيم النافعة لـ `uservec`: عنوان مكدس النواة الخاص بالعملية الحالية، ورقم المعالج `hartid` الحالي، وعنوان الدالة البرمجية `usertrap`، وعنوان جدول صفحات النواة. تستعيد `uservec` هذه القيم، وتبدل `satp` ليشير لجدول صفحات النواة، وتقفز إلى الدالة `usertrap` بلغة C.

تتلخص وظيفة `usertrap` في تحديد سبب المصيدة، ومعالجتها، ثم العودة. تقوم أولاً بتغيير `stvec` لكي تُعالج أية مصيدة تقع أثناء التواجد في النواة بواسطة `kernelvec` بدلاً من `uservec`. وتحفظ المسجل `sepc` (مؤشر تعليمات المستخدم المحفوظ) للاستخدام المستقبلي عند العودة لمساحة المستخدم. إذا كانت المصيدة استدعاء نظام، تدعو `usertrap` الدالة `syscall` لمعالجته؛ وإذا كانت مقاطعة جهاز، تدعو `devintr`؛ وإذا كانت خطأ صفحة، تدعو `vmfault`؛ وإلا فإنها استثناء (مثل استخدام عنوان غير صالح)، وتقوم النواة بإنهاء العملية المخالفة. يضيف مسار استدعاء النظام الرقم 4 إلى مؤشر تعليمات المستخدم المحفوظ لأن RISC-V في حالة استدعاء النظام يترك مؤشر التعليمات إشارياً إلى تعليمة `ecall` نفسها، بينما تحتاج شفرة المستخدم لاستئناف التنفيذ في التعليمات التالية. تفحص `usertrap` ما إذا كانت العملية قد قُتلت أو يجب أن تتنازل عن المعالج (إذا كانت هذه المصيدة مقاطعة ميقاتي).

تكن الخطوة الأولى في العودة لمساحة المستخدم عند استدعاء `prepare_return`. تضبط هذه الدالة سجلات تحكم RISC-V للتجهز لمصيدة مستقبلية قادمة من مساحة المستخدم: ضبط `stvec` ليشير إلى `uservec` وتجهيز حقول `trapframe` التي تعتمد عليها `uservec`. تضبط `prepare_return` المسجل `sepc` بمؤشر تعليمات المستخدم المحفوظ مسبقاً. وأخيراً، ترجع `usertrap` إلى `userret` في صفحة المنطاد، وتمرر مؤشراً لجدول صفحات المستخدم في `a0`.

تبدل `userret` المسجل `satp` ليشير لجدول صفحات المستخدم الخاص بالعملية. تذكر أن جدول صفحات المستخدم يرسم كلاً من صفحة المنطاد و `TRAPFRAME` ولا يرسم أي شيء آخر من النواة. يتيح رسم صفحة المنطاد عند نفس العنوان الافتراضي في جدول صفحات المستخدم والنواة لـ `userret` الاستمرار في التنفيذ بعد تغيير `satp`. ابتداءً من هذه النقطة، تكون البيانات الوحيدة التي يمكن لـ `userret` استخدامها هي محتويات السجلات ومحتوى `trapframe`. تحمل `userret` عنوان `TRAPFRAME` في `a0`، وتستعيد سجلات المستخدم المحفوظة من `trapframe` عبر `a0`، وتستعيد السجل `a0` الخاص بالمستخدم، وتنفذ تعليمة `sret` للعودة إلى مساحة المستخدم.

تُكتب `uservec` و `userret` بلغة التجميع لأنه من الصعب كتابة شفرة C لحفظ أو استعادة كافة السجلات أو النجاة من تغيير جداول الصفحات.

5.3 الكود البرمجي: استدعاء استدعاءات النظام

تستدعي برامج المستخدم دوال المكتبة من أجل تنفيذ استدعاءات النظام. على سبيل المثال، يعرض موجه الأوامر (Shell) علامة الحث بالاستدعاء التالي (في `user/sh.c`):

```
;write(2, "$ ", 2)
```

فيما يلي دالة المكتبة المخصصة لذلك في `user/usys.S`:

```
:write
li a7, SYS_write
ecall
ret
```

تولّد الشفرة التي ينشئها مترجم C لاستدعاء الدالة تحميلاً للمعاملات الثلاثة في السجلات `a0` و `a1` و `a2`. ثم تقوم الدالة `write()` بتحميل رقم استدعاء النظام `SYS_write` (الرقم 16) في `a7`. ستفحص النواة تلك السجلات للتعرف

على استدعاء النظام المقصود، وما هي المعاملات الممررة. تجبر تعليمة `ecall` وقوع مصيدة من مساحة المستخدم في النواة وتتسبب في تنفيذ `uservec` ثم `usertrap` ثم `syscall`.

عند هذه النقطة، يُرجى قراءة الملفات: `kernel/syscall.c` و `sys_write()` في `kernel/sysfile.c`، و `copyout()`، `copyinstr()` في `kernel/vm.c`.

تستعيد الدالة `syscall` رقم استدعاء النظام من السجل `a7` المحفوظ في `trapframe` وتستخدمه كفهرس في المصفوفة `syscalls`. بالنسبة لمثالنا، يحتوي `a7` على `SYS_write` مما يؤدي لاستدعاء الدالة المنفذة لاستدعاء النظام `sys_write`.

عندما ترجع `sys_write` قيمتها، تسجل `syscall` القيمة المرجعة في `a0` `trapframe->a0`. سيتسبب هذا في جعل الاستدعاء الأصلي في مساحة المستخدم لـ `write()` يرجع تلك القيمة، نظراً لأن قواعد الاستدعاء لـ C في RISC-V تضع القيم المرجعة في `a0`. ترجع استدعاءات النظام عادةً أرقاماً سالبة للإشارة لأخطاء، وصفر أو أرقاماً موجبة عند النجاح.

5.4 الكود البرمجي: معاملات استدعاءات النظام

تبدأ معاملات استدعاءات النظام في سجلات المستخدم، ثم تُنقل إلى هيكل المصيدة عبر شفرة مصاد النواة. تستعيد دوال النواة `argint` و `argaddr` و `argfd` المعامل رقم `n` لاستدعاء النظام من هيكل المصيدة كعدد صحيح، أو مؤشر، أو واصف ملف.

تمرر بعض استدعاءات النظام مؤشرات كمعاملات، ويجب على النواة استخدام تلك المؤشرات لقراءة أو كتابة ذاكرة المستخدم. على سبيل المثال، يمرر استدعاء النظام `write` للنواة مؤشراً في مساحة المستخدم للبيانات المراد كتابتها. تفرض هذه المؤشرات تحديين: الأول: قد يكون برنامج المستخدم به خطأ برمجي أو خبيثاً، وقد يمرر للنواة مؤشراً غير صالح أو مؤشراً يهدف لخداع النواة للوصول لذاكرة النواة بدلاً من ذاكرة المستخدم. الثاني: رسومات جدول صفحات النواة بـ xv6 ليست متطابقة مع رسومات جدول صفحات المستخدم، لذا لا يمكن للنواة استخدام التعليمات العادية للتحميل أو التخزين من العناوين المزودة بواسطة المستخدم.

تنفذ النواة دوال تنقل البيانات بأمان من وإلى العناوين المزودة بواسطة المستخدم. تعد الدالة `fetchstr` مثلاً على ذلك. تستخدم استدعاءات نظام ملفات مثل `exec` الدالة `fetchstr` لاستعادة المعاملات النصية لأسماء الملفات من مساحة المستخدم. تدعو `fetchstr` الدالة `copyinstr` لتنفيذ العمل الشاق.

تنسخ الدالة `copyinstr` ما يصل إلى `max` بايت إلى `dst` من العنوان الافتراضي `srcva` في جدول صفحات المستخدم `pagetable`. ونظراً لأن `pagetable` ليس هو جدول الصفحات الحالي، تستخدم `copyinstr` الدالة `walkaddr` (التي تدعو `walk`) للبحث عن `srcva` في `pagetable` للحصول على العنوان الفيزيائي `pa0`. يرسم جدول صفحات النواة كافة الذاكرة الفيزيائية RAM عند عناوين افتراضية مساوية للعنوان الفيزيائي للذاكرة. يتيح هذا لـ `copyinstr` نسخ بايتات النص مباشرة من `pa0` إلى `dst`. تفحص `walkaddr` ما إذا كان العنوان الافتراضي المزود بواسطة المستخدم جزءاً من مساحة عناوين المستخدم للعمليات، حتى لا تتمكن البرامج من خداع النواة لقراءة ذاكرة أخرى. وتقوم دالة شبيهة تُسمى `copyout` بنسخ البيانات من النواة إلى عنوان مزود بواسطة المستخدم.

5.5 المصايد القادمة من مساحة النواة

يُرجى قراءة `kernel/kernelvec.S` و `kerneltrap()` في `kernel/trap.c`.

يتعامل xv6 مع المصايد القادمة من شفرة النواة بأسلوب مختلف عن المصايد القادمة من شفرة المستخدم. عند الدخول للنواة، توجه `usertrap` المسجل `stvec` إلى شفرة التجميع في `kernelvec`. ونظراً لأن `kernelvec` لا تنفذ إلا إذا كان xv6 في النواة بالفعل، يمكن لـ `kernelvec` الاعتماد على كون `satp` مضبوطاً لجدول صفحات النواة، وعلى كون مؤشر المكس يشير لمكدس نواة صالح. تدفع `kernelvec` كافة السجلات المحفوظة بواسطة المستدعي على المكس الحالي، والتي ستستعيدها لاحقاً حتى تتمكن شفرة النواة المنقطعة من استئناف التنفيذ دون اضطراب. تحفظ `kernelvec` السجلات على مكس خيط النواة المنقطع، وهو أمر منطقي لأن قيم السجلات تنتمي لذلك الخيط. ويُعد هذا أمراً هاماً بصفة خاصة إذا تسببت المصيدة في التبديل لخيط مختلف - في تلك الحالة سترجع المصيدة في الواقع من مكس الخيط الجديد، تاركة السجلات المحفوظة للخيط المنقطع بأمان على مكده.

تقفز kernelvec إلى kerneltrap بعد حفظ السجلات. تتجهز kerneltrap لنوع واحد فقط من المصائد: مقاطعات الأجهزة. وتدعو devintr لمعالجتها. إذا لم تكن المصيدة مقاطعة جهاز، يجب أن تكون استثناءً، مثل محاولة شفرة النواة استخدام مؤشر غير صالح. لا يمكن أن ينتج هذا إلا عن خطأ برمجي في شفرة النواة. لا تملك النواة طريقة للتعافي في هذه الحالة، لذا تدعو panic() التي تطبع رسالة خطأ ثم تتوقف.

إذا استدعيت kerneltrap بسبب مقاطعة ميقاتي، وكان خيط النواة لعملية ما قيد التنفيذ (على عكس خيط الجدولة)، تدعو kerneltrap الدالة yield لمنح الخيوط الأخرى فرصة للتنفيذ. في نقطة ما ستتنازل إحدى تلك الخيوط، وتدع خيطنا و kerneltrap الخاصة به يستأنفان التنفيذ مجدداً. يشرح الفصل الفصل 9 ما يحدث في yield.

عندما يكتمل عمل kerneltrap؛ تحتاج للعودة لأية شفرة كانت منقطعة بواسطة المصيدة. ونظراً لأن yield قد تتسبب في اضطراب sepc والنمط السابق في sstatus؛ تقوم kerneltrap بحفظهما عند بدء عملها. تستعيد الآن سجلات التحكم تلك وترجع إلى kernelvec. تسحب kernelvec السجلات المحفوظة من المكس و تنفذ sret التي تنسخ sepc إلى pc وتستأنف شفرة النواة المنقطعة.

يضبط xv6 المسجل stvec في المعالج لـ kernelvec عندما يدخل ذلك المعالج النواة من مساحة المستخدم؛ يمكنك رؤية ذلك في usertrap. ولكن توجد نافذة زمنية عندما تبدأ النواة في التنفيذ بينما لا يزال stvec مضبوطاً لـ uservec؛ ويُعد من الجوهري عدم وقوع أية مقاطعة جهاز في تلك النافذة. لحسن الحظ يعطل RISC-V المقاطعات دائماً عندما يبدأ في استقبال مصيدة، ولا تفعلها usertrap مجدداً إلا بعد ضبط stvec.

5.6 العالم الحقيقي

يمكن الاستغناء عن الحاجة لصفحات المنطاد إذا كانت ذاكرة النواة مرسومة في جدول صفحات المستخدم لكل عملية (مع إلغاء PTE_U). كان سيلغي ذلك أيضاً الحاجة لتبديل جدول الصفحات عند الوقوع في مصيدة من مساحة المستخدم للنواة. وكان سيتيح ذلك بدوره لتنفيذ استدعاءات النظام في النواة الاستفادة من كون ذاكرة المستخدم للعملية الحالية مرسومة، مما يتيح لشفرة النواة فك إشارة مؤشرات المستخدم مباشرة. استخدمت العديد من نظم التشغيل هذه الأفكار لزيادة الكفاءة. يتجنبها xv6 من أجل تقليل فرص الأخطاء الأمنية في النواة الناتجة عن الاستخدام غير المقصود لمؤشرات المستخدم، ولتقليل بعض التعقيد الذي سيكون مطلوباً لضمان عدم تداخل العناوين الافتراضية للمستخدم والنواة.

5.7 تمارين

1. هل يمكن كتابة بعض أو كل الشفرة في trampoline.S و kernelvec.S بلغة C بدلاً من التجميع؟
2. هل توجد طريقة لإلغاء رسم صفحة TRAPFRAME الخاصة في كل مساحة عناوين مستخدم؟ على سبيل المثال، هل يمكن تعديل uservec لتكتفي بدفع سجلات المستخدم الـ 32 على مكس النواة، أو تخزينها في هيكل proc؟
3. هل يمكن تعديل xv6 لإلغاء رسم صفحة TRAMPOLINE الخاصة؟

6 أخطاء الصفحات

يولد معالج RISC-V استثناء خطأ الصفحة (Page fault) عندما يُستخدم عنوان افتراضي لا يحوز رسماً في جدول الصفحات، أو يحوز رسماً علامته PTE_V صفرية، أو رسماً حظرت بتات أدوناته (PTE_R, PTE_W, PTE_X, PTE_U) العملية المجهزة. يميز RISC-V ثلاثة أنواع من أخطاء الصفحات: أخطاء صفحات التحميل (الناتجة عن تعليمات التحميل)، أخطاء صفحات التخزين (الناتجة عن تعليمات التخزين)، وأخطاء صفحات التعليمات (الناتجة عن استدعاءات التعليمات المراد تنفيذها). يشير المسجل scause لنوع خطأ الصفحة، ويحتوي المسجل stval على العنوان الذي لم يمكن ترجمته.

المنج بين جداول الصفحات وأخطاء الصفحات أداة قوية. تمنح جداول الصفحات النواة مستوى من عدم المباشرة بين العناوين الافتراضية والفيزيائية، لكي تتمكن النواة من التحكم بتركيب ومحتوى مساحات العناوين. تتيح أخطاء الصفحات للنواة الاعتراض للتحميل والتخزين، وعبر تعديل جدول الصفحات، التحديد في الهواء لما تشير إليه تلك المراجع من بيانات. يمكن للنواة استخدام هذه الإمكانيات لزيادة الكفاءة: على سبيل المثال، fork بـ النسخ عند الكتابة تتيح للنواة مشاركة الذاكرة بأسلوب شفاف بين الأب والابن، متجنباً كلفة نسخ الصفحات التي لا يكتبها أي منهما. مبرمجو التطبيقات يمكنهم الاستفادة أيضاً. أحد الاحتمالات هو الملفات المربوطة بالذاكرة، حيث تستخدم النواة تحويل الصفحات لتتسبب في جعل محتوى ملف يظهر في مساحة عناوين تطبيق ما، قراءة كتل الملف

بشفافية استجابة لأخطاء الصفحات. احتمال آخر هو التخصيص الكسول للذاكرة، والذي يتيح لبرنامج طلب مساحة عناوين افتراضية عملاقة، ولكن مع دفع كلفة تخصيص ذاكرة فيزيائية فقط للصفحات التي يقرأها ويكتبها البرنامج بالفعل. تستخدم xv6 أخطاء الصفحات لغرض واحد فقط: التخصيص الكسول.

قبل المتابعة، يُرجى قراءة الدوال البرمجية sys_sbrk() في kernel/sysproc.c و vmfault في kernel/vm.c. ابحث عن الاستدعاءات لـ vmfault في kernel/trap.c و kernel/vm.c.

6.1 التخصيص الكسول للذاكرة

يجوز التخصيص الكسول (Lazy allocation) بـ xv6 جزأين. أولاً، عندما يطلب تطبيق ما ذاكرة عبر دعوة sbrk مع العلامة SBRK_LAZY، تسجل النواة الزيادة في الحجم، لكنها لا تخصص ذاكرة فيزيائية ولا تنشئ PTEs للنطاق الجديد من العناوين الافتراضية. ثانياً، عند وقوع خطأ صفحة على أحد تلك العناوين الجديدة، تخصص النواة صفحة ذاكرة فيزيائية وترسمها في جدول الصفحات. تنفذ النواة التخصيص الكسول بأسلوب شفاف للتطبيقات: لا حاجة لأي تعديلات على التطبيقات لكي تستفيد منه.

التخصيص الكسول مريح للتطبيقات لأنها لا يتعين عليها التنبؤ بدقة بمقدار الذاكرة التي ستحتاجها. على سبيل المثال، تطبيق قد يعالج مدخلات، لكنه لا يعلم مسبقاً كم سيكون حجم المدخلات. مع التخصيص الكسول، يمكن للتطبيق طلب ذاكرة للحالة الأسوأ، لكن دون دفع ثمن تلك الحالة الأسوأ: لا يتعين على النواة القيام بأي عمل للصفحات التي لا يستخدمها التطبيق مطلقاً، والتطبيقات الأخرى يمكنها تخصيص الصفحات غير المستخدمة.

علاوة على ذلك، إذا كان التطبيق يطلب تنمية مساحة العناوين بدرجة كبيرة، فإن sbrk بدون تخصيص كسول يكون مكلفاً: إذا طلب تطبيق غيغابايت من الذاكرة، يتعين على النواة تخصيص وتصفير 262,144 صفحة فيزيائية بحجم 4096 بايت. يتيح التخصيص الكسول توزيع هذه الكلفة عبر الوقت. ومن جهة أخرى، يتسبب التخصيص الكسول في نفقات إضافية لأخطاء الصفحات، والتي تتضمن انتقالاً بين المستخدم والنواة. يمكن لنظم التشغيل تقليل هذه الكلفة عبر تخصيص دفعة من الصفحات المتتالية لكل خطأ صفحة بدلاً من صفحة واحدة وعبر تخصيص شفرة دخول/خروج النواة لمثل أخطاء الصفحات تلك (رغم أن xv6 لا تفعل أي منهما).

ومن جهة أخرى، عند التعامل مع خطأ صفحة لصفحة مخصصة كسولاً، قد تجد النواة أنه لا تتوفر ذاكرة حرة للتخصيص. في هذه الحالة، لا تجد النواة طريقة سهلة لإرجاع خطأ نفاذ الذاكرة للتطبيق وتكتفي بقتل التطبيق بدلاً من ذلك. بالنسبة للتطبيقات التي تفضل خطأ عند تخصيص فاشل، تتيح xv6 للتطبيق تخصيص الذاكرة بشغف عبر دعوة sbrk مع العلامة SBRK_EAGER.

6.2 الكود البرمجي: التخصيص الكسول

استدعاء النظام sbrk(n) ينمي (أو يقلص إذا كانت n سالبة) حجم ذاكرة العملية بمقدار n بايت، ويرجع بداية المنطقة المخصصة حديثاً (أي الحجم القديم). تنفيذ النواة هو sys_sbrk.

إذا حدد التطبيق SBRK_EAGER، يُنفذ استدعاء النظام بواسطة الدالة growproc. تدعو growproc الدالة uvmalloc. تخصص uvmalloc ذاكرة فيزيائية بـ kalloc، وتصفر الذاكرة المخصصة، وتضيف PTEs لجدول صفحات المستخدم بـ mappages.

إذا خصص التطبيق الذاكرة كسولاً، تكتفي sys_sbrk بزيادة حجم العملية (myproc()->sz) بمقدار n وترجع الحجم القديم؛ لا تخصص ذاكرة فيزيائية ولا تضيف PTEs لجدول صفحات العملية.

عندما تحمل العملية أو تخزن في عنوان افتراضي يفتقر لرسم صالح بجدول الصفحات، سيولد المعالج استثناء خطأ الصفحة (Page-fault exception). تفحص usertrap هذه الحالة وتدعو vmfault لمعالجة خطأ الصفحة. تفحص vmfault يكون العنوان المسبب للخطأ في المنطقة الممنوحة سابقاً بواسطة sbrk، وتخصص صفحة ذاكرة فيزيائية بـ kalloc، وتصفر الصفحة المخصصة، وتضيف PTE لجدول صفحات المستخدم بـ mappages. تضبط xv6 العلامات PTE_U، PTE_R، PTE_W في PTE للصفحة الجديدة. ثم تستأنف usertrap العملية عند التعليمات التي تسببت في الخطأ. نظراً لأن PTE أصبح صالحاً الآن، فإن تعليمة التحميل أو التخزين المعاد تنفيذها ستنفذ بدون خطأ.

إذا أفرغ تطبيق ما ذاكرة باستخدام sbrk، تدعو sys_sbrk الدالة shrinkproc والتي تدعو uvmdealloc. العمل الحقيقي ينفذ بواسطة uvmunmap التي تستخدم walk للعثور على PTEs. نظراً لأن بعض الصفحات قد تكون لم

تُستخدم مطلقاً بواسطة العملية وبالتالي لم تُخصص بـ vmfault مطلقاً، تتجاوز uvmunmap الـ PTES ذات العلامة PTE_V الصفري. إذا كان رسم PTE صالحاً، تدعو uvmunmap الدالة kfree لتحرير الذاكرة الفيزيائية التي تشير إليها. لاحظ أن xv6 تستخدم جدول صفحات العملية ليس فقط لإخبار العتاد بكيفية رسم العناوين الافتراضية للمستخدم، ولكن أيضاً كمسجل وحيد لصفحات الذاكرة الفيزيائية المخصصة لتلك العملية. هذا هو السبب في أن تحرير ذاكرة المستخدم (في uvmunmap) يتطلب فحص جدول صفحات المستخدم.

6.3 العالم الحقيقي: النسخ عند الكتابة (COW Fork)

تستخدم نظم كثيرة (رغم أن xv6 ليست منها) أخطاء الصفحات لتنفيذ النسخ عند الكتابة (Copy-On-Write / COW fork). عد استدعاء النظام fork يكون الابن يشاهد ذاكرة محتواها الأولي متطابق مع ذاكرة الأب في وقت الاستدعاء. إحدى الطرق لتنفيذ هذا هي نسخ كامل ذاكرة الأب لذاكرة فيزيائية مخصصة حديثاً للابن؛ وهذا ما تفعله xv6. النسخ قد يكون بطيئاً، ويكون أكثر كفاءة لو استطاع الابن مشاركة ذاكرة الأب الفيزيائية. تنفيذ مباشر لهذا لن يعمل، نظراً لأنه يتسبب في اضطراب تنفيذ الأب والابن لبعضهما البعض بكتابتهما في المكس والهيبت المشتركين. النسخ عند الكتابة يتسبب في جعل الأب والابن يتشاركان الذاكرة الفيزيائية بأمان عبر الاستخدام المناسب لأدوات جداول الصفحات وأخطاء الصفحات. الخطة الأساسية هي أن يتشارك الأب والابن أولاً كافة الصفحات الفيزيائية، ولكن مع رسم كل منهما لها ك للقراءة فقط (مع تفسير العلامة PTE_W). يمكن للأب والابن عندئذ القراءة من الذاكرة الفيزيائية المشتركة. إذا كتب أحدهما صفحة مشتركة، يولد معالج RISC-V استثناء خطأ صفحة. النواة الداعمة لـ COW تستجيب عبر تخصيص صفحة جديدة من الذاكرة الفيزيائية ونسخ الصفحة المشتركة في تلك الصفحة الجديدة. ثم تغير النواة الـ PTE المناظر في جدول صفحات العملية المسببة للخطأ للإشارة للنسخة والسماح بالكتابة والقراءة، ثم تستأنف العملية المسببة للخطأ عند التعليمة التي تسببت في الخطأ. نظراً لأن الـ PTE يتيح الكتابة الآن، فإن تعليمة التخزين المعاد تنفيذها ستنفذ بدون خطأ، وتعديل نسخة خاصة من الصفحة بدلاً من الصفحة المشتركة.

يتطلب النسخ عند الكتابة حوسبة ودفتر تتبع للمساعدة في القرار بخصوص متى يمكن تحرير الصفحات الفيزيائية، نظراً لأن كل صفحة يمكن الإشارة إليها بواسطة عدد متغير من جداول الصفحات اعتماداً على تاريخ الاستدعاءات لـ fork وأخطاء الصفحات و exec و exit. يتيح هذا الدفتر تحسناً هاماً: إذا تعرضت عملية لخطأ صفحة تخزين وكانت الصفحة الفيزيائية مشاراً إليها فقط من جدول صفحات تلك العملية، فلا حاجة للنسخ.

يجعل النسخ عند الكتابة fork أسرع، نظراً لأن fork لا تحتاج لنسخ الذاكرة. بعض الذاكرة سيتعين نسخها لاحقاً عند الكتابة، ولكن غالباً ما تكون معظم الذاكرة لا تحتاج للنسخ مطلقاً. مثال شائع هو fork متبوعة بـ exec: يضع صفحات قد تُكتب بعد fork ولكن بعد ذلك يفرغ exec الابن معظم الذاكرة الموروثة من الأب. النسخ عند الكتابة بـ fork يلغي الحاجة لنسخ هذه الذاكرة مطلقاً. علاوة على ذلك فإن COW fork شفاف: لا حاجة لأية تعديلات على التطبيقات لكي تستفيد منه.

6.4 العالم الحقيقي: استدعاء الصفحات عند الطلب (Demand Paging)

ميزة أخرى واسعة الاستخدام تستغل أخطاء الصفحات هي استدعاء الصفحات عند الطلب (Demand paging). عند استدعاء النظام exec تحمل xv6 كافة نصوص وبيانات التطبيق في الذاكرة قبل بدء التطبيق. نظراً لأن التطبيقات قد تكون ضخمة والقراءة من القرص تستغرق وقتاً، فإن كلفة البدء هذه قد تكون ملحوظة للمستخدمين. لتقليل وقت البدء، لا تحمل النواة الحديثة الملف التنفيذي أولاً في الذاكرة، وتكتفي بإنشاء جدول صفحات المستخدم مع تعليم كافة الـ PTES كغير صالحة. تبدأ النواة تشغيل البرنامج؛ وفي كل مرة يستخدم فيها البرنامج صفحة للمرة الأولى، يقع خطأ صفحة، واستجابة لذلك تقرأ النواة محتوى الصفحة من القرص وترسمه في مساحة عناوين المستخدم. مثل COW fork والتخصيص الكسول، يمكن للنواة تنفيذ هذه الميزة بأسلوب شفاف للتطبيقات.

البرامج المنفذة على حاسوب قد تحتاج ذاكرة أكثر مما يحوزها الحاسوب من RAM. للتعامل مع ذلك بمرونة، قد تنفذ نظام التشغيل تحويل الصفحات للقرص (Paging to disk). الفكرة هي تخزين جزء فقط من صفحات المستخدم في RAM، وتخزين الباقي على القرص في منطقة تحويل الصفحات (Paging area). تعلم النواة الـ PTES المناظرة للذاكرة المخزنة في منطقة التبدل (وبالتالي ليست في RAM) كغير صالحة. إذا حاول تطبيق استخدام إحدى الصفحات التي جرى تحويلها للقرص (paged out)، سيتعرض التطبيق لخطأ صفحة، وتجب استعادة

الصفحة (paged in): سيعالج معالج المصيدة بالنواة الأمر عبر تخصيص صفحة RAM فيزيائية، وقراءة الصفحة من القرص في RAM، وتعديل الـ PTE المناظر للإشارة لـ RAM.

ماذا يحدث إذا احتاجت صفحة لاستعادتها للقرص لكن لا تتوفر RAM فيزيائية حرة؟ في هذه الحالة يجب على النواة أولاً تحرير صفحة فيزيائية عبر تحويلها للقرص أو إخلائها (evicting) لمنطقة تحويل الصفحات على القرص، وتعليم الـ PTES المشاركة لتلك الصفحة الفيزيائية كغير صالحة. الإخلاء مكلف، لذا فإن تحويل الصفحات يؤدي بأفضل شكل إذا كان غير متكرر: إذا كانت التطبيقات تستخدم فقط طقماً فرعياً من صفحات ذاكرتها واتحاد الأطقم الفرعية يتسع في RAM. هذه الخاصية يُشار إليها غالباً بـ حوز مكانية مرجعية جيدة. كما هو الحال مع تقنيات الذاكرة الافتراضية، تنفذ النوى عادة تحويل الصفحات للقرص بأسلوب شفاف للتطبيقات.

تنفذ الحاسوبيات غالباً مع ذاكرة فيزيائية حرة قليلة أو معدومة، بغض النظر عن مقدار RAM الذي يوفر العتاد. على سبيل المثال، يوفر موفرو السحابة مضاعفة لعملاء متعددين على آلة واحدة لاستخدام عتادهم بكفاءة كلفة. كمثال آخر، ينفذ المستخدمون تطبيقات كثيرة على الهواتف الذكية في مقدار صغير من الذاكرة الفيزيائية. في مثل هذه الإعدادات فإن تخصيص صفحة قد يتطلب أولاً إخلاء صفحة قائمة. وهكذا عندما تكون الذاكرة الفيزيائية الحرة شحيحة، فإن التخصيص يكون مكلفاً.

التخصيص الكسول واستدعاء الصفحات عند الطلب نافعان بصفة خاصة عندما تكون الذاكرة الحرة شحيحة وتستخدم البرامج بفعالية جزءاً فقط من ذاكرتها المخصصة. يمكن لهذه التقنيات أيضاً تجنب العمل المهدور عندما تُخصص صفحة أو تُحمل ولكنها إما لا تُستخدم مطلقاً أو تُحلى قبل الاستخدام.

6.5 العالم الحقيقي: الملفات المربوطة بالذاكرة (Memory-Mapped Files)

الميزات الأخرى التي تجمع تحويل الصفحات واستثناءات أخطاء الصفحات تتضمن توسيع المكدرات تلقائياً و الملفات المربوطة بالذاكرة (Memory-mapped files)، وهي ملفات يرسمها البرنامج في مساحة عناوينه باستخدام استدعاء النظام mmap لكي يتمكن البرنامج من قراءتها وكتابتها باستخدام تعليمات التحميل والتخزين.

6.6 تمارين

1.

اكتب برنامج مستخدم ينمي مساحة عناوينه ببايت واحد عبر دعوة `sbrk(1)`. نفذ البرنامج وافحص جدول الصفحات للبرنامج قبل الاستدعاء لـ `sbrk` وبعد الاستدعاء لـ `sbrk`. كم مساحة خصصت النواة؟ ماذا يحوي الـ PTE للذاكرة الجديدة؟

2.

نفذ النسخ عند الكتابة (COW fork).

3.

نفذ `mmap`.

7 المقاطعات وبرامج تشغيل الأجهزة

يُعرف برنامج تشغيل الجهاز (Driver) بأنه الشفرة في نظام التشغيل التي تدير جهازاً محدداً؛ فهي تضبط عتاد الجهاز، وتخير الجهاز بتنفيذ العمليات، وتتعامل مع المقاطعات المترتبة، وتتفاعل مع العمليات التي تستخدم الجهاز. قد تكون شفرة برنامج التشغيل دقيقة لأن برنامج التشغيل ينفذ بالتوازي مع الجهاز، وغالباً بالتوازي مع العمليات التي تستخدم الجهاز. بالإضافة لذلك، يجب على برنامج التشغيل فهم واجهة عتاد الجهاز، والتي قد تكون معقدة وضعيفة التوثيق.

الأجهزة التي تحتاج اهتماماً من نظام التشغيل يمكن عادةً تهيئتها لتوليد مقاطعات، والتي تُعد نوعاً من المصايد. تتعرف شفرة معالجة المصايد بالنواة على الوقت الذي أحدث فيه جهاز ما مقاطعة وتدعو معالج المقاطعة لبرنامج التشغيل؛ بـ `xv6` يقع هذا التوجيه في `devintr`.

عديد برامج تشغيل الأجهزة تنفذ الشفرة في سياقين: نصف سفلي (Bottom half) ينفذ في خيط النواة لعملية ما، و نصف علوي (Top half) ينفذ في وقت المقاطعة. يُدعى النصف السفلي عبر استدعاءات نظام مثل `read` و `write` التي تريد من الجهاز تنفيذ إدخال/إخراج. هذه الشفرة قد تطلب من العتاد بدء عملية (مثلاً، طلب قراءة

كتلة من القرص)؛ ثم تنتظر الشفرة إكمال العملية. في النهاية يكمل الجهاز العملية ويولد مقاطعة. يعمل معالج المقاطعة ببرنامج التشغيل ك نصف علوي، فيكتشف أية عملية اكتملت، ويوقظ عملية منتظرة إذا كان ذلك مناسباً، ويخبر العتاد ببدء العمل في العملية التالية إن وجدت.

7.1 الكود البرمجي: مدخلات لوحة التحكم

برنامج تشغيل لوحة التحكم توضيح بسيط لتركيبة برنامج التشغيل. يتقبل برنامج تشغيل لوحة التحكم الحروف المكتوبة بواسطة إنسان، عبر عتاد المنفذ التسلسلي UART المربوط بـ RISC-V. يجمع برنامج تشغيل لوحة التحكم سطرًا من المدخلات في الوقت الواحد، معالجاً الحروف الخاصة كـمسح الحرف والتحكم-u. تستخدم عمليات المستخدم، مثل موجه الأوامر، استدعاء النظام read لجلب أسطر المدخلات من لوحة التحكم. عندما تكتب مدخلات لـ xv6 في QEMU، تُسلم ضربات مفاتيحك لـ xv6 عن طريق عتاد UART المحاكى بـ QEMU.

عتاد UART الذي تتحدث معه برنامج التشغيل هو شريحة 16550 محاكاة بواسطة QEMU. على حاسوب حقيقي فإن 16550 ستدير رابطاً تسلسلياً RS232 متصلاً بطرفية أو حاسوب آخر. عند تنفيذ QEMU تكون متصلة في لوحة مفاتيحك وشاشتك.

يظهر عتاد UART للبرمجيات كقطع من سجلات التحكم المربوطة بالذاكرة (Memory-mapped). أي توجد بعض العناوين الفيزيائية المتصلة بجهاز UART، بحيث تتفاعل التحويلات والتخزينات مع عتاد الجهاز بدلاً من RAM. تبدأ العناوين المربوطة بالذاكرة لـ UART عند 0x10000000 أو UART0. توجد حفنة من سجلات تحكم UART بعرض بايت واحد لكل منها. إزاحتها من UART0 معرفة في الشفرة. على سبيل المثال، يحوي المسجل LSR بتات تشير إلى ما إذا كانت حروف مدخلة تنتظر أن تقرأ بواسطة برنامج التشغيل. هذه الحروف (إن وجدت) تكون متاحة للقراءة من المسجل RHR. في كل مرة يُقرأ فيها حرف، يحذفه عتاد UART من طابور FIFO داخلي للحروف المنتظرة، ويصقّر بت «الجهوزية» في LSR عندما يفرغ طابور FIFO. للإرسال يكتب برنامج التشغيل بايتاً للمسجل THR مما يتسبب في جعل UART يضيف البايت لطابور FIFO من البايتات التي سيرسلها UART على الرابط التسلسلي RS232. عتاد الإرسال والاستقبال بـ UART مستقلان بدرجة كبيرة عن بعضهما البعض.

تدعو main بـ xv6 الدالة consoleinit لتهيئة عتاد UART. تضبط هذه الشفرة UART لتوليد مقاطعة استقبال عندما يستقبل UART كل بايت مدخلات، ومقاطعة إكمال الإرسال (Transmit complete) في كل مرة ينهي فيها UART إرسال بايت مخرجات.

يقرأ موجه أوامر xv6 من لوحة التحكم عن طريق واصف ملف مفتوح بواسطة init.c. تنفذ الاستدعاءات لاستدعاء النظام read طريقها عبر النواة لـ consleread. تنتظر consleread وصول المدخلات (عبر المقاطعات) وتخزينها مؤقتاً في cons.buf، وتنسخ المدخلات لمساحة المستخدم، و(بعد وصول سطر كامل) ترجع للعملية في مساحة المستخدم. إذا لم يكتب المستخدم سطرًا كاملاً بعد، فإن أية عمليات قارئة ستنتظر في الاستدعاء sleep (يشرح الفصل الفصل 10 تفاصيل sleep).

عندما يكتب المستخدم حرفاً، يطلب عتاد UART من RISC-V توليد مقاطعة، مما يفعل معالج مصاد xv6. يدعو معالج المصاد الدالة devintr التي تنظر في مسجل RISC-V المسجل scause لاكتشاف أن المقاطعة من جهاز خارجي. ثم تطلب من وحدة عتادية تُسمى PLIC إخبارها بأي جهاز أحدث المقاطعة. إذا كان UART، تدعو devintr الدالة uartintr.

تقرأ uartintr أية حروف مدخلات منتظرة من عتاد UART وتسلمها لـ consoleintr؛ لا تنتظر الحروف نظراً لأن المدخلات المستقبلية ستولد مقاطعة جديدة. مهمة consoleintr هي تجميع حروف المدخلات في cons.buf حتى يصل سطر كامل. تعامل consoleintr مسح الحرف وبضعة حروف أخرى معاملة خاصة. عندما يصل سطر جديد، توقظ consoleintr القارئ المنتظر بـ consleread (إن وجد).

بمجرد استيقاظها، ستشاهد consleread سطرًا كاملاً في cons.buf وتنسخه لمساحة المستخدم وترجع (عبر آليات استدعاءات النظام) لمساحة المستخدم.

نمط ينبغي ملاحظته هو فك ارتباط نشاط الجهاز عن نشاط العملية عبر التخزين المؤقت والمقاطعات. يمكن لبرنامج تشغيل لوحة التحكم معالجة المدخلات حتى وإن لم تكن أية عملية تنتظر قراءتها؛ القراءة اللاحقة ستشاهد المدخلات. فك الارتباط هذا يمكنه زيادة الأداء عبر السماح للعمليات بالتنفيذ بالتوازي مع إدخال/إخراج الجهاز،

ويكون هاماً بصفة خاصة عندما يكون الجهاز بطيئاً (كما مع UART) أو يحتاج اهتماماً فورياً (كما مع إعطاء صدى للحروف المكتوبة). تُسمى هذه الفكرة أحياناً بـ توازي الإدخال/الإخراج (I/O concurrency).

7.2 الكود البرمجي: مخرجات لوحة التحكم

استدعاء النظام write على واصف ملف متصل بلوحة التحكم يصل لـ consolewrite التي تنسخ دفعات من البايتات من مساحة المستخدم وتسلم كل دفعة لـ uartwrite. قبل كتابة كل بايت في مسجل THR بـ UART، يجب على uartwrite الانتظار لكي يصبح UART متجهزاً لتقبل مخرجات أكثر. نظراً لأن UART بطيء نسبياً، تنتظر uartwrite باستخدام sleep (والتي تنازل عن المعالج) بدلاً من حلقة تكرارية مشغولة. عندما ينهي UART إرسال أحدث بايت، يولد مقاطعة. روتين المقاطعة uartintr يتعاون مع uartwrite: تضبط الأخيرة العلامة tx_busy عندما ترسل كل حرف، وتنتظر لتصفير العلامة؛ ويصفّر روتين المقاطعة tx_busy ويوقظ خيط الكتابة عندما يشير UART بكونه متجهزاً لمخرجات أكثر.

7.3 التزامن في برامج تشغيل الأجهزة

قد تكون لاحظت الاستدعاءات لـ acquire في consolaread و consoleintr. تستحوذ هذه الاستدعاءات على قفل يحمي هياكل بيانات برنامج تشغيل لوحة التحكم من الوصول المتزامن. توجد ثلاثة مخاطر تزامن هنا: عمليتان على معالجين مختلفين قد تدعوان consolaread في الوقت نفسه؛ العناد قد يطلب من معالج تسليم مقاطعة لوحة تحكم (في الواقع UART) بينما ذلك المعالج ينفذ بالفعل في consolaread؛ والعناد قد يسلم مقاطعة لوحة تحكم على معالج مختلف بينما consolaread تنفذ. يشرح الفصل الفصل 8 كيفية استخدام الأقفال لضمان ألا تقود هذه المخاطر لنتائج غير صحيحة.

طريقة أخرى يتطلب بها التزامن عناية في برامج التشغيل هي أن عملية ما قد تكون منتظرة مدخلات من جهاز، لكن مقاطعة الإشارة بوصول المدخلات قد تصل عندما تنفذ عملية مختلفة (أو لا عملية على الإطلاق). وهكذا لا يُسمح لمعالجات المقاطعة بالتفكير في العملية أو الشفرة التي قطعها. على سبيل المثال، لا يمكن لمعالج المقاطعة استدعاء copyout بأمان مع جدول صفحات العملية الحالية. تنفذ معالجات المقاطعة عادة عملاً قليلاً نسبياً (مثلاً، مجرد نسخ بيانات المدخلات لحاوي)، وتوقظ شفرة النصف السفلي للقيام بالباقي.

7.4 مقاطعات الميقاتي

تستخدم xv6 مقاطعات الميقاتي للتحكم في فكرتها عن الوقت الحالي والتبديل بين العمليات المحصورة بالحوسبة. تأتي مقاطعات الميقاتي من عتاد الساعة المربوط بكل معالج RISC-V. تبرمج xv6 عتاد الساعة لكل معالج لقطع المعالج دورياً.

تضبط الشفرة بـ start.c بعض بنات التحكم التي تتيح لنمط المشرف الوصول لسجلات تحكم الميقاتي، ثم تطلب أول مقاطعة ميقاتي. يحوي مسجل التحكم time عدداً يزيد العناد بمعدل ثابت؛ يعمل هذا كمفهوم للوقت الحالي. يحوي المسجل stimecmp وقتاً سيولد عنده المعالج مقاطعة ميقاتي؛ ضبط stimecmp للقيمة الحالية لـ time مضافاً إليها x س يحدد مقاطعة x وحدة زمنية في المستقبل. بالنسبة لمحاكاة RISC-V بـ qemu فإن 1000000 وحدة زمنية هي تقريباً خمس ثانية.

تصل مقاطعات الميقاتي عبر usertrap أو kerneltrap و devintr مثل مقاطعات الأجهزة الأخرى. تصل مقاطعات الميقاتي مع البنات الأدنى لـ scause مضبوطة لخمسة؛ تكتشف devintr بـ trap.c هذه الحالة وتدعو clockintr. تزيد الدالة الأخيرة ticks مما يتيح للنواة تتبع مرور الوقت. الزيادة تقع على معالج واحد فقط لتجنب مرور الوقت أسرع في حالة وجود معالجات متعددة. توقظ clockintr أية عمليات منتظرة عند استدعاء النظام pause وتجدول مقاطعة الميقاتي التالية عبر كتابة stimecmp.

ترجع devintr الرقم 2 لمقاطعة الميقاتي لتشير لـ kerneltrap أو usertrap بكونهما ينبغي أن تدعوان yield لكي تتمكن المعالجات من التقاسم بالمشاركات الزمنية بين العمليات القابلة للتشغيل.

حقيقة أن شفرة النواة يمكن أن تُقطع بمقاطعة ميقاتي تجبر تبديل سياق عبر yield هي جزء من السبب في أن الشفرة المبكرة بـ usertrap حذرة في حفظ حالة مثل sepc قبل تفعيل المقاطعات. تبديلات السياق هذه تعني أيضاً أن شفرة النواة يجب أن تُكتب مع العلم بأنها قد تنتقل من معالج لآخر دون تحذير.

7.5 العالم الحقيقي

تتيح xv6 مثل نظم تشغيل كثيرة المقاطعات وحتى تبديلات السياق (عبر `yield`) أثناء التنفيذ في النواة. السبب في هذا هو الاحتفاظ بأوقات استجابة سريعة أثناء استدعاءات النظام المعقدة التي تنفذ لوقت طويل. ومع ذلك فكما لوحظ أعلاه فإن السماح بالمقاطعات في النواة هو مصدر لبعض التعقيد؛ ونتيجة لذلك فإن بضع نظم تشغيل تتيح المقاطعات فقط أثناء تنفيذ شفرة المستخدم.

دعم كافة الأجهزة على حاسوب معهود بكامل المجد هو عمل ضخم، نظراً لوجود أجهزة كثيرة والميزات كثيرة في الأجهزة ويمكن للبروتوكول بين الجهاز وبرنامج التشغيل أن يكون معقداً وضعيف التوثيق. في عديد نظم التشغيل تشكل برامج التشغيل شفرة أكثر من النواة الجوهرية.

يجلب برنامج تشغيل UART البيانات بائتاً في الوقت الواحد عبر قراءة سجلات تحكم UART؛ يُسمى هذا النمط بـ الإدخال/الإخراج المبرمج (Programmed I/O) نظراً لأن البرمجيات تقود حركة البيانات. الإدخال/الإخراج المبرمج بسيط، ولكنه بطيء جداً للاستخدام في معدلات بيانات عالية. الأجهزة التي تحتاج نقل بيانات كثيرة بسرعة عالية تستخدم عادة الوصول المباشر للذاكرة (Direct Memory Access / DMA). عتاد أجهزة DMA يكتب مباشرة البيانات القادمة لـ RAM، ويقرأ البيانات الخارجة من RAM. أقراص النظم الحديثة وأجهزة الشبكة تستخدم DMA. برنامج تشغيل لجهاز DMA سيتجهز البيانات في RAM ثم يستخدم كتابة واحدة لسجل تحكم لإخبار الجهاز بمعالجة البيانات المجيزة.

تكون المقاطعات منطقية عندما يحتاج جهاز اهتماماً في أوقات غير متوقعة، وليس بكثرة شديدة. لكن المقاطعات تحوز نفقات معالج عالية. وهكذا فإن الأجهزة عالية السرعة، مثل متحكمات الشبكة والأقراص، تستخدم حيلة تقلل الحاجة للمقاطعات. حيلة واحدة هي توليد مقاطعة واحدة لكامل دفعة من الطلبات القادمة أو الخارجة. حيلة أخرى هي أن يعطل برنامج التشغيل المقاطعات تماماً، وبفحص الجهاز دورياً لرؤية ما إذا كان يحتاج اهتماماً. تُسمى هذه التقنية بـ الاقتراع (Polling). يكون الاقتراع منطقياً إذا كان الجهاز ينفذ عمليات بمعدل عال، ولكنه يضع وقت المعالج إذا كان الجهاز خاملاً معظم الوقت. بعض برامج التشغيل تبدل ديناميكياً بين الاقتراع والمقاطعات اعتماداً على حمل الجهاز الحالي.

ينسخ برنامج تشغيل UART البيانات القادمة أولاً لحاوي في النواة، ثم لمساحة المستخدم. هذا منطقي في معدلات البيانات المنخفضة، لكن مثل هذا النسخ المزدوج يمكنه تقليل الأداء بدرجة كبيرة للأجهزة التي تولد أو تستهلك بيانات بسرعة كبيرة. بعض نظم التشغيل قادرة على نقل البيانات مباشرة بين حواشي مساحة المستخدم وعتاد الأجهزة، غالباً مع DMA.

كما ذكر في الفصل الفصل 2، تظهر لوحة التحكم للتطبيقات كملف عادي، وتقرأ التطبيقات المدخلات وتكتب المخرجات باستخدام استدعاءات النظام `read` و `write`. قد ترغب التطبيقات في التحكم في جوانب من الجهاز لا يمكن التعبير عنها عبر استدعاءات نظام الملفات القياسية (مثلاً، تفعيل/تعطيل التخزين المؤقت للأسطر في برنامج تشغيل لوحة التحكم). توفر نظم تشغيل Unix استدعاء نظام `ioctl` لمثل هذه الحالات.

بعض استخدامات الحاسوبات تتطلب استجابات «لحظية» للحوادث الخارجية: استجابات مضمون وقوعها في وقت محدود. على سبيل المثال في النظم الحرجة أمنياً فإن تفويت موعد نهائي قد يقود لكوارث. xv6 ليست مناسبة للإعدادات لحظية الاستجابة. من بين أمور أخرى فإن جدول xv6 لا يأخذ في الحسبان المواعيد النهائية لحظية الاستجابة عندما يقرر أية عملية سينفذ تالياً، وتحوز xv6 مسارات شفرة نواة طويلة مع تعطيل المقاطعات، لكي لا تتمكن من الاستجابة للمقاطعات سريعاً. نظام التشغيل لحظي الاستجابة يجب ليس فقط إصلاح هذه المشاكل بل أيضاً أن يُهيكل بطريقة تتيح تحليل أوقات الاستجابة في الحالة الأسوأ.

7.6 تمارين

1.

عدل `uart.c` لعدم استخدام المقاطعات على الإطلاق. قد تحتاج لتعديل `console.c` أيضاً.

2.

أضف برنامج تشغيل لبطاقة شبكة Ethernet.

8 الأفقال

معظم النوى، بما فيها xv6، تداخل تنفيذ الأنشطة المتعددة. أحد مصادر التداخل هو عتاد متعدد المعالجات: حواسيب بمعالجات متعددة تنفذ بأسلوب مستقل، مثل معالج RISC-V الخاص بـ xv6. تتشارك هذه المعالجات المتعددة الذاكرة العشوائية الفيزيائية RAM، وتستغل xv6 المشاركة للحفاظ على هياكل بيانات النواة التي تقرأها وتكتبها كافة المعالجات. هذه المشاركة تطرح احتمالية قراءة معالج ما لهيكل بيانات بينما معالج آخر في منتصف تحديثه، أو حتى تحديث معالجات متعددة لنفس البيانات في الوقت نفسه؛ وبدون تصميم حذر يُرجح أن يتسبب هذا الوصول المتوازي في إعطاء نتائج غير صحيحة أو هيكل بيانات مدمر. حتى على نظام أحادي المعالج، قد تبدل النواة المعالج بين عدد من الخيوط، متنسبة في جعل تنفيذها متداخلاً. أخيراً، فإن معالج مقاطعة جهاز الذي يعدل نفس البيانات كشفرة قابلة للقطع قد يلف البيانات إذا وقعت المقاطعة في نفس اللحظة التي تعدل فيها الشفرة المنقطعة البيانات أيضاً. نشر كلمة التزامن (Concurrency) للحالات التي تتداخل فيها تدفقات تعليمات متعددة، بفضل التوازي متعدد المعالجات، تبديل الخيوط، أو المقاطعات.

النوى ملأى ببيانات قابلة للوصول المتزامن. على سبيل المثال، معالجان يمكنهما الاستدعاء المتزامن لـ `kalloc` في الوقت نفسه، وبالتالي السحب المتزامن من رأس القائمة الحرة. يحب مصممو النوى السماح بكثرة من التزامن، نظراً لأنه يزيد الأداء والاستجابة. ومع ذلك يتعين عليهم إقناع أنفسهم بالصحة على الرغم من التزامن. الاستراتيجيات الهادفة للصحة تحت التزامن، والمجردات الداعمة لها، تُسمى تقنيات التحكم في التزامن (Concurrency control).

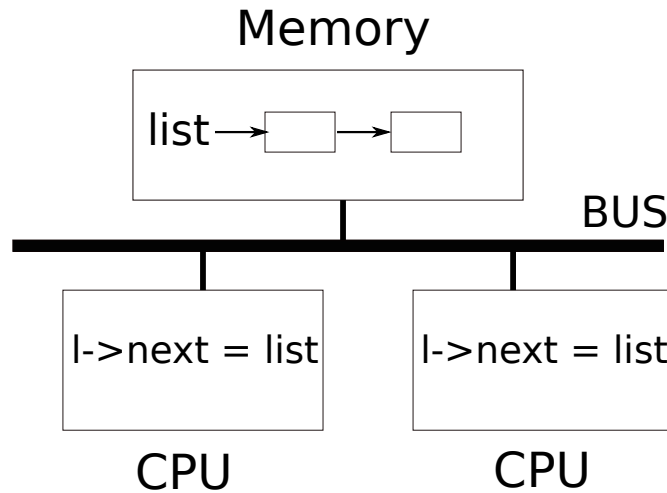
يركز هذا الفصل على تقنية تحكم في التزامن واسعة الاستخدام: القفل (Lock). عاين، فيما يلي كيف يبدو استخدام القفل في نواة xv6:

```
struct spinlock lk
;...
;acquire(&lk)
... قراءة أو كتابة بعض البيانات المشتركة ...
;release(&lk)
```

يوفر القفل الاستبعاد المتبادل، متأكداً من أن معالجاً واحداً فقط في الوقت الواحد يمكنه احتجاز القفل. إذا ربط المبرمج قفلاً مع كل عنصر بيانات مشترك، وكانت الشفرة تحتجز دائماً القفل المرتبط عند استخدام عنصر ما، فإن العنصر سيُستخدم بواسطة معالج واحد فقط في الوقت الواحد. في هذه الحالة، نقول إن القفل يحمي عنصر البيانات. على الرغم من أن القفل آلية تحكم في التزامن سهلة الفهم، إلا أنه قد يحد من الأداء عبر تسلسل العمليات المتزامنة على البيانات التي يحميها.

يشرح بقية هذا الفصل لماذا تحتاج xv6 الأفقال، كيف تنفذها xv6 وكيف تستخدمها.

8.1 حالات التنافس



شكل 9: بنية SMP مبسطة.

كمثال على لماذا نحتاج الأفعال، انظر لعمليتين بحوزتهما أبناء خارجون تدعوان استدعاء النظام wait على معالجين مختلفين. تفرغ wait ذاكرة الابن. وهكذا على كل معالج، استدعو النواة kfree لتحرير صفحات ذاكرة الأبناء. يحتفظ مخصص النواة بقائمة ترابطية للصفحات الحرة: تسحب kalloc () صفحة ذاكرة من القائمة، وتدفع kfree () صفحة في القائمة. لأداء أفضل، قد نأمل أن الاستدعاءات لـ kfree لعمليتي الأب تنفذان بالتوازي دون حاجة أي منهما للانتظار للأخرى، لكن هذا لن يكون صحيحاً بناءً على تنفيذ kfree بـ xv6.

يوضح الشكل شكل 9 الإعداد بتفصيل أكثر: القائمة الترابطية للصفحات الحرة تتواجد في الذاكرة المشتركة بواسطة المعالجين، واللذان يتعاملان مع القائمة باستخدام تعليمات التحميل والتخزين. (في الواقع المعالجات تحوز مخازن مؤقتة، لكن منطقياً تنصرف النظم متعددة المعالجات كما لو كانت هناك ذاكرة واحدة مشتركة). لو لم تكن هناك طلبات متزامنة، لكان بمقدورك تنفيذ عملية دفع لداخل قائمة كالتالي:

```

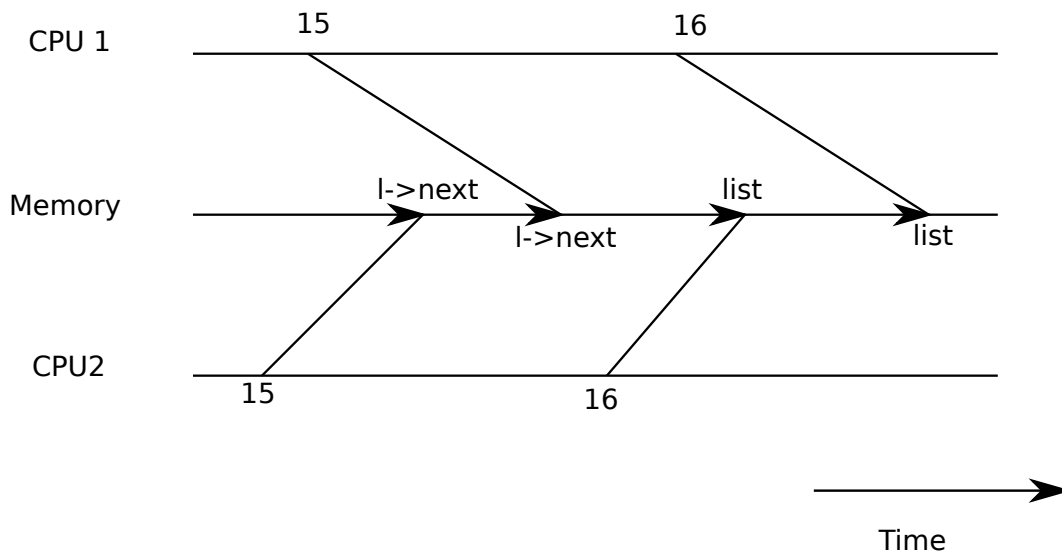
    } struct element
      ;int data
    ;struct element *next
    ;{

    ;struct element *list = 0

    void
    push(int data)
    {
    ;struct element *l

    ;l = malloc(sizeof *l)
    ;l->data = data
    ;l->next = list
    ;list = l
    {

```



شكل 10: مثال لـ حالة تنافس.

هذا التنفيذ صحيح لو نُفذ بمعزل. ومع ذلك، الشفرة ليست صحيحة لو تنفذت أكثر من نسخة بالتزامن. لو نفذ معالجان push في الوقت نفسه، قد ينفذ كلاهما السطر `l->next = list` كما يوضح الشكل شكل 9، قبل أن ينفذ أي منهما السطر `list = l` والذي ينتج عنه نتيجة غير صحيحة كما يوضح الشكل شكل 10. سيتواجد عندئذ عنصران قائمة مع ضبط next في كلاهما لنفس القيمة السابقة لـ list. عندما يقع التعيينان لـ list في السطر الأخير، فإن الثاني سيكتب فوق الأول؛ والعنصر المتضمن في التعيين الأول سيضيع.

التحديث المفقود بالسطر الأخير هو مثال على حالة تنافس (Race). حالة التنافس هي موقف يُوصل فيه لموقع ذاكرة بالتزامن، ويكون وصول واحد على الأقل هو كتابة. غالباً ما تكون حالة التنافس علامة على خطأ برمجي، إما تحديث مفقود (إذا كانت الوصلات كتابات) أو قراءة لهيكل بيانات غير مكتمل التحديث. نتيجة حالة التنافس تعتمد على شفرة الآلة المولدة بواسطة المترجم، التوقيت للمعالجين المتضمنين، وكيفية ترتيب عملياتهما للذاكرة بواسطة نظام الذاكرة، مما قد يجعل الأخطاء الناتجة عن التنافس صعبة التكرار والاستكشاف. على سبيل المثال إضافة عبارات طباعة أثناء استكشاف أخطاء push قد يغير توقيت التنفيذ بدرجة كافية لجعل التنافس يختفي.

الطريقة المعهودة لتجنب التنافس هي استخدام قفل. تضمن الأفقال الاستبعاد المتبادل (Mutual exclusion)، لكي يتمكن معالج واحد فقط في الوقت الواحد من تنفيذ الأسطر الحساسة من push؛ هذا يجعل السيناريو أعلاه مستحيلًا. النسخة المقفولة بأسلوب صحيح من الشفرة أعلاه تضيف مجرد بضعة أسطر:

```
;struct element *list = 0
;struct lock listlock

void
push(int data)
{
    ;struct element *l
    ;l = malloc(sizeof *l)
    ;l->data = data
```

```

;acquire(&listlock)
;lk->next = list
;list = lk
;release(&listlock)
{

```

متتالية التعليمات بين acquire و release تُسمى غالباً بـ المقطع الحرج (Critical section). يُقال إن القفل يحمي list.

عندما نقول إن قفلاً يحمي بيانات، فنحن نعني بفعالية أن القفل يحمي بعض طقم الثوابت الملازمة التي تنطبق على البيانات. الثوابت الملازمة هي خصائص لهياكل البيانات المحافظ عليها عبر العمليات. عادةً ما يعتمد السلوك الصحيح لعملية ما على كون الثوابت الملازمة صحيحة عندما تبدأ العملية. قد تخرق العملية الثوابت الملازمة مؤقتاً ولكن يجب عليها إعادة إرثائها قبل الانتهاء. على سبيل المثال، في حالة القائمة الترابطية فإن الثابت الملازم هو أن list تشير للعنصر الأول في القائمة وأن الحقل next لكل عنصر يشير للعنصر التالي. تنفيذ push يخرق هذا الثابت الملازم مؤقتاً؛ في السطر list = lk->next يشير lk لعنصر القائمة التالي لكن list لا تشير لـ lk بعد. التنافس الذي فحصناه أعلاه وقع لأن معالجاً ثانياً نفذ شفرة اعتمدت على ثوابت القائمة الملازمة بينما كانت (مؤقتاً) مخروقة. الاستخدام السليم للقفل يضمن أن معالجاً واحداً فقط في الوقت الواحد يمكنه العمل على هيكل البيانات في المقطع الحرج، لكي لا ينفذ أي معالج عملية لـ هيكل البيانات عندما تكون ثوابته الملازمة غير صالحة.

يمكنك التفكير في القفل كمتسلسل للمقاطع الحرجة المتزامنة لكي تنفذ واحدة في الوقت الواحد، وبذلك تحافظ على الثوابت الملازمة (بافتراض كون المقاطع الحرجة صحيحة بمعزل). يمكنك أيضاً التفكير في المقاطع الحرجة المحروسة بنفس القفل بكونها ذرية فيما يتعلق ببعضها البعض، لكي تشاهد كل منها فقط الطقم الكامل للتغيرات من المقاطع الحرجة الأكبر، ولا تشاهد مطلقاً تحديثات مكتملة جزئياً.

على الرغم من نفعها للصحة، فإن الأقفال تقيد بطبيعتها الأداء. على سبيل المثال لو دعت عمليتان kfree بالتزامن، فإن الأقفال ستسلسل المقطعين الحرجين، لكي لا توجد فائدة من تشغيلهما على معالجات مختلفة. نقول إن عمليات متعددة تتضارب إذا أرادت نفس القفل في الوقت نفسه، أو إن القفل يتعرض لـ تنافس (Contention). التحدي الرئيسي في تصميم النواة هو تجنب التنافس على الأقفال سعيًا للتوازي. تفعل xv6 القليل من ذلك، لكن النوى المعقدة تنظم هياكل البيانات والخوارزميات تحديداً لتجنب التنافس على الأقفال. في مثال القائمة، قد تحتفظ النواة بـ قائمة حرة منفصلة لكل معالج وتلمس فقط القائمة الحرة لمعالج آخر إذا كانت قائمة المعالج الحالي فارغة ويجب سرقة ذاكرة من معالج آخر. حالات الاستخدام الأخرى قد تتطلب تصاميم أكثر تعقيداً.

موقع الأقفال هام أيضاً للأداء. على سبيل المثال سيكون من الصحيح نقل acquire لأبكر في push قبل الاستدعاء لـ malloc. لكن هذا سيقطع الأداء لأن الاستدعاءات لـ malloc ستصبح مسلسلة عندئذ. يوفر القيد «استخدام الأقفال» أدناه بعض الإرشادات لإدراج استدعاءات acquire و release.

8.2 الكود البرمجي: الأقفال

يُرجى قراءة kernel/spinlock.h و kernel/spinlock.c.

تحوّز xv6 نوعين من الأقفال: أقفال الدوران (Spinlocks) وأقفال النوم (Sleep-locks). سنبدأ بأقفال الدوران. تمثل قفل الدوران كـ struct spinlock. الحقل الهام في الهيكل هو locked وهو كلمة تكون صفراً عندما يكون القفل متاحاً وغير صفري عندما يكون محتجزاً. لو لم تكن هناك تزامنات لتمكنت xv6 من الاستحواذ على قفل بشفرة كـ:

```

void
acquire(struct spinlock *lk) // لا تعمل!
{
    for (;;) {
        if (lk->locked == 0) {
            lk->locked = 1;
            break;
        }
    }
}

```



```
{
{
```

لسوء الحظ هذا التنفيذ لا يضمن الاستبعاد المتبادل في مواجهة التزامن. معالجان يمكنهما في الوقت نفسه الوصول للسطر `if(lk->locked == 0)` ومشاهدة أن `lk->locked` صفر ثم يمسك كلاهما القفل بتنفيذ `lk->locked = 1`. في هذه النقطة محتجز معالجان مختلفان القفل، مما يخرق خاصية الاستبعاد المتبادل. ما نحتاجه هو طريقة لجعل السطرين ينفذان كخطوة ذرية (Atomic) (غير قابلة للتقسيم).

لأن الأقفال واسعة الاستخدام، فإن المعالجات متعددة النوى توفر عادة تعليمة يمكن استخدامها لجعل السطرين ذريين. بـ RISC-V هذه التعليمة هي `amoswap r, r, a`. تقرأ التعليمة `amoswap` القيمة عند العنوان `a` في الذاكرة، وتكتب محتويات المسجل `r` لذلك العنوان، وتضع القيمة التي قرأتها في `r`. أي أنها تبديل محتويات المسجل والموقع في الذاكرة. تنفذ هذا التسلسل ذرياً باستخدام عتاد خاص لمنع أي معالج آخر من استخدام عنوان الذاكرة بين القراءة والكتابة.

يوفر مترجم C عائلة من الدوال المدمجة «الذرية» التي تولد تعليمات ذرية. تستخدم `acquire()` بـ xv6 الدالة `__atomic_exchange_n(addr, value, ...)` والتي تتلخص في التعليمة `amoswap`؛ ترجع الدالة المحتويات القديمة (المبدلة) في `addr*`. فيما يلي طريقة جيدة لكتابة الحلقة التكرارية بـ `acquire`:

```
while (__atomic_exchange_n(&lk->locked, 1, __ATOMIC_ACQUIRE) != 0)
;
```

تستخدم `acquire` بـ xv6 الحلقة التكرارية أعلاه. كل تكرار يبذل واحداً في `lk->locked` ويفحص القيمة السابقة؛ إذا كانت القيمة السابقة صفراً فإننا نكون قد استحوذنا على القفل، والمبادلة ستكون قد ضبطت `lk->locked` لـ واحد. إذا كانت القيمة السابقة واحداً فإن معالجاً آخر يحتجز القفل، وحقيقة أننا بادلنا ذرياً واحداً في `lk->locked` لم تغير قيمته.

بمجرد الاستحواذ على القفل تسجل `acquire` المعالج الذي استحوذ على القفل لاستكشاف الأخطاء. الحقل `lk->cpu` محمي بالقفل ويجب ألا يتغير إلا أثناء احتجاز القفل.

الدالة `release` هي العكس لـ `acquire`: تصفر الحقل `lk->cpu` ثم تحرر القفل. منطقياً، التحرير يكفي بتعيين صفر لـ `lk->locked`. يتيح معيار C للمترجمات تنفيذ التعيين بتعليمات تخزين متعددة، لذا التعيين بـ C قد يكون غير ذري فيما يتعلق بالشفرة المتزامنة. بدلاً من ذلك تستخدم `release` الدالة الذرية لـ `__atomic_store_n(addr, 0, ...)` والتي تخزن الصفر للعنوان في عملية واحدة غير قابلة للتقسيم.

8.3 الكود البرمجي: استخدام الأقفال

تستخدم xv6 الأقفال في أماكن كثيرة لتجنب التنافسات. كما وُصف أعلاه تشكل `kalloc` و `kfree` مثلاً جيداً. جرب التمرينين 1 و 2 لمشاهدة ما يحدث لو أغفلت تلك الدوال الأقفال. ستجد يرجح أنه من الصعب إطلاق سلوك غير صحيح، مما يوجي بكونه من الصعب الاختبار الموثوق لكون الشفرة خالية من أخطاء الأقفال والتنافسات. قد تحوز xv6 تنافسات لم تُكتشف بعد.

جزء صلب بخصوص استخدام الأقفال هو القرار بمقدار الأقفال للبرمجة وأي البيانات والثوابت الملازمة ينبغي لكل قفل حمايتها. توجد بضع مبادئ أساسية. أولاً في أية شفرة قد تتسبب في كتابة موقع ذاكرة بواسطة معالج واحد في الوقت نفسه الذي يقرأه أو يكتبه فيه معالج آخر، يجب استخدام قفل لمنع التداخل بين العمليتين. ثانياً تذكر أن الأقفال تحمي الثوابت الملازمة: لو يتضمن ثابت ملازم مواقع ذاكرة متعددة فإن جميعها يرجح احتياجها للحماية بقفل واحد لضمان الحفاظ على الثابت الملازم.

القواعد أعلاه تخبر متى تكون الأقفال ضرورية لكنها لا تقول شيئاً عن متى تكون غير ضرورية، ومن الهام للكفاءة عدم القفل بكثرة، لأن الأقفال تقلل التوازي. لو لم يكن التوازي هاماً لتمكن المرء من الترتيب لحوز خيط واحد فقط وعدم القلق بخصوص الأقفال. نواة بسيطة يمكنها فعل هذا على متعدد معالجات عبر حوز قفل واحد؛ تستحوذ النواة على القفل في كل مرة تُدخل فيها النواة من مساحة المستخدم لاستدعاء نظام أو مقاطعة؛ وتحرر النواة القفل عندما ترجع لمساحة المستخدم. عديد نظم التشغيل أحادية المعالج تحولت للتنفيذ على متعددة المعالجات باستخدام هذا النهج، المسمى أحياناً بـ قفل النواة الضخم (Big kernel lock)، لكن النهج يصحى بالتوازي: معالج واحد فقط يمكنه التنفيذ في النواة في الوقت الواحد. لو كانت النواة تستهلك وقتاً كبيراً من المعالج، يمكن الحصول

على توازٍ أكثر عبر حماية كائنات أو وحدات مختلفة بأقفال مختلفة، لكي تتمكن معالجات مختلفة من التنفيذ في أجزاء مختلفة من النواة في الوقت نفسه.

كمثال على القفل غليظ الدرجة، يحوز مخصص `kalloc.c` بـ `xv6` قائمة حرة واحدة محمية بقفل واحد. لو حاولت عمليات متعددة على معالجات مختلفة تخصيص صفحات في الوقت نفسه، فإن كل منها سيتعين عليها الانتظار لدورها عبر الدوران في `acquire`. الدوران يضع وقت المعالج نظراً لأنه ليس عملاً نافعاً. لو أضع التنافس على القفل جزءاً كبيراً من وقت المعالج ربما أمكن تحسين الأداء عبر تغيير المخصص ليحوز قائمة حرة منفصلة لكل معالج، كل بقفلها الخاص، للسماح بتخصيص متواز حقاً.

كمثال على القفل دقيق الدرجة، تحوز `xv6` قفلاً منفصلاً لكل ملف، لكي تتمكن العمليات التي تتعامل مع ملفات مختلفة غالباً من المتابعة دون انتظار أقفال بعضها البعض. مخطط قفل الملفات يمكن جعله حتى أكثر دقة الدرجة لو أراد المرء السماح للعمليات بالكتابة المتزامنة لمناطق مختلفة من نفس الملف. في النهاية قرارات درجة الأقال يجب أن تقاد بمتطلبات الأداء بالإضافة لاعتبارات التعقيد.

كما تشرح الفصول التالية كل جزء من `xv6` فإنها ستذكر أمثلة لاستخدام `xv6` للأقال للتعامل مع التزامن. كمعينة، يعرض الجدول جدول 2 كافة الأقال بـ `xv6`.

الوصف	القفل
حماية تخصيص مدخلات المخزن المؤقت لكتل القرص.	<code>bcache.lock</code>
تسلسل معالجة قراءة مدخلات لوحة التحكم.	<code>cons.lock</code>
تسلسل الوصول لعناد مخرجات لوحة التحكم (<code>uart</code>).	<code>tx_lock</code>
تسلسل تخصيص هيكل <code>file</code> في جدول الملفات.	<code>ftable.lock</code>
حماية تخصيص مدخلات العقد الفهرسية في الذاكرة.	<code>itable.lock</code>
تسلسل الوصول لعناد القرص وطابور واصفات <code>DMA</code> .	<code>vdisk_lock</code>
تسلسل تخصيص الذاكرة.	<code>kmem.lock</code>
تسلسل العمليات على سجل المعاملات.	<code>log.lock</code>
تسلسل العمليات على كل أنبوب.	<code>pipe's pi->lock</code>
تسلسل زيادات <code>next_pid</code> .	<code>pid_lock</code>
تسلسل التغيرات لحالة كل عملية.	<code>proc's p->lock</code>
مساعدة <code>wait</code> لتجنب الاستيقاظ المفقود.	<code>wait_lock</code>
تسلسل العمليات على عداد الدقات <code>ticks</code> .	<code>tickslock</code>
تسلسل العمليات على كل عقدة فهرسية ومحتواها.	<code>inode's ip->lock</code>
تسلسل العمليات على كل مخزن مؤقت للكتل.	<code>buf's b->lock</code>

جدول 2: الأقال بـ `xv6`.

8.4 الانسداد المتبادل وترتيب الأقال

افترض أن الدوال المنفذة على المعالجين `C1` و `C2` كلاهما تحوز نقطة يحتاج عندها كل منهما احتجاز كل من القفل `A` والقفل `B`، واستحوذا عليهما في ترتيبين مختلفين:

```

المعالج C1:      المعالج C2:
acquire(B);      acquire(A);
acquire(A);      acquire(B);

```

مع القليل من سوء الحظ قد ينفذ `C1` و `C2` أول استدعاء `acquire` لهما في نفس اللحظة بالضبط؛ كلاهما يمكنه النجاح نظراً لأنهما يطلبان أقفالاً مختلفة. ولكن عندئذ سيتعين على كل من `C1` و `C2` الانتظار في استدعاءاتهما الثانية لـ `acquire` () نظراً لأن كلا القفلين محتجز بالفعل بواسطة المعالج الآخر. لأن كلا المعالجين ينتظران بعضهما البعض فلن يحرر أي منهما قفلاً أبداً وكلاهما سينتظر للأبد. هذه الحالة تُسمى الانسداد المتبادل (`Deadlock`).

المشكلة الرئيسية في مثال C1/C2 هي أن المعالجات استحوذوا على الأقفال في ترتيبين مختلفين. لو أنهما حاولا الاستحواذ على A أولاً لكان أحدهما قد استحوذ على A ثم B ثم حررها كلاهما، وكان المعالج الآخر قد تمكن عندئذ من المتابعة. على نحو أعم فإن شفرة الأقفال يجب أن تتبع هذه القاعدة لتجنب الانسداد المتبادل: كافة مسارات الشفرة التي تحتجز أقفالاً متعددة يجب أن تستحوذ على الأقفال في نفس الترتيب. الحاجة لترتيب الاستحواذ العام هذا تعني أن الأقفال هي بفعالية جزء من مواصفة كل دالة: المستدعون يجب أن يدعوا الدوال بطريقة تتسبب في جعل الأقفال تُستحوذ في الترتيب المتفق عليه.

تحوز xv6 سلاسل ترتيب أقفال كثيرة بطول اثنين تتضمن الأقفال الخاصة بكل عملية (القفل في كل struct proc) بفضل الطريقة التي تعمل بها آلية sleep/wakeup (انظر الفصل الفصل 10). على سبيل المثال consoleintr هي روتين المقاطعة الذي يتعامل مع الحروف المكتوبة. عندما يصل سطر جديد فإن أية عملية تنتظر مدخلات لوحة التحكم ينبغي إيقافها. لقيام بذلك تحتجز consoleintr القفل cons.lock أثناء دعوة wakeup والتي تستحوذ على قفل العملية المنتظرة من أجل إيقافها. ونتيجة لذلك فإن ترتيب الأقفال العام المتجنب للانسداد المتبادل يتضمن القاعدة يكون cons.lock يجب الاستحواذ عليه قبل أي قفل عملية. شفرة نظام الملفات تحوي أطول سلاسل أقفال بـ xv6. على سبيل المثال إنشاء ملف يتطلب في الوقت نفسه احتجاز قفل على المجلد، قفل على العقدة الفهرسية للملف الجديد، قفل على حاجر كتلة القرص، القفل vdisk_lock لبرنامج تشغيل القرص، والقفل p->lock للعملية المستدعية. لتجنب الانسداد المتبادل فإن شفرة نظام الملفات تستحوذ دائماً على الأقفال في الترتيب المذكور في الجملة السابقة.

احترام ترتيب عام متجنب للانسداد المتبادل قد يكون صلب بشكل مفاجئ. أحياناً يتضارب ترتيب الأقفال مع البنية المنطقية للبرنامج، مثلاً ربما وحدة الشفرة M1 تدعو الدالة M2 لكن ترتيب الأقفال يتطلب أن قفلاً في M2 يُستحوذ عليه قبل قفل في M1. أحياناً لا تكون هويات الأقفال معروفة مقدماً، ربما لأن قفلاً واحداً يجب احتجازه من أجل اكتشاف هوية القفل المراد الاستحواذ عليه تالياً. هذا النوع من الحالات ينشأ في نظام الملفات بينما يبحث عن المكونات المتتالية في اسم المسار، وفي الشفرة المخصصة لاستدعاء النظام wait و exit بينما تبحثان في جدول العمليات عن العمليات الابن. أخيراً فإن خطر الانسداد المتبادل يكون غالباً قيدياً على مدى دقة الدرجة التي يمكن بها جعل مخطط الأقفال، نظراً لأن الأقفال الأكثر تعني فرصاً أكثر للانسداد المتبادل. الحاجة لتجنب الانسداد المتبادل تكون غالباً عاملاً رئيسياً في تنفيذ النواة.

سؤال ينشأ أحياناً هو ماذا ينبغي أن يحدث لو حاول معالج الاستحواذ على قفل يحتجزه ذلك المعالج بالفعل. خط من التفكير يقول إن هذا ينبغي السماح به: لا يمكن لمعالج آخر احتجاز القفل، لذا لا حاجة للقلق بخصوص تعارض المعالجات مع استخدام بعضها البعض للبيانات المحمية. نظم الأقفال التي تتيح للمعالج إعادة الاستحواذ على قفل يحتجزه بالفعل تُسمى معادة الدخول (Re-entrant) أو تكرارية (Recursive). من ناحية أخرى لو كان القفل محتجزاً بالفعل، حتى على نفس المعالج، فهذا يعني أن عملية ما ربما خرقت مؤقتاً ولم تستعد بعد بعض الثوابت الملازمة؛ السماح لعملية جديدة بالبداية بينما الثوابت الملازمة غير قائمة يبدو ك دعوة للأخطاء البرمجية. تأخذ xv6 هذا الرأي الأخير، وتحظر على معالج يحتجز حالياً قفلاً إعادة الاستحواذ عليه. اكتشاف هذه الحالة هو العرض من الاستدعاء لـ holding في acquire.

8.5 الأقفال والمقاطع

بعض أقفال الدوران بـ xv6 تحمي بيانات تُستخدم بواسطة كل من الخيوط ومعالجات المقاطعة. على سبيل المثال معالج مقاطعة الميقاتي clockintr قد يزيد ticks في الوقت نفسه تقريباً الذي يقرأ فيه خيط نواة ticks في sys_pause. القفل tickslock يسلسل الوصولين.

تفاعل أقفال الدوران والمقاطع يطرح خطراً محتملاً. افترض أن sys_pause تحتجز tickslock و قُطع معالجها بمقاطعة ميقاتي. ستحاول clockintr الاستحواذ على tickslock وتكتشف كونه محتجزاً وتنتظر تحريره. في هذه الحالة لن يُحرر tickslock مطلقاً؛ فقط sys_pause يمكنها تحريره، لكن sys_pause لن تستمر في التشغيل حتى ترجع clockintr. لذا سيتعرض المعالج لانسداد متبادل، وأية شفرة تحتاج أيّاً من القفلين ستتجمد أيضاً.

لتجنب هذه الحالة، لو كان قفل دوران يُستخدم بواسطة معالج مقاطعة فلا يجوز لمعالج احتجاز ذلك القفل مطلقاً مع تفعيل المقاطعات. xv6 أكثر تحفظاً: عندما يستحوذ معالج على أي قفل فإن xv6 تعطل دائماً المقاطعات على

ذلك المعالج. المقاطعات قد تقع لا تزال على معالجات أخرى، لذا فإن acquire مقاطعة يمكنها الانتظار لخيطة ليحرر قفل دوران؛ مجرد ليس على نفس المعالج.

تعيد xv6 تفعيل المقاطعات عندما لا يحتجز معالج أية أقفال دوران؛ يجب عليها القيام بقليل من دفتر التتبع للتعامل مع المقاطع الحرجة المتداخلة. تدعو acquire الدالة push_off وتدعو release الدالة pop_off لتتبع مستوى التداخل للأقفال على المعالج الحالي. عندما يصل ذلك العداد للصفر تستعيد pop_off حالة تفعيل المقاطعات التي تواجده في بداية المقطع الحرج الخارجي. تخبر push_off و pop_off المعالج بتفعيل وتعطيل المقاطعات عبر تغيير البت SIE في المسجل sstatus باستخدام intr_on و rc_sstatus(SSTATUS_SIE)؛ والأخيرة تصفر البت SIE. من الهام أن تدعو acquire الدالة push_off صرامة قبل ضبط lk->locked. لو عكس الاثنان لكانت هناك نافذة صغيرة عندما يكون القفل محتجزاً مع تفعيل المقاطعات، ومقاطعة بتوقيت سيء كانت ستتسبب في انسداد متبادل للنظام. بالمثل من الهام أن تدعو release الدالة pop_off فقط بعد تحرير القفل.

8.6 ترتيب التعليمات والذاكرة

من الطبيعي التفكير في البرامج بكونها تنفذ في الترتيب الذي تظهر به عبارات الشفرة المصدرية. هذا نموذج عقلي معقول للشفرة أحادية الخيط، ولكنه ليس ما يحدث في الواقع.

أحد الأسباب هو أن المترجمات تصدر تعليمات تحميل وتخزين في ترتيبات مختلفة عن تلك الضمنية في الشفرة المصدرية، وقد تحذفها تماماً (على سبيل المثال عبر التخزين المؤقت للبيانات في السجلات). سبب آخر هو أن المعالج قد ينفذ التعليمات خارج الترتيب لزيادة الأداء. على سبيل المثال قد يلاحظ معالج أنه في متتالية تسلسلية من التعليمات فإن A و B ليستا معتمدتين على بعضهما البعض. قد يبدأ المعالج التعليم B أولاً، إما لأن مدخلاتها متجهزة قبل مدخلات A، أو من أجل تدخل تنفيذ A و B.

إعادة ترتيب الوصول للذاكرة بواسطة المعالج والمترجم قد تتسبب في مشاكل عندما تتفاعل خيوط متعددة عبر ذاكرة مشتركة. كمثال على ما قد يمر بأسلوب خاطئ، في هذه الشفرة لـ push سيكون كارثة لو نقل المترجم أو المعالج التخزين المناظر لـ list = next لـ نقطة بعد release:

```
l = malloc(sizeof *l);
l->data = data;
acquire(&listlock);
l->next = list;
list = l;
release(&listlock);
```

لو وقعت مثل هذه إعادة الترتيب لكانت هناك نافذة يمكن خلالها لمعالج آخر الاستحواذ على القفل وملاحظة list المحدثة، لكن مع مشاهدة list->next غير مهياً.

الخبر الجيد هو أن المترجمات والمعالجات توفر طرقاً للمبرمجين لمنع إعادة الترتيب. تظهر هذه بـ xv6 ك معاملات ATOMIC_ACQUIRE و ATOMIC_RELEASE للدوال الذرية لـ C المستخدمة بـ acquire و release. لكل منها أثران. أولاً يخبر كل منهما المترجم بعدم نقل مراجع الذاكرة بعد الاستدعاء الذري. ثانياً يخبر كل منهما المترجم بإنشاء تعليمات تخبر المعالج بعدم نقل مراجع ذاكرة وقت التنفيذ بعد التعليمات المولدة للاستدعاء الذري. مثل هذه النقطة «لا تنقل بعد» تُسمى حاجز ذاكرة (Memory barrier) أو سياجاً (Fence).

الحواجز في دوال أقفال الدوران بـ xv6 تحوز أثرين هامين على الشفرة التي تستخدم الأقفال. أولاً كافة مراجع الذاكرة (القراءات والكتابات) التي تقع في شفرة C قبل release () يُضمن لها أن تأخذ أثرها على نظام الذاكرة قبل أن يكتمل release (). يتضمن هذا كلاً من مراجع الذاكرة في المقطع الحرج ومراجع الذاكرة قبل المقطع الحرج. ثانياً كافة مراجع الذاكرة في شفرة C بعد acquire يُضمن لها عدم أخذ أثر قبل الإتمام الناجح لـ amoswap في acquire (). مجدداً يتضمن هذا كلاً من المراجع في المقطع الحرج والمراجع بعد release.

تنتج هذه الحواجز أثراً متعدياً هاماً. على سبيل المثال لو كتب معالج لكائن بيانات مخصص حديثاً (دون احتجاز أقفال) ثم، في مقطع حرج، ربط ذلك الكائن في قائمة، فإن أي معالج يستحوذ لاحقاً على قفل ويحصل على مؤشر للكائن من القائمة يُضمن له مشاهدة كتابات المعالج الأول.

الحواجز بـ acquire و release الخاصة بـ xv6 تجبر الترتيب في كافة الحالات تقريباً التي يهتم فيها ذلك، نظراً لأن xv6 تستخدم الأفقال حول الوصلات للبيانات المشتركة. يناقش الفصل الفصل 13 بضع استثناءات.

8.7 البرمجة بالعمليات الذرية

على الرغم من أن xv6 لا تفعل هذا، إلا أنه من الممكن أحياناً تحديث البيانات المشتركة مع تجنب التنافسات عبر الاستخدام المباشر للعمليات الذرية، دون استخدام الأفقال. معظم المعالجات توفر بمرونة عشرات التعليمات الذرية، عادة بالنمط الذي تقرأ فيه موقع ذاكرة، تعدل القيمة، وتكتب القيمة الجديدة لنفس موقع الذاكرة، ذرياً. التعليمة amoadد بـ RISC-V مثال على ذلك: تضيف ذرياً محتويات مسجل لموقع في الذاكرة. يوفر مترجم C أطرفة «ذرية» لهذه التعليمات. وهكذا يمكن للشفرة زيادة عداد مشترك دون أفقال باستخدام:

```
__atomic_fetch_add(&counter, 1, __ATOMIC_SEQ_CST);
```

قد يكون نافعاً تجنب الأفقال لأنها تستهلك وقتاً من المعالج، أو لأن سبباً هيكلياً مثل تجنب الانسداد المتبادل يجعل استخدام الأفقال غير ملائم.

8.8 أفقال النوم

أحياناً تحتاج xv6 لاحتجاز قفل لوقت طويل. على سبيل المثال يحتفظ نظام الملفات بملف مقفول بينما يقرأ ويكتب محتواه على القرص، وهذه العمليات القرصية قد تستغرق عشرات الملي ثانية. احتجاز قفل دوران لذلك الوقت الطويل سيقود لهدر لو أرادت عملية أخرى الاستحواذ عليه، نظراً لأن العملية المستحوذة ستضيع الوقت في الدوران لوقت طويل. عيب آخر لأفقال الدوران هو أن العملية لا يمكنها التنازل عن المعالج مع الاحتفاظ بقفل دوران؛ نود فعل هذا لكي تتمكن عمليات أخرى من استخدام المعالج بينما تنتظر العملية الحائزة للقفل القرص. التنازل أثناء احتجاز قفل دوران غير قانوني لأنه قد يقود لانسداد متبادل لو حاول خيط ثانٍ عندئذ الاستحواذ على قفل الدوران؛ نظراً لأن acquire لا تتنازل عن المعالج فإن دوران الخيط الثاني قد يمنع الخيط الأول من التشغيل وتحرير القفل. التنازل أثناء احتجاز قفل يخرق أيضاً المتطلب بكون المقاطعات يجب أن تكون معطلة أثناء احتجاز قفل دوران. وهكذا نود نوعاً من الأفقال يتنازل عن المعالج أثناء انتظار الاستحواذ، ويسمح بالتنازلات (والمقاطعات) أثناء احتجاز القفل.

توفر xv6 مثل هذه الأفقال في شكل أفقال النوم (Sleep-locks). تتنازل acquiresleep عن المعالج أثناء الانتظار، باستخدام تقنيات ستُشرح في الفصل الفصل 10. على مستوى عالٍ يحوز قفل النوم حقل locked محمي بقفل دوران، ودعوة sleep لـ acquiresleep تتنازل ذرياً عن المعالج وتحرر قفل الدوران. النتيجة هي أن خيوطاً أخرى يمكنها التنفيذ بينما تنتظر acquiresleep.

لأن أفقال النوم تترك المقاطعات مفعلة فلا يمكن استخدامها في معالجات المقاطعة. ولأن acquiresleep قد تتنازل عن المعالج فلا يمكن استخدام أفقال النوم في المقاطع الحرجة لأفقال الدوران (على الرغم من إمكانية استخدام أفقال الدوران في المقاطع الحرجة لأفقال النوم).

أفقال الدوران تكون أنسب للمقاطع الحرجة القصيرة، نظراً لأن انتظارها يضع وقت المعالج؛ وأفقال النوم تعمل جيداً للعمليات المطولة.

8.9 العالم الحقيقي

البرمجة بالأفقال تظل مليئة بالتحدي على الرغم من سنين البحث في مجردات التزامن والتوازي. من الأفضل غالباً إخفاء الأفقال في تركيبات أعلى مستوى كـ الطواوير المتزامنة، على الرغم من أن xv6 لا تفعل هذا. لو برمجت بأفقال فمن الحكمة استخدام أداة تحاول التعرف على التنافسات، لأنه من السهل إغفال ثابت ملازم يتطلب قفلاً. معظم نظم التشغيل تدعم خيوط POSIX (المسماة Pthreads)، والتي تتيح لعملية مستخدم حوز عدة خيوط تنفذ بالتوازي على معالجات مختلفة. تحوز Pthreads دعماً لأفقال مستوى المستخدم، الحواجز، إلخ. تتيح Pthreads أيضاً للمبرمج اختيارياً التحديد بكون القفل ينبغي أن يكون معاد الدخول.

دعم Pthreads في مستوى المستخدم يتطلب دعماً من نظام التشغيل. على سبيل المثال ينبغي أن يكون الحال أنه لو حطر p-thread عند استدعاء نظام فإن p-thread آخر لنفس العملية ينبغي أن يكون قادراً على التشغيل على ذلك المعالج. كمثال آخر لو غير p-thread مساحة عناوين عملياته (مثلاً، رسم أو ألغى رسم ذاكرة) فيجب على

النواة الترتيب بحيث المعالجات الأخرى التي تشغل خيوطاً لنفس العملية تحدّث جداول صفحاتها العتادية لتعكس التغير في مساحة العنوانين.

من الممكن تنفيذ الأقفال دون تعليمات ذرية، ولكنه مكلف، لذا معظم نظم التشغيل تستخدم تعليمات ذرية.

الأقفال قد تكون مكلفة لو حاولت معالجات كثيرة الاستحواذ على نفس القفل في الوقت نفسه. لو حاز معالج قفلاً مخزناً مؤقتاً في مخزنه المحلي وحاول معالج آخر الاستحواذ على القفل، فإن التعليمات الذرية لتحديث خط المخزن المؤقت الحائز للقفل يجب أن تنقل الخط من مخزن المعالج الأول لمخزن المعالج الآخر، وربما إلغاء صلاحية أية نسخ أخرى لخط المخزن المؤقت. جلب خط مخزن مؤقت من مخزن معالج آخر قد يكون أضعافاً أكثر كلفة من جلب خط من مخزن محلي.

لتجنب النفقات المرتبطة بالأقفال، تستخدم العديد من نظم التشغيل هياكل بيانات وخوارزميات خالية من الأقفال (Lock-free). على سبيل المثال من الممكن تنفيذ قائمة ترابطية ك تلك في بداية الفصل لا تتطلب أقفالاً أثناء أبحاث القائمة، وتعليمات ذرية واحدة لإدراج عنصر في قائمة. البرمجة الخالية من الأقفال، مع ذلك، أكثر تعقيداً من البرمجة بأقفال؛ على سبيل المثال يتعين على المرء القلق بخصوص إعادة ترتيب التعليمات والذاكرة. البرمجة بأقفال هي بالفعل صلبة، لذا تتجنب xv6 التعقيد الإضافي للبرمجة الخالية من الأقفال.

8.10 تمارين

1. علّق الاستدعاءات لـ acquire و release في kalloc. يبدو أن هذا يتسبب في مشاكل لشفرة النواة التي تدعو kalloc؛ ما الأعراض التي تتوقع مشاهدتها؟ عندما تشغل xv6 هل تشاهد هذه الأعراض؟ ماذا عن تشغيل user tests؟ لو لم تشاهد مشكلة فلماذا؟ انظر ما إذا كان بمقدورك إثارة مشكلة عبر إدراج حلقات تكرارية وهمية في المقطع الحرج لـ kalloc.
2. افترض أنك بدلاً من ذلك علقت القفل في kfree (بعد استعادة القفل في kalloc). ماذا قد يمر بأسلوب خاطئ الآن؟ هل غياب الأقفال في kfree أقل ضرراً منه في kalloc؟
3. لو دعت عمليتان kalloc في الوقت نفسه فستعين على إحدهما الانتظار للأخرى وهو ما يُعد سيئاً للأداء. عدل kalloc.c لتحوز توازياً أكثر لكي تتمكن الاستدعاءات المتزامنة لـ kalloc من معالجات مختلفة من المتابعة دون انتظار بعضها البعض.
4. اكتب برنامجاً متوازياً باستخدام خيوط POSIX، المدعومة على معظم نظم التشغيل. على سبيل المثال نفذ جدول تجزئة متوازياً وقس ما إذا كان عدد عمليات الوضع/الجلب يتوسع مع زيادة عدد المعالجات.
5. نفذ طقمًا فرعيًا من Pthreads بـ xv6. أي نفذ مكتبة خيوط بمستوى المستخدم لكي تتمكن عملية مستخدم من حوز أكثر من خيط واحد والترتيب بحيث يمكن لهذه الخيوط التنفيذ بالتوازي على معالجات مختلفة. اخرج بتصميم يتعامل بأسلوب صحيح مع خيط ينفذ استدعاء نظام حاجب ويغير مساحة عناوينه المشتركة.

9 الجدولة

أي نظام تشغيل يُرجح تنفيذه بوجود عمليات أكثر مما تحوزه الحاسوب من معالجات، لذا تُعد الحاجة ماسة لخطّة لتقاسم المعالجات بالمشاركات الزمنية بين العمليات. مثاليًا تكون المشاركة شفافة لعمليات المستخدم. نهج شائع هو توفير الوهم لكل عملية بأنها تحوز معالجها الافتراضي الخاص عبر مضاعفة التقاسم (Multiplexing) للعمليات على المعالجات العتادية. يشرح هذا الفصل كيف يحقق xv6 هذه المضاعفة.

قبل المتابعة في هذا الفصل، يُرجى قراءة kernel/proc.h و kernel/swtch.S والدوال yield() و sched() و scheduler() في kernel/proc.c.

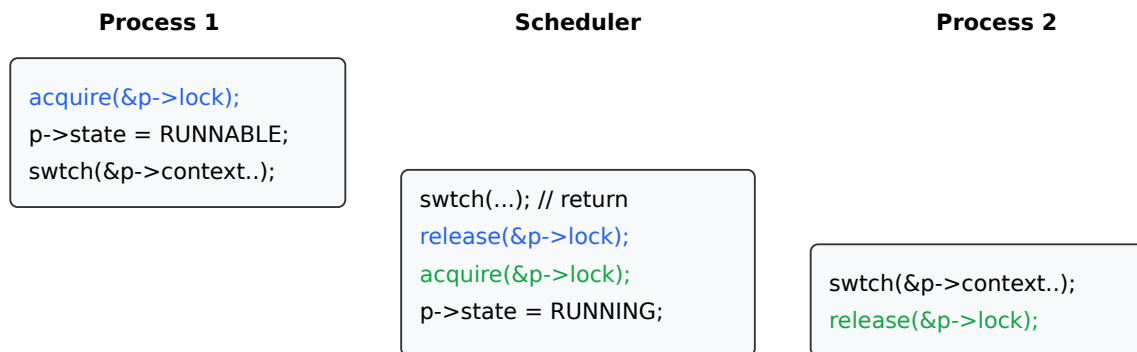
9.1 المضاعفة

تضاعف xv6 التقاسم عبر تبديل كل معالج من عملية لأخرى في حالتي. أولاً، تبديل xv6 عندما تنفذ عملية ما استدعاء نظام يحظر (يتعين عليه الانتظار)، على سبيل المثال read أو wait. ثانياً، تجبر xv6 تبديلاً دورياً للتعامل مع العمليات التي تحسب لفترات طويلة دون حظر. الأولى تُسمى تبديلات طوعية، والأخيرة غير طوعية.

تنفيذ المضاعفة يطرح بضعة تحديات. أولاً، كيف يتم التبديل من عملية لأخرى؟ الفكرة الأساسية هي حفظ واستعادة سجلات المعالج، على الرغم من أن حقيقة عدم إمكانية التعبير عن هذا بـ C تجعل الأمر دقيقاً. ثانياً، كيف تجبر التبديلات بأسلوب شفاف لعمليات المستخدم؟ تستخدم xv6 التقنية القياسية التي تقود فيها مقاطعات ميقاتي العتاد تبديلات السياق. ثالثاً، كافة المعالجات تبديل بين نفس الطقم من العمليات، لذا يلزم مخطط قفل لتجنب أخطاء مثل قرار معالجات تنفيذ نفس العملية في الوقت نفسه. رابعاً، ذاكرة العملية ومواردها الأخرى يجب تحريرها عندما تخرج العملية، لكنها لا يمكنها إتمام كل هذا بنفسها. خامساً، كل معالج بآلة متعددة النوى يجب أن يتذكر أية عملية ينفذ لكي تؤثر استدعاءات النظام على حالة النواة الصحيحة للعملية.

9.2 نظرة عامة على تبديل السياق

يشير المصطلح «تبديل السياق» للخطوات المتضمنة في ترك معالج ما لتنفيذ خيط نواة لعملية (عادة لاستئنافه لاحقاً)، واستئناف تنفيذ خيط نواة لعملية مختلفة؛ هذا التبديل هو قلب المضاعفة. لا تبديل xv6 السياق مباشرة من خيط نواة لعملية لخيط نواة لعملية أخرى؛ بدلاً من ذلك، يتخلى خيط النواة عن المعالج عبر تبديل السياق لـ «خيط المجدول» الخاص بذلك المعالج، ويختار خيط المجدول خيط نواة لعملية مختلفة لتنفيذ، ويبدل السياق لذلك الخيط.



شكل 11: التبديل من عملية مستخدم لأخرى.

على نطاق أوسع، الخطوات المتضمنة في التبديل من عملية مستخدم لأخرى موضحة في الشكل شكل 11: مصيدة (استدعاء نظام أو مقاطعة) من مساحة مستخدم العملية القديمة لخيط نواتها، تبديل سياق لخيط المجدول للمعالج الحالي، تبديل سياق لخيط نواة عملية جديدة، ورجوع مصيدة لعملية مستوى المستخدم.

9.3 الكود البرمجي: تبديل السياق

تحتوي الدالة `swtch()` في `kernel/swtch.S` قلب تبديل سياق الخيوط: تحفظ سجلات المعالج للخيط المتبديل منه، وتستعيد السجلات المحفوظة سابقاً للخيط المتبديل إليه. السبب الأساسي لكون هذا كافياً هو أن حالة الخيط تتكون من بيانات في الذاكرة (مثلاً مكدهه) بالإضافة لسجلات معالجه؛ ذاكرة الخيط لا تحتاج للحفظ والاستعادة لأن الخيوط المختلفة تحتفظ ببياناتها في مناطق مختلفة من RAM؛ لكن المعالج يحوز طقماً واحداً فقط من السجلات لذا يجب تبديلها (حفظها واستعادتها) بين الخيوط.

هيكل `struct proc` لكل خيط يتضمن `struct context` يحتفظ بسجلات الخيط المحفوظة عندما لا يكون قيد التشغيل. هيكل `struct context` لخيط مجدول المعالج يتواجد في `struct cpu` لذلك المعالج. عندما يرغب الخيط X في التبديل للخيط Y، يدعو الخيط X الاستدعاء `swtch(&X's context, &Y's context)`. تحفظ `swtch()` سجلات المعالج الحالية في سياق X، ثم تحمل محتوى سياق Y في سجلات المعالج، ثم ترجع.

فيما يلي نسخة مختصرة من `swtch`:

```

:swtch
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    ...
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ...
    ld s11, 104(a1)

    ret

```

يحوز a0 المعامل الأول للدالة، و a1 الثاني؛ في هذه الحالة، مؤشر struct context. يشير 16(a0) لإزاحة 16 بايت في الذاكرة المشار إليها بـ a0؛ بالرجوع لتعريف struct context في kernel/proc.h فإن هذا هو حقل الهيكل المسمى s0.

إلى أين ترجع تعليمة ret لـ swtch؟ ترجع للتعليمة التي يشير إليها المسجل ra. في المثال الذي يدعو فيه الخيط X الدالة swtch () للتبديل لـ Y، عندما تنفذ ret يكون ra قد حُمل تَوّاً من struct context لـ Y. و ra في struct context لـ Y حُفظ أصلاً بواسطة دعوة Y لـ swtch عندما تخلص Y عن المعالج في الماضي. لذا ترجع ret للتعليمة بعد النقطة التي دعا عندها Y الدالة swtch (); أي أن دعوة X لـ swtch () ترجع كما لو كانت راجعة من دعوة Y الأصلية لـ swtch (). وسيكون sp مؤشر مكّس Y، نظراً لأن swtch حملت sp من struct context لـ Y؛ وبذلك عند الرجوع سينفذ Y على مكّسه الخاص. لا تحتاج swtch () للحفظ المباشر أو الاستعادة لمؤشر البرنامج؛ يكفي حفظ واستعادة ra.

تحفظ swtch السجلات المحفوظة بواسطة المستدعي (s0..s11, sp, ra) وليس السجلات المحفوظة بواسطة المستدعي. تتطلب قواعد استدعاء RISC-V أنه إذا احتاجت الشفرة الحفاظ على القيمة في سجل محفوظ بواسطة المستدعي عبر استدعاء دالة، يجب على المترجم إنشاء تعليمات تحفظ المسجل للمكّس قبل استدعاء الدالة، وتستعيد من المكّس عندما ترجع الدالة. لذا يمكن لـ swtch الاعتماد على أن الدالة التي دعتها تكون قد حفظت بالفعل السجلات المحفوظة بواسطة المستدعي (إما ذلك، أو أن الدالة المستدعية لم تكن بحاجة للقيمة في السجلات).

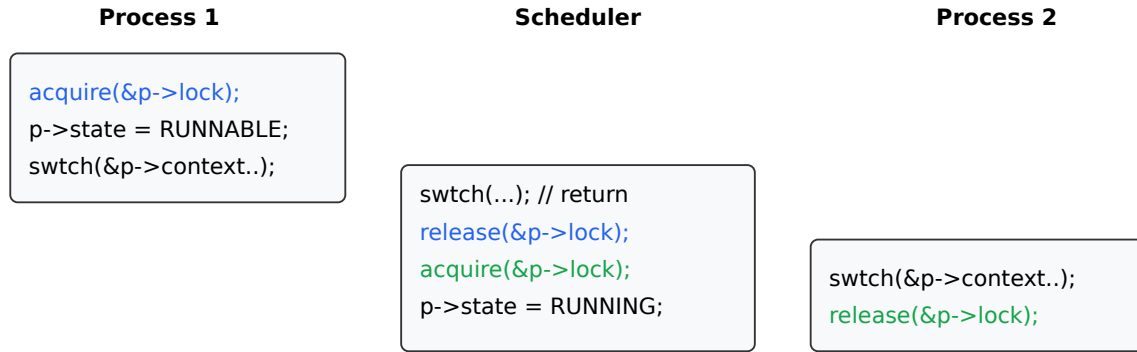
9.4 الكود البرمجي: الجدولة

فحص القيد السابق التفاصيل الداخلية لـ swtch؛ الآن دعنا نأخذ swtch كمعطى ونفحص التبديل من خيط نواة لعملية عبر المجدول لعملية أخرى. يتواجد المجدول في شكل خيط خاص لكل معالج، كل منها ينفذ الدالة scheduler. هذه الدالة مسؤولة عن اختيار أية عملية ستنفذ تالياً. لكل معالج خيط مجدوله الخاص لأن أكثر من معالج قد يكون يبحث عن شيء ليتنفذ في أي وقت محدد. تبديل العمليات يمر دائماً عبر خيط المجدول، بدلاً من المباشر من عملية لأخرى، لتجنب بعض الحالات التي لن يتوفر فيها مكّس لتنفيذ المجدول (مثلاً إذا كانت العملية القديمة قد خرجت، أو لا توجد عملية أخرى ترغب حالياً في التشغيل).

العملية التي ترغب في التخلي عن المعالج يجب أن تستحوذ على قفل عملياتها الخاص p->lock، وتحرر أية أقفال أخرى تحتجزها، وتحديث حالتها الخاصة (p->state)، ثم تدعو sched. يمكنك مشاهدة هذا التسلسل في yield و sleep و kexit. تدعو الدالة swtch لحفظ السياق الحالي في p->context والتبديل لسياق المجدول في cpu->context. ترجع swtch على مكّس المجدول كما لو كانت swtch الخاصة بـ scheduler قد رجعت.

تنفذ scheduler حلقة تكرارية: تجد عملية للتنفيذ، تنفذ swtch () إليها، وفي النهاية ستنفذ هي swtch () راجعة للمجدول، والذي يستمر في حلقة التكرارية. يدور المجدول في حلقة تكرارية على جدول العمليات بحثاً عن عملية قابلة للتشغيل، واحدة حائزة RUNNABLE == p->state. بمجرد عثوره على عملية، يضبط متغير العملية الحالية لكل معالج c->proc، ويعلم العملية كـ RUNNING، ثم يدعو swtch لبدء تشغيلها. عند نقطة ما في الماضي، تكون

العملية الهدف قد دعت swtch (); ودعوة المجدول لـ swtch () ترجع بفعالية من ذلك الاستدعاء الأبعد. يوضح الشكل شكل 12 هذا النمط.



شكل 12: حيازة p->lock عبر الاستدعاءات لـ swtch.

تحتفظ xv6 بـ p->lock عبر الاستدعاءات لـ swtch: يستحوذ المستدعي لـ swtch على القفل، ولكنه يُحرر في الهدف بعد رجوع swtch. هذا الترتيب غير معهود: المعهود أكثر أن الخيط الذي يستحوذ على قفل يقوم هو أيضاً بتحريره. تبديل السياق بـ xv6 يكسر هذا العرف لأن p->state و p->context يجب تحديثهما معاً ذرياً. على سبيل المثال لو حُرر p->lock قبل استدعاء swtch لكان معالج مختلف c قد قرر تنفيذ العملية لأن حالتها RUNNABLE. سيدعو المعالج c الدالة swtch والتي تستعيد من p->context بينما المعالج الأصلي لا يزال يحفظ في p->context. ستكون النتيجة استعادة العملية بسجلات محفوظة جزئياً على المعالج c وكلا المعالجين سيستخدمان نفس المكس، مما يتسبب في فوضى. بمجرد أن تبدأ yield تعديل حالة عملية منفذة لجعلها RUNNABLE يجب أن يظل p->lock محتجزاً حتى تحفظ العملية كافة سجلاتها ويكون المجدول ينفذ على مكسده. أبكر نقطة تحرير صحيحة تكون بعد أن يصقّر scheduler (المنفذ على مكسده الخاص) المتغير c->proc. بالمثل، بمجرد أن تبدأ scheduler تحويل عملية RUNNABLE إلى RUNNING فلا يمكن تحرير القفل حتى يكون خيط نواة العملية ينفذ بالكامل (بعد swtch على سبيل المثال في yield).

توجد حالة واحدة لا تنتهي فيها دعوة المجدول لـ swtch في sched. تضبط allocproc مسجل ra في السياق لعملية جديدة إلى forkret لكي ترجع أول swtch لها لبداية تلك الدالة. تتواجد forkret لتحرير p->lock وإعداد بعض سجلات التحكم وحقول trapframe المطلوبة للعودة لمساحة المستخدم. في النهاية تحاكي forkret مسار العودة العادي من استدعاء نظام راجعاً لمساحة المستخدم.

9.5 الكود البرمجي: mycpu و myproc

تحتاج xv6 غالباً مؤشراً لهيكل proc للعملية الحالية. على نظام أحادي المعالج يمكن حوز متغير عام يشير لـ proc الحالية. هذا لا يعمل على آلة متعددة النوى، نظراً لأن كل معالج ينفذ عملية مختلفة. طريقة حل هذه المشكلة هي استغلال حقيقة أن كل معالج يحوز طقمه الخاص من السجلات.

بينما ينفذ معالج محدد في النواة، تضمن xv6 أن المسجل tp لذلك المعالج يحتفظ دائماً بـ hartid المعالج. يرقم RISC-V معالجه معطياً كل منها hartid فريداً. تستخدم mycpu المسجل tp لفهرسة مصفوفة من هياكل cpu وإرجاع الهيكل الخاص بالمعالج الحالي. هيكل cpu struct يحتفظ بمؤشر لهيكل proc للعملية المنفذة حالياً على ذلك المعالج (إن وجدت)، وسجلات محفوظة لخيط مجدول المعالج، وعداد أقفال دوران متداخلة مطلوب لإدارة تعطيل المقاطعات.

ضمان احتفاظ tp في المعالج بـ hartid المعالج دقيق قليلاً، نظراً لأن شفرة المستخدم حرة في تعديل tp. تضبط start المسجل tp مبكراً في تسلسل إقلاع المعالج، بينما لا تزال في نمط الآلة. أثناء التجهز للعودة لمساحة المستخدم تحفظ prepare_return المسجل tp في صفحة المنطاد، في حالة تعديل شفرة المستخدم له. أخيراً تستعيد uservec ذلك tp المحفوظ عند دخول النواة من مساحة المستخدم. يضمن المترجم عدم تعديل tp مطلقاً في شفرة النواة. سيكون من الأسهل لو استطاعت xv6 سؤال عتاد RISC-V عن hartid الحالي كلما دعت الحاجة، لكن RISC-V تتيح ذلك فقط في نمط الآلة، وليس في نمط المشرف.

القيم المرجوعة لـ `cpuid` و `mycpu` هشة: لو قطعت الميقاتي وتسببت في جعل الخيط يتنازل ويستأنف التنفيذ لاحقاً على معالج مختلف، فإن القيمة المرجوعة سابقاً لن تكون صحيحة بعد ذلك. لتجنب هذه المشكلة، تتطلب `xv6` من الشفرة تعطيل المقاطعات قبل دعوة `cpuid()` أو `mycpu()`، وتفعيل المقاطعات فقط عند الانتهاء من استخدام القيمة المرجوعة.

ترجع الدالة `myproc` مؤشر `struct proc` للعملية التي تنفذ على المعالج الحالي. تعطل `myproc` المقاطعات، وتدعو `mycpu` وتجلب مؤشر العملية الحالية (`c->proc`) خارج `struct cpu` ثم تفعل المقاطعات. القيمة المرجوعة لـ `myproc` آمنة للاستخدام حتى وإن كانت المقاطعات مفعلة: لو نقلت مقاطعة ميقاتي العملية المستدعية لمعالج مختلف، فإن مؤشر `struct proc` الخاص بها سيظل كما هو.

9.6 العالم الحقيقي

ينفذ جدول `xv6` سياسة جدولة بسيطة تنفذ كل عملية بالدور. تُسمى هذه السياسة الجدولة الدورية (`Round robin`). تنفذ نظم التشغيل الحقيقية سياسات أكثر تعقيداً تتيح، على سبيل المثال، للعمليات حوز أولويات. الفكرة هي أن عملية حائزة أولوية عالية وقابلة للتشغيل سيفضلها الجدول على عملية حائزة أولوية منخفضة وقابلة للتشغيل. يمكن أن تصبح هذه السياسات معقدة لأن توجد غالباً أهداف متنافسة: على سبيل المثال، قد ترغب نظام التشغيل أيضاً بضمان العدالة والإنتاجية العالية.

9.7 تمارين

1. عدل `xv6` لاستخدام تبديل سياق واحد فقط عند التبديل من خيط نواة لعملية لآخر، بدلاً من التبديل عبر خيط الجدول. الخيط المتنازل سيتعين عليه اختيار الخيط التالي بنفسه ودعوة `swtch`. التحديات ستكون منع معالجات متعددة من تنفيذ نفس الخيط بالصدفة؛ ضبط القفل بصورة صحيحة؛ وتجنب الانسدادات المتبادلة.

10 النوم والاستيقاظ

تساعد الجدولة والأفقال في النواة على إخفاء أفعال خيط تنفيذ عن خيط آخر، لكننا نحتاج أيضاً إلى مجردات تساعد الخيوط على التفاعل والتنسيق القاصد فيما بينها. على سبيل المثال، قد يحتاج القارئ لأنبوب في `xv6` إلى الانتظار حتى تنتج عملية كاتبة بيانات؛ وقد يحتاج استدعاء الأب لـ `wait` إلى الانتظار حتى تخرج عملية ابن؛ والعملية التي تقرأ من القرص تحتاج للانتظار حتى ينهي عتاد القرص الصلب عملية القراءة. تستخدم نواة `xv6` آلية تُسمى النوم والاستيقاظ (`Sleep and Wakeup`) في هذه الحالات (وفيد العديد من الحالات الأخرى). يتيح النوم لخيط النواة الانتظار حتى يتحقق شرط (`Condition`) معين؛ ويمكن لخيط آخر أو لمعالج المقاطعة التسبب في جعل الشرط صحيحاً (عادةً عبر تعديل متغير أو متغيرة) ثم دعوة الاستيقاظ `wakeup` للإشارة إلى أن الخيوط المنتظرة للشرط يجب أن تستأنف عملها. تُسمى آليتنا النوم والاستيقاظ غالباً بـ تنسيق المتتاليات (`Sequence coordination`) أو المزامنة الشرطية (`Conditional synchronization`) المعتمدة في مراقبات المزامنة (`Monitors`).

قبل المتابعة، يُرجى قراءة الدوال البرمجية: `wakeup`، `sleep()`، `sleep_prepare()` في `kernel/proc.c` وكافة محتويات الملف `kernel/pipe.c`.

10.1 نظرة عامة

تبدو الواجهة البرمجية لآليتي النوم والاستيقاظ كالتالي:

```
void sleep_prepare(void *chan)
void sleep()
void wakeup(void *chan)
```

تنام العملية من أجل الانتظار حتى يصبح شرط ما صحيحاً، ويسمى المعامل `chan` (المسمى قناة الانتظار `Wait channel`) الشرط المرغوب. تعلن `sleep_prepare(chan)` عن أن العملية قد تنام قريباً للانتظار في قناة الانتظار المحددة؛ وترجع فوراً. تُدعى `wakeup(chan)` عندما يكون الشرط المسمى بـ `chan` قد أصبح (أو قد يكون أصبح) صحيحاً؛ وتقوم بإيقاظ كافة العمليات التي دعت `sleep_prepare(chan)` ثم `sleep()` بنفس قناة الانتظار `chan`. ترجع `sleep()` فوراً إذا كانت `wakeup(chan)` قد استدعيت بالفعل بعد الاستدعاء السابق (المطلوب) لـ `sleep_prepare(chan)`؛ وإذا لم تكن كذلك، تميز `sleep()` العملية المستدعية ك محظورة ونائمة (`SLEEPING`) وليس (`RUNNABLE`) وتتخلّى عن المعالج عبر تبديل السياق إلى الجدول، لكي تتمكن العمليات الأخرى من التنفيذ. تُدعى

`sleep_prepare()` و `sleep()` بواسطة نفس العملية (عادةً في أسطر قليلة تفصل بينهما)؛ بينما تُدعى `wakeup()` بواسطة عملية مختلفة أو بواسطة معالج المقاطعة.

تعامل هذه الدوال المعامل `chan` كقيمة 64-بت غامضة؛ والشيء الوحيد الذي تفعله معها هو المقارنة من أجل المساواة. النمط المعهود للمستدعين هو تمرير عنوان كائن مناسب كمعامل لـ `chan`. يكون مبرمج النواة مسؤولاً عن اختيار اعراف قنوات الانتظار، وعن تمرير نفس `chan` إلى استدعاءات `sleep_prepare()` و `wakeup()` التي ينبغي أن تتفاعل معاً.

تنام شفرة النواة من أجل الانتظار حتى يصبح شرط ما صحيحاً. على سبيل المثال، تنام شفرة النواة التي تقرأ من أنبوب إذا كان حاوي الأنبوب فارغاً حالياً؛ والشرط في هذه الحالة هو أن يصبح حاوي الأنبوب غير فارغ (بسبب كتابة عملية أخرى في الأنبوب). لا تعرف `sleep_prepare()` و `sleep()` كيفية فحص الشرط؛ فالشفرة المستدعية هي الوحيدة التي تعرف ذلك. النمط المعهود للعملية المحتمل نومها هو الاستحواذ أولاً على قفل يحمي الشرط؛ ثم فحص الشرط؛ وإذا لم يكن صحيحاً، تدعو `sleep_prepare()`، وتحرر القفل، وتدعو `sleep()`. وتدعو الشفرة التي تتسبب لاحقاً في جعل الشرط صحيحاً الدالة `wakeup`.

فيما يلي مخطط لكيفية نوم شفرة الأنبوب في نواة xv6:

```

}piperead(pipe)
;acquire(&pipe->lock)
}while(there's no data in pipe->buffer)
;sleep_prepare(&pipe)
;release(&pipe->lock)
;()sleep
;acquire(&pipe->lock)
{
;remove the data from the pipe
;release(&pipe->lock)
}
;pipewrite(pipe)
;acquire(&pipe->lock)
;append data to pipe->buffer
;wakeup(&pipe)
;release(&pipe->lock)
}

```

تستخدم هذه الشفرة عنوان هيكل بيانات الأنبوب كقناة انتظار.

يجب دعوة `sleep_prepare(chan)` إما قبل فحص الشرط، أو قبل تحرير القفل الذي يحمي الشرط. يسجل الاستدعاء حقيقة أن العملية قد تحتاج للنوم في `chan` المحدد. إذا دعت عملية أخرى (أو مقاطعة) `wakeup(chan)` بين الاستدعاءات لـ `sleep_prepare(chan)` و `sleep()`، تضبط `wakeup()` علامة في العملية المشاركة على النوم تشير لكونها لا ينبغي أن تنام في الواقع. إذا شاهدت `sleep()` هذه العلامة، فإنها ترجع بدلاً من النوم.

ما هو السبب وراء `sleep_prepare(chan)` هناك احتمالية لأن يصبح الشرط صحيحاً، وتُدعى `wakeup()`، بين الوقت الذي يشاهد فيه قارئ الأنبوب أن الحاوي فارغ، وبين الوقت الذي يدعو فيه `sleep`. بدون بعض الذكاء، لن تجد `wakeup()` أية عمليات نائمة، ولن يكون لها أي أثر. عندها ستدعو العملية `sleep()` على الرغم من أن حاوي الأنبوب يحتوي بيانات للقراءة؛ وإذا كان (كما قد تكون الحالة) الكاتب للأنبوب لن يكتب أي شيء آخر مطلقاً، فقد تنام القارئ خاطئاً إلى الأبد. تُسمى هذه الحالة غير المرغوبة بـ الاستيقاظ المفقود (Lost wake-up). يتجنب السلوك الموصوف في الفقرة السابقة هذه المشكلة.

في مثال الأنبوب، كما هو الحال في معظم الشفرات النائمة، يوجد قفل يحمي الشرط ويمنع دعوة `wakeup()` بين الوقت الذي تفحص فيه العملية المشاركة على النوم الشرط وبين الوقت الذي تدعو فيه `sleep_prepare()`. يُسمى هذا القفل غالباً بـ قفل الشرط (Condition lock) أو قفل المراقب. يجب على العملية تحرير القفل قبل دعوة

sleep() (حتى تتمكن عملية أخرى أو معالج مقاطعة من تعديل الشرط ودعوة wakeup())، وإعادة الاستحواذ على القفل بعد رجوع sleep().

10.2 الكود البرمجي: النوم والاستيقاظ

تُنفذ sleep_prepare و sleep و wakeup بـ xv6 الواجهة المستخدمة في المثال أعلاه. الفكرة الأساسية هي أن تجعل sleep العملية الحالية محظورة ونائمة كـ SLEEPING ثم تدعو sched لتخيل المعالج؛ وتفحص wakeup العمليات النائمة في قناة الانتظار المحددة وتميزها كـ RUNNABLE.

تسجل sleep_prepare(chan) قناة الانتظار في p->chan وترجع. إذا كانت p->chan لا تزال غير صفيرية عندما تُدعى sleep()، تغير sleep() العملية المستدعية إلى الحالة المحظورة SLEEPING وتدعو sched() لتترك العمليات الأخرى تنفذ.

تعالج wakeup(chan) حالتيين فيما يتعلق بعملية أخرى قد تكون بالتوازي في مرحلة النوم في chan. إذا كانت تلك العملية الأخرى قد وصلت لمرحلة تميز نفسها كـ SLEEPING، فستشاهدها wakeup في تلك الحالة وتغيرها إلى RUNNABLE؛ وسيُرجع استدعاؤها لـ sleep(). وإذا كانت العملية الأخرى قد وصلت لمرحلة استدعاء sleep_prepare(chan) (ولكن sleep()) الخاصة بها لم تصل لمرحلة الاستحواذ على lock (p->lock)، ستشاهد wakeup(chan) أن p->chan مساوٍ لـ chan، وستضبط wakeup قيمة p->chan إلى الصفر. في تلك الحالة، عندما تدعو العملية الأخرى sleep بالفعل، ستشاهد الصفر وترجع فوراً. الغاية هي تجنب الاستيقاظ المفقود إذا استدعيت wakeup() في نفس اللحظة التي توشك فيها عملية أخرى على دعوة sleep().

نمنعنا قواعد استخدام sleep_prepare() من القلق بشأن الحالة التي تكون فيها العملية المشاركة على النوم لم تدع sleep_prepare(). بعد. فإذا أن تحيط تلك العملية فحص شرطها و sleep_prepare() بقفل يمنع أي استدعاء لـ wakeup() للقناة المعنية، أو يجب على تلك العملية دعوة sleep_prepare() قبل فحص الشرط.

في بعض الأحيان تكون عمليات متعددة نائمة في نفس القناة؛ على سبيل المثال، أكثر من عملية تقرأ من أنبوب. استدعاء واحد لـ wakeup سيوقفها جميعاً. إحداها ستنفذ أولاً (وفي حالة الأنابيب) تستحوذ على قفل الأنبوب وتقرأ أية بيانات منتظرة. العمليات الأخرى ستجد أنه على الرغم من إيقافها، لا توجد بيانات لتقرأ. من وجهة نظرها كان الاستيقاظ زائفاً («spurious»)، ويجب عليها النوم مجدداً. لهذا السبب يحدث النوم دائماً في حلقة تكرارية تعيد فحص الشرط، كما في piperead أعلاه.

لا يقع أي ضرر إذا اختار استخدامان لـ sleep/wakeup نفس القناة بالصدفة: سيُشاهدان استيقاظاً زائفاً، لكن الحلقة التكرارية كما هو موصوف أعلاه ستتحمل هذه المشكلة. كثير من جاذبية sleep/wakeup تكمن في كونها خفيفة الوزن (لا حاجة لإنشاء هياكل بيانات خاصة لتعمل كقنوات انتظار) وتوفر طبقة من عدم المباشرة (لا يحتاج المستدعون لمعرفة أية عملية المحددة يتفاعلون معها).

10.3 الكود البرمجي: الأنابيب

تُعد الأنابيب بـ xv6 مثالاً على الشفرة التي تنام لمزامنة المنتجين والمستهلكين. شاهدنا واجهة الأنابيب في الفصل 2: البايتات المكتوبة في أحد طرفي الأنبوب تُنسخ لحاوي في النواة ويمكن قراءتها من الطرف الآخر للأنبوب. ستفحص الفصول المستقبلية دعم واصفات الملفات المحيط بالأنابيب، ولكن دعنا ننظر الآن في تنفيذي piperead و pipewrite.

يُمثل كل أنبوب بـ struct pipe التي تحتوي على lock وحاوي بيانات data. تحسب الحقول nread و nwrite العدد الإجمالي للبايتات المقروءة والمكتوبة في الحاوي. يدور الحاوي دافعاً: البايت التالي المكتوب بعد buf[PIPE_SIZE-1] يكون buf[0]. العدادات لا تدور. تيح هذه القاعدة للتنفيذ التمييز بين حاوي ممتلئ (nwrite == nread + PIPE_SIZE) وبين حاوي فارغ (nwrite == nread)، ولكنها تعني أن الفهرسة في الحاوي يجب أن تستخدم buf[nread % PIPE_SIZE] بدلاً من مجرد buf[nread].

دعنا نفترض أن الاستدعاءات لـ piperead و pipewrite تحدث بالتزامن على معاليتين مختلفتين. تبدأ pipewrite بالاستحواذ على قفل الأنبوب الذي يحمي العدادات، والبيانات، وثوابتها المرتبطة. تحاول piperead عندئذ الاستحواذ على القفل أيضاً، لكنها لا تستطيع. تدور في acquire منتظرة القفل. بينما تنتظر piperead، تدور pipewrite في حلقة تكرارية على البايتات المكتوبة، مضافة كل منها في الأنبوب بالدور. خلال هذه الحلقة، قد يحدث أن يمتلئ

الحاوي. في هذه الحالة، تدعو `pipewrite` الدالة `wakeup` لتنبيه أي قراء نائمين لحقيقة وجود بيانات منتظرة في الحاوي، وتحرر القفل، وتنام في قناة الانتظار `pi->nwrite` ليأخذ بعض البايتات خارج الحاوي.

تستحوذ `piperead` الآن على قفل الأنبوب وتدخل مقطعها الحرج: تجد أن `pi->nread != pi->nwrite` (تستحوذ `pipewrite`) ذهبت للنوم لأن `PIPE_SIZE` `pi->nread == pi->nwrite`، لذا تسقط في حلقة `for` التكرارية، وتنسخ البيانات خارج الأنبوب، وتزيد `nread` بعدد البايتات المنسوخة. تلك المساحة في الحاوي أصبحت الآن متاحة للكتابة، لذا تدعو `piperead` الدالة `wakeup` لإيقاظ أي كتاب نائمين قبل أن ترجع. تجد `wakeup` عملية نائمة في `pi->nwrite` (العملية التي كانت تنفذ `pipewrite` وتوقفت عندما امتلأ الحاوي). تميز `wakeup` تلك العملية كـ `RUNNABLE`.

تستخدم شفرة الأنبوب قنوات انتظار مستقلة للقارئ وال كاتب (`pi->nread` و `pi->nwrite`)؛ قد يجعل هذا النظام أكثر كفاءة في حالة وجود العديد من القراء والكتاب المنتظرين لنفس الأنبوب. تنام شفرة الأنبوب في حلقة تفحص شرط النوم؛ إذا كان هناك قراء أو كتاب متعددون، فإن كافة العمليات ما عدا العملية الأولى التي تستيقظ ستشاهد أن الشرط خاطئ وتنام مجدداً.

10.4 الكود البرمجي: الانتظار، الخروج، والقتل

يُرجى قراءة الشفرة البرمجية للدوال `kwait()`، `kexit()`، `kill` في `kernel/proc.c`؛ هذه هي التنفيذات الداخلية لاستدعاءات النظام المناظرة.

من الأمثلة الممتعة على النوم، والتي قُدمت في الفصل 2، التفاعل بين خروج الابن `exit` وانتظار الأب `wait`. في وقت وفاة الابن، قد يكون الأب نائماً بالفعل في `wait`، أو قد يفعل شيئاً آخر؛ في الحالة الأخيرة، يجب أن يشاهد استدعاء لاحق لـ `wait` وفاة الابن، ربما بعد وقت طويل من استدعائه لـ `exit`. الطريقة التي يسجل بها `xv6` وفاة الابن حتى يشاهدها `wait` هي أن تضع `exit` المستدعي في حالة الميت `ZOMBIE`، حيث يظل حتى يلاحظه `wait` الأب، ويغير حالة الابن إلى `UNUSED`، وينسخ حالة خروج الابن، ويرجع رقم عملية الابن للأب. إذا خرج الأب قبل الابن، يعطي الأب الابن لعملية `init` التي تدعو `wait` باستمرار؛ وبذلك يكون لكل ابن أب لينظف بعده. التحدي هو تجنب التنافس والانسداد المتبادل بين `wait` الأب المتزامن و `exit` الابن، وكذلك بين `exit` و `exit` المتزامنين.

تبدأ `kwait` التنفيذ الداخلي لـ `wait` بالاستحواذ على `wait_lock` التي تعمل كقفل الشرط الذي يساعد في ضمان ألا تفوت `kwait` استدعاء `wakeup` قادم من ابن يخرج. ثم تتصفح جدول العمليات. إذا وجدت ابناً في حالة `ZOMBIE`، تحرر موارد ذلك الابن وهيكل `proc` الخاص به، وتنسخ حالة خروج الابن للعنوان المزود لـ `wait` (إذا لم يكن 0)، وترجع رقم عملية الابن. إذا وجدت `kwait` أبناء ولكن لم يخرج أي منهم، تنام بانتظار خروج أي منهم، ثم تتصفح مجدداً. تستحوذ `kwait` غالباً على قفلين: `wait_lock` وقفل عملية ما `pp->lock`؛ والترتيب المتجنب للانسداد المتبادل هو استحواذ `wait_lock` أولاً ثم `pp->lock`.

تسجل `kexit` حالة الخروج، وتحرر بعض الموارد، وتدعو `reparent` لإعطاء أي أبناء لعملية `init`، وتوقظ الأب في حالة وجوده في `wait`، وتميز المستدعي كـ `zombie`، وتتخلى نهائياً عن المعالج. تستحوذ `kexit` على كلاً من `wait_lock` و `pp->lock` خلال هذا التسلسل. تستحوذ على `wait_lock` لأنه قفل الشرط لـ `wakeup(pp->parent)`، مانعاً الأب في `wait` من فقدان الاستيقاظ. ويجب على `kexit` الاستحواذ على `pp->lock` لهذا التسلسل أيضاً، لمنع الأب في `wait` من مشاهدة أن الابن في حالة `ZOMBIE` قبل أن يدعو الابن `swtch` نهائياً. تستحوذ `kexit` على هذه الأقفال بنفس الترتيب كـ `kwait` لتجنب الانسداد المتبادل.

قد يبدو من غير الصحيح لـ `kexit` إيقاظ الأب قبل ضبط حالته لـ `ZOMBIE`، لكن ذلك آمن: على الرغم من أن `wakeup` قد تتسبب في تنفيذ الأب، إلا أن الحلقة التكرارية في `kwait` الأب لا يمكنها فحص الابن حتى يُحرر قفل الابن `pp->lock` بواسطة `scheduler`، لذا لا يمكن لـ `kwait` النظر في العملية الخارجة إلا بعد أن تضبط `kexit` حالتها لـ `ZOMBIE`.

بينما يتيح `exit` للعملية إنهاء نفسها، يتيح استدعاء النظام `kill` لعملية ما طلب إنهاء عملية أخرى. سيكون من المعقد جداً لـ `kill` تدمير العملية الضحية مباشرة، لأن الضحية قد تكون تنفذ على معالج آخر، ربما في منتصف تسلسل حساس لتحديثات هياكل بيانات النواة. لذا تفعل `kill` القليل جداً: تكتفي بضبط `pp->killed` الخاص بالضحية وإذا كانت نائمة تفحص إيقاظها. في النهاية ستدخل الضحية أو تخرج من النواة، وفي تلك النقطة ستدعو الشفرة في `usertrap` الدالة `kexit` إذا كانت `pp->killed` مضبوطة (تفحص عن طريق دعوة `killed`). إذا كانت الضحية تنفذ في مساحة المستخدم، ستشاهد أنها قُتلت في المرة التالية التي تدخل فيها النواة عبر تنفيذ استدعاء نظام أو لأن الميقاتي (أو جهاز آخر) قطعها.

إذا كانت العملية الضحية محظورة ونائمة، تضبط `kill` حالتها لـ `RUNNABLE` لكي ترجع الضحية من `sleep`. هذا أمر قد يكون خطيراً لأن الشرط المنتظر قد لا يكون صحيحاً. ومع ذلك، فإن استدعاءات `xv6` لـ `sleep` محاطة دائماً في حلقة `while` تكرارية تعيد فحص الشرط بعد رجوع `sleep`. بعض الاستدعاءات لـ `sleep` تفحص أيضاً `p->killed` في الحلقة، وتتخلّى عن النشاط الحالي إذا كانت مضبوطة. يتم هذا فقط عندما يكون ذلك التخلي صحيحاً. على سبيل المثال، شفرة قراءة وكتابة الأنبوب ترجع إذا كانت علامة القتل مضبوطة؛ وفي النهاية سترجع الشفرة للمصيدة، والتي ستفحص مجدداً `p->killed` وتخرج.

بعض الحلقات التكرارية لـ `sleep` بـ `xv6` لا تفحص `p->killed` لأن الشفرة تكون في منتصف استدعاء نظام متعدد الخطوات يجب أن يكون ذرياً (أي سيكون غير صحيح إذا تم التخلي عنه في المنتصف). برنامج تشغيل `virtio` مثال على ذلك؛ لا يفحص `p->killed` لأن عملية القرص قد تكون واحدة من طقم كتابات مطلوبة جميعها لكي يُترك نظام الملفات في حالة صحيحة. العملية التي تُقتل بينما تنتظر إدخال/إخراج القرص لن تخرج حتى تكمل استدعاء النظام الحالي وتُشاهد `usertrap` علامة القتل.

10.5 قفل العملية

القفل المرتبط بكل عملية (`p->lock`) هو القفل الأكثر تعقيداً بـ `xv6`. طريقة بسيطة للتفكير في `p->lock` هي أنه يجب الاستحواذ عليه أثناء قراءة أو كتابة أي من حقول `struct proc` التالية: `p->state`, `p->chan`, `p->killed`, `p->xstate`, `p->pid`. يمكن استخدام هذه الحقول بواسطة عمليات أخرى، أو بواسطة خيوط المجدول على نوى أخرى، لذا فمن الطبيعي أن تكون محمية بقفل.

ومع ذلك، فإن معظم استخدامات `p->lock` تحمي ثوابت أعلى مستوى لهياكل بيانات وخوارزميات العمليات بـ `xv6`. فيما يلي الطقم الكامل للأشياء التي يفعلها `p->lock`:

- جنباً إلى جنب مع `p->state`,
- يمنع التنافس في تخصيص خانات `proc` [] للعمليات الجديدة.
- يخفي العملية عن الأنظار بينما يتم إنشاؤها أو تدميرها.
- يمنع الأب من جمع عملية قامت بضبط حالتها لـ `ZOMBIE` ولكنها لم تتخل بعد عن المعالج.
- يمنع مجدول معالج آخر من القرار بتنفيذ عملية متخلية بعد ضبط حالتها لـ `RUNNABLE` ولكن قبل إتمامها لـ `swtch`.
- يضمن أن مجدول معالج واحد فقط يقرر تنفيذ عملية في حالة `RUNNABLE`.
- يمنع مقاطعة الميقاتي من التسبب في جعل العملية تتنازل بينما هي في `swtch`.
- يمنع التنافس بين `sleep` و `wakeup`.
- يمنع العملية الضحية لـ `kill` من الخروج وربما إعادة تخصيصها بين فحص `kill` لـ `p->pid` وضبط `p->killed`.
- يجعل فحص وكتابة `kill` لـ `p->state` أمراً ذرياً.

الحقل `p->parent` محمي بالقفل العام `wait_lock` بدلاً من `p->lock`. الأب للعملية هو الوحيد الذي يعدل `p->parent` على الرغم من أن الحقل يُقرأ بواسطة العملية نفسها وبواسطة عمليات أخرى تبحث عن أبنائها. الغرض من `wait_lock` هو أن يعمل كقفل الشرط عندما تنام `wait` منتظرة خروج أي ابن. الابن الخارج يحتفظ إما بـ `wait_lock` أو `p->lock` حتى بعد ضبط حالته لـ `ZOMBIE` وإيقاظ أبيه والتخلي عن المعالج. تسلسل `wait_lock` أيضاً الخروج المتزامن بواسطة الأب والابن، بحيث يضمن إيقاف عملية `init` (التي ترث الابن) من `wait` الخاصة بها. `wait_lock` قفل عام بدلاً من قفل لكل عملية في كل أب، لأنه حتى تستحوذ العملية عليه، لا يمكنها معرفة من هو أبوها.

10.6 العالم الحقيقي

تُعد `sleep` و `wakeup` طريقة مزامنة بسيطة وفعالة في مراقبات المزامنة، ولكن توجد طرق أخرى كثيرة؛ الإشارات الضوئية (Semaphores) مثال على ذلك. التحدي الأول فيها جميعاً هو تجنب مشكلة «الاستيقاظ المفقود» التي شاهدها في بداية الفصل. كانت `sleep` في نواة Unix الأصلية تعطل المقاطعات ببساطة، وهو ما كان كافياً لأن Unix كان ينفذ على نظام بمعالج واحد. تستخدم `sleep` في نظام Plan 9 دالة استدعاء راجع تنفذ وقفل الجدولة محتجز قبل النوم مباشرة؛ تعمل الدالة كفحص في اللحظة الأخيرة لشرط النوم لتجنب الاستيقاظ المفقود. تستخدم `sleep` في نواة Linux طابور عمليات صريح، يُسمى طابور الانتظار (Wait queue)، بدلاً من قناة الانتظار؛ وللطابور قفله الداخلي الخاص.

مسح الطقم الكامل للعمليات في wakeup أمر غير كفء. الحل الأفضل هو استبدال chan في كل من sleep و wakeup بهيكل بيانات يحفظ قائمة بالعمليات النائمة في ذلك الهيكل، مثل طاوور الانتظار بـ Linux. تسمى sleep و wakeup بـ Plan 9 ذلك الهيكل بـ نقطة التلاقي (Rendezvous point). تشير العديد من مكتبات الخيوط لنفس الهيكل ك متغير شرطي (Condition variable)؛ في ذلك السياق تُسمى العمليتان sleep و wakeup بـ wait و signal. كافة هذه الآليات تشترك في نفس الطابع: شرط النوم محمي بنوع من الأقفال يُترك ذرياً أثناء النوم.

توقظ wakeup بـ xv6 كافة العمليات المنتظرة في قناة انتظار محددة. إذا كان هناك أكثر من واحدة، ستحاول جميعاً الاستحواذ على قفل الشرط وإعادة فحص الشرط؛ في العديد من الحالات ستكون واحدة فقط قادرة على فعل شيء نافع (مثل قراءة كافة البيانات المنتظرة في أنبوب). البقية ستجد أن الشرط لم يعد صحيحاً وتعود للنوم؛ كان ذلك إهداراً لوقت المعالج لإيقاظها. نتيجة لذلك، توفر معظم تصاميم المتغيرات الشرطية أمرين بدائيين: signal التي توقظ واحدة من العمليات المنتظرة للمتغير الشرطي، و broadcast التي توقظها جميعاً.

القتل القسري للعمليات يطرح بعض المشاكل. على سبيل المثال، العملية المقتولة قد تكون عميقاً في النواة تنام، وفك مكدها يتطلب عناية، لأن كل دالة على مكدها الاستدعاء قد تحتاج لفعل بعض التنطيف. تساعد بعض اللغات عبر توفير آلية استثناءات، ولكن ليس C. علاوة على ذلك، هناك أحداث أخرى يمكنها التسبب في إيقاف عملية نائمة، على الرغم من أن الحدث الذي تنتظره لم يحدث بعد. على سبيل المثال، عندما تكون عملية Unix نائمة، قد ترسل عملية أخرى إشارة إليها. في هذه الحالة ترجع العملية من استدعاء النظام المنقطع بقيمة -1 ومع ضبط رمز الخطأ لـ EINTR. يمكن للتطبيق فحص تلك القيم والقرار بخصوص ما يفعله. لا يدعم xv6 الإشارات ولا ينشأ هذا التعقيد.

دعم xv6 لـ kill ليس مرضياً تماماً؛ توجد حلقات تكرارية لـ sleep ينبغي ربما أن تفحص p->killed. مشكلة مرتبطة هي أنه حتى بالنسبة لحلقات sleep التكرارية التي تفحص p->killed، توجد حالة تنافس بين sleep و kill؛ الأخيرة قد تضبط p->killed وتحاول إيقاف الضحية فور فحص حلقة الضحية لـ p->killed ولكن قبل دعوة sleep. إذا حدثت هذه المشكلة، لن تلاحظ الضحية p->killed حتى يحدث الشرط الذي تنتظره. قد يكون ذلك لاحقاً بقليل أو حتى لا يحدث أبداً (على سبيل المثال، إذا كانت الضحية تنتظر إدخالاً من لوحة التحكم، لكن المستخدم لا يكتب أي إدخال).

10.7 تمارين

1. نفذ الإشارات الضوئية العداة (Counting semaphores) بـ xv6. اختر بضعة استخدامات لـ sleep و wakeup بـ xv6 واستبدلها بالإشارات الضوئية. احكم على النتيجة.
2. هل يمكنك تنفيذ مخطط sleep/wakeup يحتاج فقط استدعاء لـ sleep () (ربما بمعاملات إضافية)، ولا يحتاج أي شيء مثل الاستدعاء لـ sleep_prepare ()؟
3. أصلح حالة التنافس المذكورة أعلاه بين kill و sleep، بحيث تؤدي kill التي تحدث بعد فحص حلقة نوم الضحية لـ p->killed ولكن قبل دعوة sleep إلى تخلي الضحية عن استدعاء النظام الحالي.
4. صمم خطة بحيث تفحص كل حلقة نوم p->killed بحيث، على سبيل المثال، العملية التي في برنامج تشغيل virtio يمكنها الرجوع سريعاً من حلقة while التكرارية إذا قُتل بواسطة عملية أخرى.

11 نظام الملفات

تتمثل الغاية الجوهرية لنظام الملفات في تنظيم البيانات وتخزينها. تدعم نظم الملفات عادةً مشاركة البيانات بين المستخدمين والتطبيقات المختلفة، فضلاً عن توفير خاصية الاستمرارية (Persistence) بحيث تظل البيانات متاحة ومحفوظة حتى بعد إعادة تشغيل الجهاز.

يوفر نظام الملفات بـ xv6 ملفات ودلائل وأسماء مسارات شجرية شبيهة بنظام Unix (انظر الفصل الفصل 2)، ويخزن بياناته على القرص الصلب لضمان الاستمرارية. يتولى نظام الملفات معالجة عدة تحديات هندسية رئيسية:

- يحتاج نظام الملفات إلى هياكل بيانات على القرص لتمثيل شجرة المجلدات والملفات المسماة، وتسجيل معرفات الكتل التي تحمل محتوى كل ملف، وتسجيل المناطق الحرة المتاحة على القرص.
 - قد تعمل عمليات متعددة على نظام الملفات في الوقت نفسه، لذا يجب أن تتنسق شفرة نظام الملفات للحفاظ على الثوابت المحفوظة.
 - يكون الوصول إلى القرص أبطأ بدرجات متعددة مقارنة بالوصول إلى الذاكرة الرئيسية RAM، لذا يجب أن يحتفظ نظام الملفات بمخزن مؤقت في الذاكرة للكتل الأكثر استخداماً.
 - يجب أن يدعم نظام الملفات التعافي من الانهيارات المفاجئة (Crash recovery)؛ أي عند وقوع انهيار (مثل انقطاع التيار الكهربائي)، يجب أن يعمل نظام الملفات بشكل صحيح بعد إعادة التشغيل. وتكمن المخاطرة في أن الانهيار قد يقاطع تسلسلاً من التحديثات ويترك هياكل بيانات القرص في حالة غير متسقة (مثل كتلة قرصية تكون مستخدمة في ملف ومسجلة في الوقت نفسه ككتلة حرة).
- يشرح بقية هذا الفصل كيف يعالج xv6 التحديات الثلاثة الأولى، بينما يركز الفصل الفصل 12 على التعافي من الانهيارات.

11.1 نظرة عامة

يُنظم تنفيذ نظام الملفات في xv6 في سبع طبقات متكاملة، كما يتضح في الشكل شكل 13: تُقرأ طبقة القرص وتكتب الكتل على محرك الأقراص الصلبة virtio؛ و virtio هو قرص محاكى يتوفره qemu. تخزن طبقة المخزن المؤقت للكتل كتل القرص مؤقتاً وتنسق الوصول إليها، متأكدة من أن عملية نواة واحدة فقط في الوقت الواحد يمكنها تعديل البيانات المخزنة في أية كتلة محددة. تتيح طبقة السجل للطبقات الأعلى تغليف التحديثات لعدة كتل في معاملة ذرية (Transaction)، وتضمن تحديث الكتل ذرياً في مواجهة الانهيارات (أي يتم تحديثها جميعاً أو لا شيء منها). توفر طبقة العقد الفهرسية ملفات فردية يمثل كل منها بـ عقدة فهرسية (Inode) تحمل رقماً فريداً وكتلاً تحمل بيانات الملف. تنفذ طبقة الدلائل كل مجلد كنوع خاص من العقد الفهرسية يحتوي محتواه على متتالية من مدخلات الدليل، كل مدخل يحوي اسم الملف ورقم عقده الفهرسية. توفر طبقة أسماء المسارات أسماء مسارات شجرية مثل usr/rtm/xv6/fs.c وتحلها عبر البحث التكراري. تجرد طبقة واصفات الملفات العديد من موارد Unix (مثل الأنابيب، الأجهزة، الملفات، إلخ) باستخدام واجهة نظام الملفات، مما يسهل حياة مبرمجي التطبيقات.

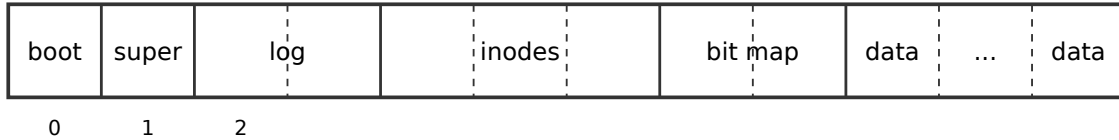
File descriptor
Pathname
Directory
Inode
Logging
Buffer cache
Disk

شكل 13: طبقات نظام الملفات في xv6.

يعرض عتاد القرص البيانات على القرص كمتتالية مرقمة من القطاعات (Sectors) بحجم 512 بايتاً لكل قطاع: القطاع 0 هو أول 512 بايتاً، والقطاع 1 هو التالي، وهكذا. يدعم عتاد القرص عمليات القراءة والكتابة فقط بمضاعفات صحيحة من القطاعات. وهكذا، على سبيل المثال، إذا احتاج نظام التشغيل تغيير بايت واحد في القرص، يجب عليه قراءة القطاع المحيط كاملاً في الذاكرة، ثم تعديل البايت الواحد، ثم إعادة كتابة القطاع كاملاً للقرص.

لهذا السبب، تخصص نظم الملفات المساحة القرصية بكتل تتكون من قطاع واحد أو أكثر. تُسمى درجة التخصيص التي يستخدمها نظام الملفات بحجم الكتلة (Block size). تستخدم معظم نظم الملفات كثيراً متعددة القطاعات لتحقيق كفاءة أفضل؛ ويستخدم xv6 حجم كتلة يتكون من قطاعين (يحدد بالثابت BSIZE في fs.h).

يحتفظ xv6 بنسخ من الكتل المقروءة في الذاكرة في كائنات من النوع struct buf. البيانات المخزنة في هذا الهيكل تكون أحياناً ليست متطابقة مع البيانات على القرص: قد لا تكون قُرئت بعد من القرص (القرص يعمل عليها ولكنه لم يُعد محتوى الكتلة بعد)، أو قد تكون عُدلت بواسطة البرمجيات ولم تُكتب بعد على القرص.



شكل 14: هيكلية تقسيم القرص في نظام الملفات في xv6.

يجب أن يمتلك نظام الملفات خطة لمكان تخزينه للعقد الفهرسية وكتل المحتوى على القرص. ولقيام بذلك، يقسم xv6 القرص إلى عدة أقسام، كما يوضح الشكل شكل 14. لا يستخدم نظام الملفات الكتلة 0 (تحمل الكتلة 0 غالباً شفرة الإقلاع، وإن كانت بـ xv6 غير مستخدمة مجرداً). تُسمى الكتلة 1 بـ الكتلة الفائقة (Superblock)؛ وتحتوي بيانات وصفية عن نظام الملفات (حجم نظام الملفات بالكتل، وعدد كتل البيانات، وعدد العقد الفهرسية، وعدد الكتل في السجل). تستقر الكتل ابتداءً من 2 في السجل. بعد السجل تأتي العقد الفهرسية، مع عقد فهرسية متعددة في الكتلة الواحدة. بعد ذلك تأتي كتل الخريطة البتية التي تتبع أية كتل بيانات مستخدمة. الكتل المتبقية هي كتل البيانات؛ كل واحدة منها إما معلمة كحزّة في كتلة الخريطة البتية، أو تحمل محتوى لملف أو مجلد.

يُنقذ نظام ملفات xv6 أولاً بواسطة برنامج خارج xv6 يُسمى mkfs. يكتب mkfs الكتلة الفائقة، ويهيئ عقدة فهرسية للمجلد الرئيسي المفرغ (Root Directory)، ويحدد كافة العقد الفهرسية وكتل البيانات كحزّة.

عند هذه النقطة، يُرجى قراءة الملفات: kernel/buf.h و kernel/fs.h و kernel/fs.c و kernel/sysfile.c و kernel/file.c.

11.2 طبقة المخزن المؤقت للكتل

يقوم المخزن المؤقت للكتل (Buffer Cache) بمهمتين: (1) تزامن الوصول لكتل القرص لضمان وجود نسخة واحدة فقط من الكتلة في الذاكرة وأن خيط نواة واحداً فقط في الوقت الواحد يستخدم تلك النسخة؛ (2) تخزين الكتل الشائعة مؤقتاً لكي لا يحتاج لإعادة قراءتها من القرص البطيء. الشفرة في bio.c.

تتكون الواجهة الرئيسية المخرجة بواسطة المخزن المؤقت من bread و bwrite؛ تحصل الأولى على buf يحوي نسخة من الكتلة المقروءة والتي يمكن قراءتها أو تعديلها في الذاكرة، وتكتب الأخيرة الحاجز المعدل للكتلة المناسبة في القرص. يجب على خيط النواة تحرير الحاجز باستدعاء brelse فور انتهائه منه. يستخدم المخزن المؤقت قفل نوم لكل حاجز لضمان أن خيطاً واحداً فقط في الوقت الواحد يستخدم كل حاجز (وبذلك كل كتلة قرصية)؛ ترجع bread حواشي مقفلة، وترفع brelse القفل.

يحتوي المخزن المؤقت على عدد ثابت من الحواجز لنقل كتل القرص، مما يعني أنه إذا طلب نظام الملفات كتلة ليست موجودة في المخزن المؤقت بالفعل، يجب على المخزن المؤقت إعادة تدوير حاجز يحتفظ حالياً بكتلة أخرى. يعيد المخزن المؤقت تدوير الحاجز الأقل استخداماً مؤخراً للكتلة الجديدة. الافتراض هو أن الحاجز الأقل استخداماً مؤخراً هو الأكثر ترجيحاً لأن لا يُستخدم مجدداً في القريب العاجل.

هناك تفاعلان بين طبقة المخزن المؤقت وطبقة السجل: (1) لتنفيذ المعاملات الذرية، تحدّث الطبقات الأعلى من نظام الملفات كتلة القرص عبر استدعاء log_write والذي يعمل كوكيل لـ bwrite وطبقة السجل فقط هي التي تدعو bwrite؛ (2) لا يمكن للمخزن المؤقت إعادة تدوير حاجز مستخدم بواسطة معاملة قائمة حتى تثبت طبقة السجل المعاملة (انظر الفصل الفصل 12). لتجنب إعادة التدوير المبكر جداً، تستخدم طبقة السجل دالتي المخزن المؤقت bpin و bunpin لتثبيت وحل ثبات الحاجز في المخزن المؤقت.

عند هذه النقطة، يُرجى قراءة الملفات: `kernel/bio.c` و `kernel/log.c`.

11.3 الكود البرمجي: المخزن المؤقت للكتل

المخزن المؤقت للكتل هو قائمة ترابطية ثنائية الاتجاه من الحواجز. تُهيئ الدالة `binit` التي تستدعيها `main()` القائمة بالحواجز المحددة بـ `NBUF` في المصفوفة الثابتة `buf`. وتتم كافة عمليات الوصول الأخرى للمخزن المؤقت عبر القائمة الترابطية بواسطة `bcache.head` وليس مصفوفة `buf`.

يرتبط بالحاجز حقلان للحالة. الحقل `valid` يشير إلى أن الحاجز يحتوي على نسخة من الكتلة. والحقل `disk` يشير إلى أن محتوى الحاجز سلم للقرص، والذي قد يغير الحاجز (مثل كتابة البيانات من القرص في `data`).

تستدعي الدالة `bread` للحصول على حاجز للكتلة المحددة. إذا احتاج الحاجز للقراءة من القرص، تستدعي الدالة `bread` `virtio_disk_rw` لفعل ذلك قبل إرجاع الحاجز.

تفحص `bget` قائمة الحواجز بحثاً عن حاجز بالجهاز ورقم الكتلة المحددين. إذا وجد ذلك الحاجز، تحصل `bget` على قفل النوم المخصص للحاجز. ثم ترجع `bget` الحاجز المقفول.

إذا لم يجد حافز مخزن للكتلة المحددة، يجب على `bget` إنشاء واحد، ربما عبر إعادة استخدام حاجز كان يحتفظ بكتلة أخرى. تتفحص قائمة الحواجز للمرة الثانية، بحثاً عن حاجز ليس قيد الاستخدام (`b->refcnt == 0`)؛ أي حاجز كذلك يمكن استخدامه. تعدل `bget` البيانات الوصفية للحاجز لتسجيل الجهاز ورقم الكتلة الجديد وتستحوذ على قفل النوم المخصص له. لاحظ أن التعيين `b->valid = 0` يضمن أن `bread` ستقرأ بيانات الكتلة من القرص بدلاً من الاستخدام الخاطئ لمحتويات الحاجز السابقة.

من المهم وجود حاجز مخزن واحد بحد أقصى لكل كتلة قرصية، لضمان أن القراءة يشاهدون الكتابات، ولأن نظام الملفات يستخدم الأقفال على الحواجز للزمانة. تضمن `bget` هذا الثابت الملازم عبر احتجاز `bcache.lock` باستمرار من فحص الحلقة الأولى لما إذا كانت الكتلة مخزنة مؤقتاً وحتى إعلان الحلقة الثانية أن الكتلة مخزنة مؤقتاً الآن (عبر ضبط `dev` و `blockno` و `refcnt`). يتسبب هذا في جعل الفحص لوجود الكتلة و(إذا لم تكن موجودة) تحديد حاجز لنقل الكتلة أمراً ذرياً.

من الآمن لـ `bget` الاستحواذ على قفل النوم الخاص بالحاجز خارج المقطع الحرج لـ `bcache.lock`، نظراً لأن القيمة غير الصفريّة لـ `b->refcnt` تمنع إعادة استخدام الحاجز لكتلة قرصية مختلفة. يحمي قفل النوم للقراءات والكتابات لمحتوى الكتلة في الذاكرة، بينما يحمي `bcache.lock` المعلومات المتعلقة بأية الكتل مخزنة مؤقتاً.

إذا كانت كافة الحواجز مشغولة، فإن عمليات كثيرة تنفذ استدعاءات نظام الملفات بالتوازي في الوقت نفسه؛ وتنفذ `bget` الهلع `panic`. استجابة أكثر ملاءمة قد تكون النوم حتى يصبح حاجز حرّاً، رغم أنه سيكون هناك عندئذ احتمالية للانسداد المتبادل (`Deadlock`).

بمجرد أن تقرأ `bread` القرص (عند الحاجة) وترجع الحاجز لمستدعيه، يكون للمستدعي الاستخدام الحصري للحاجز ويمكنه قراءة أو كتابة بايتات البيانات. إذا قام المستدعي بتعديل الحاجز، يجب عليه استدعاء `bwrite` لكتابة البيانات المعدلة للقرص قبل تحرير الحاجز. تدعو الدالة `virtio_disk_rw` للتحديث مع عتاد القرص.

عندما ينتهي المستدعي من الحاجز، يجب عليه استدعاء `brelse` لتحريره. (الاسم `brelse` واختصاره `b-release` غامض ولكن يستحق التعلم: نشأ في Unix ويُستخدم بـ BSD و Linux و Solaris أيضاً). تحرر `brelse` قفل النوم وتنقل الحاجز لمقدمة القائمة الترابطية. يتسبب نقل الحاجز في جعل القائمة مرتبة بمدى استخدام الحواجز مؤقتاً (بمعنى تحريرها): الحاجز الأول في القائمة هو الأكثر استخداماً مؤخراً، والآخر هو الأقل استخداماً مؤخراً. تستفيد الحلقات في `bget` من هذا: المسح عن حاجز موجود يجب أن يعالج كامل القائمة في أسوأ الحالات، لكن فحص الحواجز الأكثر استخداماً مؤخراً أولاً (بدءاً من `bcache.head` ومتابعة مؤشرات `next`) سيقفل وقت المسح عندما توجد مكانية جيدة للمراجع. المسح لاختيار حاجز لإعادة الاستخدام يختار الحاجز الأقل استخداماً مؤخراً عبر المسح للخلف (متابعة مؤشرات `prev`).

11.4 الكود البرمجي: مخصص الكتل

تُخزن محتويات الملفات والدلائل في كتل القرص، والتي يجب تخصيصها من مجمع حر. يحتفظ مخصص كتل `xv6` بخريطة بتية حرة على القرص، مع بت واحد لكل كتلة. يشير البت الصفري إلى أن الكتلة المناظرة حرة؛ ويشير

البت الأحد إلى أنها قيد الاستخدام. عند إنشائه لنظام ملفات جديد، يضبط mkfs البتات المناظرة لقطاع الإقلاع، والكتلة الفائقة، وكتل السجل، وكتل العقد الفهرسية، وكتل الخريطة البتية.

يوفر مخصص الكتل دالتين: balloc تخصص كتلة قرصية جديدة، و bfree تحرر كتلة. تراعي الحلقة في balloc كل كتلة، بدءاً من الكتلة 0 وحتى sb.size (عدد الكتل في نظام الملفات). تبحث عن كتلة بت خريطتها البتية صفراً، مما يشير لأنها حرة. إذا وجدت balloc كتلة كذلك، تحدّث الخريطة البتية وترجع الكتلة. من أجل الكفاءة، تُقسم الحلقة إلى جزأين. تقرأ الحلقة الخارجية كل كتلة من بتات الخريطة البتية. تفحص الحلقة الداخلية كافة بتات البتات-لكل-كتلة (BPB) في كتلة خريطة بتية واحدة. حالة التنافس التي قد تقع إذا حاولت عمليتان تخصيص كتلة في الوقت نفسه تُمنع بواسطة حقيقة أن المخزن المؤقت للكتل يتيح لعملية واحدة فقط استخدام أية كتلة خريطة بتية في الوقت الواحد.

تجد bfree كتلة الخريطة البتية المناسبة وتصفّر البت المناسب. مجدداً الاستخدام الحصري الضمني بـ bread و brelse يتجنب الحاجة لقفل صريح.

كما هو الحال مع معظم الشفرة الموصوفة في بقية هذا الفصل، يجب استدعاء balloc و bfree في معاملة سجل.

11.5 طبقة العقد الفهرسية

قد يحمل المصطلح عقدة فهرسية (Inode) واحداً من معنيين مرتبطين. قد يشير إلى هيكل البيانات على القرص الحاوي حجم الملف وقائمة أرقام كتل البيانات. أو قد تشير «العقدة الفهرسية» لعقدة فهرسية في الذاكرة، والتي تحتوي نسخة من عقدة القرص بالإضافة لمعلومات إضافية مطلوبة في النواة.

تُجمع العقد الفهرسية على القرص في منطقة متصلة من القرص تُسمى كتل العقد الفهرسية. كل عقدة فهرسية تحظى بنفس الحجم، لذا فمن السهل، بإعطاء رقم n ، العثور على العقدة الفهرسية رقم n على القرص. في الواقع، هذا الرقم n ، المسمى رقم العقدة الفهرسية أو رقم- i (i-number)، هو كيفية التعرف على العقد الفهرسية في التنفيذ.

تحدد العقدة الفهرسية على القرص بواسطة struct dinode. يميز الحقل type بين الملفات، والدلائل، والملفات الخاصة (الأجهزة). يشير النوع الصفري إلى أن العقدة الفهرسية على القرص حرة. يحسب الحقل nlink عدد مدخلات الدليل التي تشير لتلك العقدة الفهرسية، من أجل التعرف على الوقت الذي ينبغي فيه تحرير العقدة الفهرسية وكتل بياناتها على القرص. يسجل الحقل size عدد بايتات المحتوى في الملف. تسجل مصفوفة addrs أرقام كتل القرص المحتفظة بمحتوى الملف.

تحتفظ النواة بطقم العقد الفهرسية النشطة في الذاكرة في جدول يُسمى itable؛ و struct inode هو النسخة في الذاكرة لـ struct dinode على القرص. تخزن النواة العقدة الفهرسية في الذاكرة فقط إذا كانت هناك مؤشرات C تشير لتلك العقدة الفهرسية. يحسب الحقل ref عدد مؤشرات C المشاركة للعقدة الفهرسية في الذاكرة، وتتخلص النواة من العقدة الفهرسية من الذاكرة إذا هبط عداد المراجع إلى الصفر. تستحوذ الدالتان iget و iput وتفرغان المؤشرات للعقدة الفهرسية، معدلتين عداد المراجع. يمكن للمؤشرات للعقدة الفهرسية أن تأتي من واصفات الملفات، المجلدات الحالية للعمل، وشفرات النواة العابرة مثل kexec.

توجد أربع أقفال أو آليات شبيهة بالأقفال في شفرة العقد الفهرسية لـ xv6: تحمي itable.lock الثابت الملازم بكون العقدة الفهرسية موجودة في جدول العقد الفهرسية بحد أقصى مرة واحدة، والثابت الملازم بكون الحقل ref في الذاكرة يحسب عدد المؤشرات في الذاكرة للعقدة الفهرسية. تحتوي كل عقدة فهرسية في الذاكرة على حقل lock يحوي قفل نوم، والذي يضمن الوصول الحصري لحقول العقدة الفهرسية (مثل طول الملف) وكذلك لكتل المحتوى في الملف أو المجلد. يتسبب الحقل ref للعقدة الفهرسية، إذا كان أكبر من الصفر، في جعل النظام يحتفظ بالعقدة في الجدول، وعدم إعادة استخدام مدخل الجدول لعقدة فهرسية مختلفة. أخيراً، تحتوي كل عقدة فهرسية على حقل nlink (على القرص وممنسوخ في الذاكرة إذا كانت في الذاكرة) يحسب عدد مدخلات الدليل التي تشير لملف؛ لن تحرر xv6 عقدة فهرسية إذا كان عداد روابطها أكبر من الصفر.

مؤشر struct inode المرجوع بواسطة iget() يضمن كونه صالحاً حتى الاستدعاء المناظر لـ iput()؛ لن تُحذف العقدة الفهرسية، والذاكرة المشار إليها بواسطة المؤشر لن تُعاد استخدامها لعقدة فهرسية مختلفة. توفر iget() وصولاً غير حصري لعقدة فهرسية، لكي يمكن وجود مؤشرات عديدة لنفس العقدة الفهرسية. تعتمد أجزاء كثيرة

من شفرة نظام الملفات على هذا السلوك لـ `iget()`، سواء للاحتفاظ بمراجع طويلة الأمد للعقد الفهرسية (كملفات مفتوحة ومجلدات حالية) ولمنع التنافسات مع تجنب الانسداد المتبادل في الشفرة التي تتعامل مع عقد فهرسية متعددة (مثل البحث في اسم المسار).

هيكـل `struct inode` المرجوع بواسطة `iget` قد لا يحتوي على أي محتوى نافع. من أجل ضمان احتوائه نسخة من العقدة الفهرسية على القرص، يجب على الشفرة دعوة `ilock`. هذا يقفل العقدة الفهرسية (لكي لا تتمكن أية عملية أخرى من تنفيذ `ilock` عليها) ويقرأ العقدة الفهرسية من القرص، إذا لم تكن قد قُربت بالفعل. تحرر `iunlock` القفل على العقدة الفهرسية. فصل الاستحواذ على مؤشرات العقد الفهرسية عن القفل يساعد في تجنب الانسداد المتبادل في بعض الحالات، على سبيل المثال أثناء البحث في الدليل. يمكن لعمليات متعددة احتجاز مؤشر `C` لعقدة فهرسية مرجوعة بواسطة `iget`، ولكن عملية واحدة فقط يمكنها قفل العقدة الفهرسية في الوقت الواحد.

يخزن جدول العقد الفهرسية العقد الفهرسية التي تحتفظ برمجيات النواة أو هياكل بياناتها بمؤشرات `C` إليها. مهمتها الرئيسية هي تزامن الوصول بواسطة عمليات متعددة. يصادف جدول العقد الفهرسية أيضاً تخزين العقد الفهرسية كثرة الاستخدام مؤقتاً، لكن التخزين المؤقت ثانوي؛ إذا كانت العقدة الفهرسية تُستخدم بكثرة، فمن المرجح أن المخزن المؤقت للكتل سيحتفظ بها في الذاكرة. الشفرة التي تعدل عقدة فهرسية في الذاكرة تكتبها للقرص بـ `iupdate`.

11.6 الكود البرمجي: العقد الفهرسية

لتخصيص عقدة فهرسية جديدة (على سبيل المثال، عند إنشاء ملف)، تدعو `xv6` الدالة `ialloc`. تُعد `ialloc` شبيهة بـ `balloc`: تدور في حلقة تكرارية على هياكل العقد الفهرسية على القرص، كتلة في الوقت الواحد، بحثاً عن واحدة معلمة كحرة. عندما تعثر على واحدة، تطالب بها عبر كتابة النوع الجديد `type` للقرص ثم ترجع مدخلاً من جدول العقد الفهرسية مع الاستدعاء الأخير لـ `iget`. العمل الصحيح لـ `ialloc` يعتمد على حقيقة أن عملية واحدة فقط في الوقت الواحد يمكنها احتجاز مرجع لـ `bp`: يمكن لـ `ialloc` التأكد من أن عملية أخرى لن تشاهد في الوقت نفسه أن العقدة الفهرسية متاحة وتحاول المطالبة بها.

تبحث `iget` عبر جدول العقد الفهرسية عن مدخل نشط (`ip->ref > 0`) بالجهاز المطلوب ورقم العقدة الفهرسية. إذا وجدت واحداً، ترجع مرجعاً جديداً لتلك العقدة الفهرسية. بينما تسمح `iget` بتنفيذ تسجيلاً لموقع أول خانة فارغة، والتي تستخدمها إذا احتاجت لتخصيص مدخل في الجدول.

يجب على الشفرة قفل العقدة الفهرسية باستخدام `ilock` قبل قراءة أو كتابة بياناتها الوصفية أو محتواها. تستخدم `ilock` قفل نوم لهذا الغرض. بمجرد أن تحصل `ilock` على الوصول الحصري للعقدة الفهرسية، تقرأ العقدة الفهرسية من القرص (الأرجح من المخزن المؤقت) إذا لزم الأمر. تحرر الدالة `iunlock` قفل النوم، مما قد يتسبب في إيقاظ أية عمليات نائمة.

تحرر `iput` مؤشر `C` لعقدة فهرسية عبر تنقيص عداد المراجع. إذا كان هذا هو المرجع الأخير، تصبح خانة العقدة الفهرسية في جدول العقد الفهرسية حرة الآن ويمكن إعادة استخدامها لعقدة فهرسية مختلفة.

إذا شاهدت `iput` أنه لا توجد مراجع مؤشرات `C` لعقدة فهرسية وأن العقدة الفهرسية لا توجد روابط تشير إليها (تقع في لا مجلد)، فعندئذ يجب تحرير العقدة الفهرسية وكتل بياناتها. تدعو الدالة `iput` الدالة `itrunc` لقطع الملف إلى صفر بايت، محربة كتل البيانات؛ وتضبط نوع العقدة الفهرسية إلى 0 (غير مخصصة)؛ وتكتب العقدة الفهرسية للقرص. بروتوكول القفل في `iput` في الحالة التي تحرر فيها العقدة الفهرسية يستحق نظرة أقرب. أحد المخاطر هو أن خيطاً متزامناً قد يكون منتظراً في `ilock` لاستخدام هذه العقدة الفهرسية (مثلاً، لقراءة ملف أو إدراج مجلد)، ولن يكون متجهزاً لاكتشاف أن العقدة الفهرسية لم تعد مخصصة. لا يمكن حدوث هذا لأنه لا توجد طريقة لاستدعاء نظام للحصول على مؤشر لعقدة فهرسية في الذاكرة إذا لم تكن لها روابط وكان `ip->ref` واحداً. ذلك المرجع الواحد هو المرجع المملوك بواسطة الخيط الداعي لـ `iput`. الخطر الرئيسي الآخر هو أن استدعاءً متزامناً لـ `ialloc` قد يختار نفس العقدة الفهرسية التي تقوم `iput` بتحريرها. يمكن حدوث هذا فقط بعد أن يكتب `iupdate` للقرص لكي تصبح العقدة الفهرسية بنوع صفر. هذا التنافس حميد؛ الخيط المخصص سينتظر بأدب للاستحواذ على قفل نوم العقدة الفهرسية قبل قراءة أو كتابة العقدة الفهرسية، وفي تلك النقطة تكون `iput` قد انتهت منها.

يمكن لـ `iput()` الكتابة للقرص. هذا يعني أن أي استدعاء نظام يستخدم نظام الملفات قد يكتب للقرص، لأن استدعاء النظام قد يكون الأخير الحائز مرجعاً للملف. حتى استدعاءات مثل `read()` التي تبدو للقراءة فقط، قد تنتهي بدعوة `iput()`. وهذا، بدوره، يعني أنه حتى استدعاءات النظام المخصصة للقراءة فقط يجب أن تحاط في معاملات سجل إذا كانت تستخدم نظام الملفات.

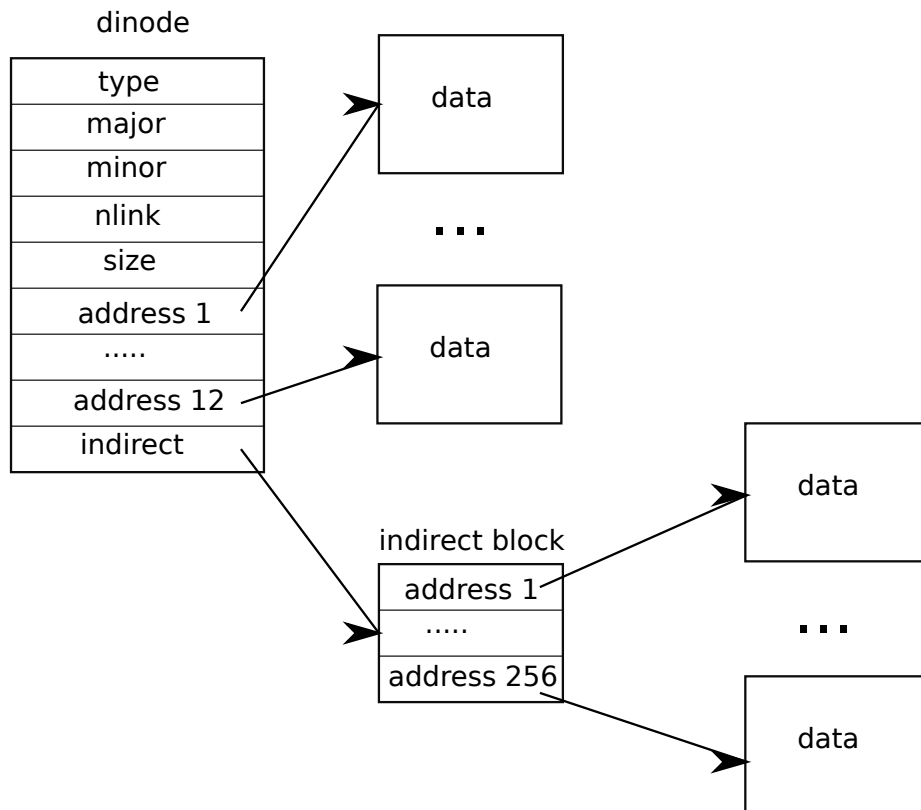
توجد تفاعلات مليئة بالتحدي بين `iput()` والانهيارات. لا تقطع `iput()` الملف فوراً عندما يهبط عداد الروابط للملف للصفر، لأن عملية ما قد تظل محتجزة لمرجع للعقدة الفهرسية في الذاكرة: عملية قد تظل تقرأ وتكتب للملف، لأنها فتحت بنجاح. ولكن، إذا وقع انهيار قبل أن تغلق العملية الأخيرة واصف الملف للملف، فعندئذ سيعلم الملف كمخصص على القرص ولكن لا يوجد مدخل دليل يشير إليه.

تتعامل نظم الملفات مع هذه الحالة بإحدى طريقتين. الحل البسيط هو أنه عند التعافي، بعد إعادة التشغيل، يسمح نظام الملفات كامل نظام الملفات عن ملفات معلمة كمخصصة ولكن لا يوجد مدخل دليل يشير إليها. إذا وجد أي ملف كذلك، فيمكنه تحرير تلك الملفات.

الحل الثاني لا يتطلب مسح نظام الملفات. في هذا الحل، يسجل نظام الملفات على القرص (مثلاً، في الكتلة الفائقة) رقم-`i` للعقدة الفهرسية للملف هبط عداد روابطه للصفر ولكن عداد مراجعة ليس صفراً. إذا أزال نظام الملفات الملف عندما يصل عداد مراجعة لـ 0، فعندئذ يحدّث القائمة في القرص عبر إزالة تلك العقدة الفهرسية من القائمة. عند التعافي، يحرر نظام الملفات أي ملف في القائمة.

لا تنفذ xv6 أيّاً من الحلين، مما يعني أن العقد الفهرسية قد تظل معلمة كمخصصة على القرص، حتى وإن لم تعد قيد الاستخدام. هذا يعني أنه مع الوقت تواجه xv6 خطر نفاذ المساحة القرصية.

11.7 الكود البرمجي: محتوى العقد الفهرسية



شكل 15: تمثيل الملف على القرص.

يهيكّل العقدة الفهرسية على القرص، `struct dinode`، يحتوي حجماً ومصفوفة من أرقام الكتل (انظر الشكل شكل 15). توجد بيانات العقدة الفهرسية في الكتل المدرجة في مصفوفة `addrs` بـ `dinode`. الكتل الـ `NDIRECT` الأولى من البيانات مدرجة في المدخلات الـ `NDIRECT` الأولى في المصفوفة؛ تُسمى هذه الكتل بـ الكتل المباشرة (`Direct`)

(blocks). الكتل الـ NINDIRECT التالية من البيانات ليست مدرجة في العقدة الفهرسية بل في كتلة بيانات تُسمى الكتلة غير المباشرة (Indirect block). المدخل الأخير في مصفوفة `addrs` يعطي عنوان الكتلة غير المباشرة. وهكذا فإن الـ 12 كيلو بايت الأولى ($NDIRECT \times BSIZE$) بايت من الملف يمكن تحميلها من كتل مدرجة في العقدة الفهرسية، بينما الـ 256 كيلو بايت التالية ($NINDIRECT \times BSIZE$) بايت يمكن تحميلها فقط بعد استشارة الكتلة غير المباشرة. هذا تمثيل جيد على القرص ولكنه معقد للعلاء. تتولى الدالة `bmap` إدارة التمثيل لكي لا تحتاج الروتينات أعلى المستوى، مثل `readi` و `writei` لإدارة هذا التعقيد. ترجع `bmap` رقم كتلة القرص للكتلة المخصصة رقم `bn` في العقدة الفهرسية `ip`. إذا لم تكن `ip` تحوز تلك الكتلة بعد، تخصص `bmap` واحدة.

تبدأ الدالة `bmap` بالنقاط الحالة السهلة: الكتل الـ `NDIRECT` الأولى مدرجة في العقدة الفهرسية نفسها. الكتل الـ `NINDIRECT` التالية مدرجة في الكتلة غير المباشرة عند `ip->addrs[NDIRECT]`. تقرأ `bmap` الكتلة غير المباشرة ثم تقرأ رقم كتلة من الموقع الصحيح في الكتلة. إذا تجاوز رقم الكتلة `NDIRECT+NINDIRECT`، تنفذ `bmap` الهلع `panic`؛ وتتضمن `writei` الفحص الذي يمنع وقوع هذا.

تخصص `bmap` الكتل حسب الحاجة. المدخل الصفري بـ `ip->addrs` أو المدخل غير المباشر الصفري يشير إلى أنه لا توجد كتلة مخصصة. عندما تواجه `bmap` أصفاراً، تستبدلها بأرقام كتل طازجة، مخصصة عند الطلب. تكرر `itrunc` تحرير كتل الملف، مجددة ضبط حجم العقدة الفهرسية للصفر. تبدأ `itrunc` بتحرير الكتل المباشرة، ثم تلك المدرجة في الكتلة غير المباشرة، وأخيراً الكتلة غير المباشرة نفسها.

تجعل `bmap` من السهل لـ `readi` و `writei` الوصول لبيانات العقدة الفهرسية. تبدأ `readi` بالتأكد من أن الإزاحة والعداد ليسا يتجاوزان نهاية الملف. القراءات التي تبدأ بعد نهاية الملف ترجع خطأ بينما القراءات التي تبدأ عند أو تعبر نهاية الملف ترجع بايتات أقل من المطلوبة. تعالج الحلقة الرئيسية كل كتلة في الملف، ناسخة البيانات من المخزن المؤقت إلى `dst`. تُعد `writei` مطابقة لـ `readi` مع ثلاثة استثناءات: الكتابات التي تبدأ عند أو تعبر نهاية الملف تنمي الملف، ووصولاً للحد الأقصى لحجم الملف؛ تكرر الحلقة نسخ البيانات في الحواشي بدلاً من الخارج؛ وإذا كانت الكتابة قد وسعت الملف، يجب على `writei` تحديث حجمه.

تنسخ الدالة `stat` البيانات الوصفية للعقدة الفهرسية في هيكل `stat` المخرج لبرامج المستخدم عبر استدعاء النظام `stat`.

11.8 الكود البرمجي: طبقة الدلائل

يُنقذ المجلد في الشفرة بشكل يشبه بالملف. العقدة الفهرسية الخاصة به تحوز النوع `T_DIR` وبياناته متتالية من مدخلات الدليل. كل مدخل هو `struct dirent` حاو اسماً ورقم عقدة فهرسية. الاسم بحد أقصى (14) `DIRSIZ` حرفاً؛ وإذا كان أقصر ينهي ببايت صفري `NULL`. مدخلات الدليل برقم عقدة فهرسية صفر تكون حرة.

السبب في احتواء مدخل الدليل على رقم العقدة الفهرسية للفيلم بدلاً من العقدة الفهرسية الكاملة للملف هو دعم «الروابط» المنشأة بواسطة استدعاء النظام `link()`. يمكن الإشارة للعقدة الفهرسية في مجلدات متعددة، وبذلك تحت أسماء مسارات متعددة؛ كل اسم من أسماء العقدة الفهرسية (أي مدخل دليل) يُسمى رابطاً. بسبب احتمالية تسمية العقدة الفهرسية في مجلدات متعددة، فمن غير المناسب تخزين العقدة الفهرسية في أي من تلك المجلدات. احتمالية الروابط المتعددة هي أيضاً السبب وراء الحقل `nlink` في العقدة الفهرسية. حقيقة تخزين العقد الفهرسية بصفة مستقلة عن الدلائل تتيح أيضاً المعالجة المنطقية لإلغاء ربط ملف (إزالته) بينما تحوز عملية ما واصف ملف يشير للملف: واصف الملف يشير للعقدة الفهرسية، وليس لأي مدخل دليل، لذا فإن واصف الملف سيظل يعمل على الرغم من إزالة الملف (في الواقع، مجرد اسمه).

تبحث الدالة `dirlookup` في مجلد عن مدخل بالاسم المحدد. إذا وجدت واحداً، ترجع مؤشراً للعقدة الفهرسية المناظرة، غير مقفول، وتضبط `poff*` لإزاحة البايت في المدخل في المجلد، في حالة رغبة المستدعي بتعديلها. إذا وجدت `dirlookup` مدخلاً بالاسم الصحيح، تحدّث `poff*` وترجع عقدة فهرسية غير مقفولة متحصل عليها عبر `iget`. `dirlookup` هي السبب في إرجاع `iget` لعقد الفهرسية غير مقفولة. المستدعي قفل `dp`، لذا لو كان البحث عن .. واسم مستعار للمجلد الحالي، فإن محاولة قفل العقدة الفهرسية قبل الإرجاع ستداول إعادة قفل `dp` والانسداد المتبادل. (توجد سيناريوهات انسداد متبادل أكثر تعقيداً تتضمن عمليات متعددة و .. واسم مستعار للمجلد الأب؛ . ليست المشكلة الوحيدة). يمكن للمستدعي تحرير قفل `dp` ثم قفل `ip` متأكداً من حيازة قفل واحد في الوقت الواحد.

تكتب الدالة `dirlink` مدخل دليل جديد بالاسم ورقم العقدة الفهرسية المحدد في المجلد `dp`. إذا كان الاسم موجوداً بالفعل، ترجع `dirlink` خطأ. تقرأ الحلقة الرئيسية مدخلات الدليل بحثاً عن مدخل غير مخصص. عندما تجد واحداً تتوقف الحلقة التكرارية مبكراً مع ضبط `off` لإزاحة المدخل المتاح. وإلا تنتهي الحلقة مع ضبط `dp->size - off`. بأية حال تنشئ `dirlink` مدخلاً جديداً في المجلد عبر الكتابة عند الإزاحة `off`.

11.9 الكود البرمجي: أسماء المسارات

يتطلب البحث في اسم المسار متتالية استدعاءات لـ `dirlookup` لكل مكون مسار. تقيم `namei` المسار وترجع العقدة الفهرسية المناظرة. الدالة `nameparent` متغير: تتوقف قبل العنصر الأخير، مرجعة العقدة الفهرسية للمجلد الأب وناسوخة العنصر النهائي في `name`. كلاهما يدعوان الدالة العامة `namex` لتنفيذ العمل الحقيقي.

تبدأ `namex` بالقرار من أين يبدأ تقييم المسار. إذا بدأ المسار بمائل، يبدأ التقييم عند الجذر؛ وإلا المجلد الحالي. ثم تستخدم `skipelem` لمراعاة كل عنصر في المسار بالدور. كل تكرار في الحلقة التكرارية يجب أن يبحث عن `name` في العقدة الفهرسية الحالية `ip`. يبدأ التكرار بقفل `ip` والفحص بكونه مجلدًا. إذا لم يكن، يفشل البحث. (قفل `ip` ضروري ليس لأن `ip->type` قد يتغير تحت الأقدام—لا يمكنه—بل لأنه حتى تنفذ `ilock` فإن `ip->type` ليس مضموناً كونه قُرى من القرص). إذا كان الاستدعاء هو `nameparent` وهذا هو عنصر المسار الأخير، تتوقف الحلقة مبكراً بصفة تعريف `nameparent`؛ عنصر المسار النهائي قُرى مسبقاً في `name` لذا تحتاج `namex` فقط لإرجاع `ip` غير المقفولة. أخيراً تبحث الحلقة عن عنصر المسار باستخدام `dirlookup` وتجهز للتكرار التالي عبر ضبط `ip = next`. عندما تنفذ الحلقة كافة عناصر المسار ترجع `ip`.

الإجراء `namex` قد يستغرق وقتاً طويلاً للإتمام: قد يتضمن عمليات قرص متعددة لقراءة العقد الفهرسية وكتل المجلدات للمجلدات المقطوعة في اسم المسار (إذا لم تكن في المخزن المؤقت للكتل). صُممت `xv6` بحذر بحيث إذا أعيق استدعاء لـ `namex` بواسطة خيط نواة على إدخال/إخراج القرص، يمكن لخيط نواة آخر يبحث في اسم مسار مختلف المتابعة بالتوازي. تقفل `namex` كل مجلد في المسار بصفة مستقلة لكي تتمكن الأبحاث في مجلدات مختلفة من المتابعة بالتوازي.

هذا التوازي يقدم بعض التحديات. على سبيل المثال بينما يبحث خيط نواة في اسم مسار قد يغير خيط نواة آخر شجرة المجلدات عبر إلغاء ربط مجلد. خطر محتمل هو أن البحث قد ينفذ في مجلد تم حذفه بواسطة خيط نواة آخر وأعيد استخدام كتله لمجلد أو ملف آخر.

تجنب `xv6` هذه التنافسات. على سبيل المثال عند تنفيذ `dirlookup` في `namex` يحتجز خيط البحث القفل على المجلد وترجع `dirlookup` عقدة فهرسية قُرى بـ `iget`. تزيد `iget` عداد المراجع للعقدة الفهرسية. فقط بعد استقبال العقدة الفهرسية من `dirlookup` تحرر `namex` القفل على المجلد. الآن قد يلغي خيط آخر ربط العقدة الفهرسية من المجلد لكن `xv6` لن تحذف العقدة الفهرسية بعد، لأن عداد المراجع للعقدة الفهرسية لا يزال أكبر من الصفر. خطر آخر هو الانسداد المتبادل. على سبيل المثال `next` يشير لنفس العقدة الفهرسية كـ `ip` عند البحث عن «...». قفل `next` قبل تحرير القفل على `ip` سينتج عنه انسداد متبادل. لتجنب هذا الانسداد المتبادل تحرر `namex` قفل المجلد قبل الاستحواذ على القفل بـ `next`. مجدداً نرى هنا لماذا يُعد الفصل بين `iget` و `ilock` أمراً هاماً.

11.10 طبقة واصفات الملفات

جانب رائع من واجهة `Unix` هو أن معظم الموارد في `Unix` تمثل كملفات، بما في ذلك الأجهزة مثل لوحة التحكم، الأنابيب، وبالطبع الملفات الحقيقية. طبقة واصفات الملفات هي الطبقة التي تحقق هذا التوحيد.

تعطي `xv6` كل عملية جدول ملفاتها المفتوحة، أو واصفات الملفات. كل ملف مفتوح يمثل بـ `struct file` والذي يعطي غلافاً حول العقدة الفهرسية أو الأنبوب، بالإضافة لإزاحة الإدخال/الإخراج. كل استدعاء لـ `open` ينشئ ملفاً مفتوحاً جديداً (`struct file` جديد): إذا فتحت عمليات متعددة نفس الملف بصفة مستقلة، ستحتل الحالات المختلفة بإزاحات إدخال/إخراج مختلفة. من ناحية أخرى فإن ملفاً مفتوحاً واحداً قد يظهر مرات متعددة في جدول ملفات عملية واحدة وأيضاً في جداول ملفات عمليات متعددة. قد يحدث هذا إذا استخدمت عملية ما `open` لفتح الملف ثم أنشأت أسماء مستعارة باستخدام `dup` أو تشاركتها مع ابن باستخدام `fork`. عداد مراجع يتتبع عدد المراجع لملف مفتوح محدد. يمكن فتح الملف للقراءة أو الكتابة أو كلاهما. الحقان `readable` و `writable` يتبعان هذا.

كافة الملفات المفتوحة في النظام تحتفظ في جدول ملفات عام `ftable`. جدول الملفات تحوزه دوال لتخصيص ملف (`filealloc`)، إنشاء مرجع مضاعف (`filedup`)، تحرير مرجع (`fileclose`)، وقراءة وكتابة البيانات (`fileread` و `filewrite`).

الدوال الثلاث الأولى تتبع النمط المعهود. تسمح `filealloc` جدول الملفات عن ملف غير مشار إليه (`f->ref == 0`) وترجع مرجعاً جديداً؛ تزيد `filedup` عداد المراجع؛ وتنقصه `fileclose`. عندما يصل عداد المراجع لملف ما إلى الصفر تحرر `fileclose` الأنبوب أو العقدة الفهرسية التحتية بناءً على النوع.

تنفذ الدوال `filestat` و `fileread` و `filewrite` عمليات `fstat` و `read` و `write` على الملفات. تُسمح `filestat` فقط على العقد الفهرسية وتدعو `stat`. تفحص `fileread` و `filewrite` يكون العملية مسموحة بنمط الفتح ثم تمرر الاستدعاء لتنفيذ الأنبوب أو العقدة الفهرسية. إذا كان الملف يمثل عقدة فهرسية، تستخدم `fileread` و `filewrite` إزاحة الإدخال/الإخراج كإزاحة للعملية ثم تقدمها. لا تحوز الأنابيب مفهوم الإزاحة. تذكر أن دوال العقد الفهرسية تتطلب من المستدعي التعامل مع القفل. قفل العقدة الفهرسية يتسبب في أثر جانبي مريح بحفظ إزاحات القراءة والكتابة محدثة ذرياً، لكي لا تتمكن عمليات كتابة متعددة لنفس الملف في الوقت نفسه من الكتابة فوق بيانات بعضها البعض.

11.11 الكود البرمجي: استدعاءات النظام

مع الدوال التي توفرها الطبقات الأدنى، فإن تنفيذ معظم استدعاءات النظام بسيط. توجد بضعة استدعاءات تستحق نظرة أقرب.

تعديل الدالتان `sys_link` و `sys_unlink` الدلائل، منشئتين أو مبرجتين مراجع للعقد الفهرسية. هما مثال آخر جيد على قوة استخدام المعاملات. تبدأ `sys_link` بجل المعاملات، نصان `old` و `new`. بافتراض أن `old` موجود وليس مجلدًا، تزيد `sys_link` عداد `ip->nlink`. ثم تدعو `sys_link` الدالة `nameparent` لإيجاد المجلد الأب وعنصر المسار النهائي بـ `new` وتنشئ مدخل دليل جديد يشير لعقدة `old` الفهرسية. المجلد الأب الجديد يجب أن يكون موجوداً وعلى نفس الجهاز كالعقدة الفهرسية القائمة: أرقام العقد الفهرسية لها معنى فريد على قرص واحد فقط. إذا وقع خطأ مثل هذا، يجب على `sys_link` العودة وتنقيص `ip->nlink`.

تبسط المعاملات التنفيذ لأنها تتطلب تحديث كتل قرص متعددة، ولكن لا يتعين علينا القلق بشأن الترتيب الذي نفعله به. إما ستنجح جميعاً أو لا شيء. على سبيل المثال بدون معاملات فإن تحديث `ip->nlink` قبل إنشاء رابط سيضع نظام الملفات في حالة غير آمنة المؤقتة والانهيال في المنتصف سيتسبب في فوضى. مع المعاملات لا يتعين علينا القلق بشأن هذا.

تنشئ `sys_link` اسماً جديداً لعقدة فهرسية قائمة. تنشئ الدالة `create` اسماً جديداً لعقدة فهرسية جديدة. هي تميم لاستدعاءات نظام إنشاء الملفات الثلاثة: `open` مع علامة `O_CREATE` تنشئ ملفاً عادي جديداً، `mkdir` تنشئ مجلدًا جديداً، و `mkdev` تنشئ ملف جهاز جديد. مثل `sys_link` تبدأ `create` بدعوة `nameparent` للحصول على العقدة الفهرسية للمجلد الأب. ثم تدعو `dirlookup` للفحص بكون الاسم موجوداً بالفعل. إذا كان الاسم موجوداً بالفعل يتوقف سلوك `create` على أية استدعاء نظام تُستخدم لأجله: `open` تحوز دلالات مختلفة عن `mkdir` و `mkdev`. إذا كانت `create` تُستخدم نيابة عن `open (type == T_FILE)` والاسم الموجود هو نفسه ملف عادي، فإن تعامل ذلك كنجاح، لذا تفعل `create` ذلك أيضاً. وإلا فإنه خطأ. إذا لم يكن الاسم موجوداً بالفعل تفعل `create` الآن تخصيص عقدة فهرسية جديدة بـ `ialloc`. إذا كانت العقدة الفهرسية الجديدة مجلدًا، تهيئها `create` بمدخلي . و ... أخيراً، الآن بعد تهيئة البيانات بأسلوب صحيح، يمكن لـ `create` ربطها في المجلد الأب. تستحوذ `create` مثل `sys_link` على قفلي عقدتين فهرسيتين في الوقت نفسه: `ip` و `dp`. لا توجد احتمالية للانسداد المتبادل لأن العقدة الفهرسية `ip` مخصصة طازجة: لا توجد عملية أخرى في النظام تستحوذ على قفل `ip` وتدعو القفل لـ `dp`.

باستخدام `create` يسهل تنفيذ `sys_open` و `sys_mkdir` و `sys_mknod`. تُعد `sys_open` الأكثر تعقيداً، لأن إنشاء ملف جديد مجرد جزء صغير مما يمكنها فعله. إذا أمضيت علامة `O_CREATE` لـ `open` تدعو `create`. وإلا تدعو `namei`. ترجع `create` عقدة فهرسية مقفولة ولكن `namei` لا تفعل ذلك، لذا يجب على `sys_open` قفل العقدة الفهرسية بنفسها. يوفر هذا مكاناً مناسباً للفحص بكون المجلدات تُفتح فقط للقراءة وليس الكتابة. بافتراض الحصول على العقدة الفهرسية بأحد الطريقتين، تخصص `sys_open` ملفاً وواصف ملف ثم تملأ الملف. لاحظ أن أية عملية أخرى لا يمكنها الوصول للملف المهيأ جزئياً نظراً لأنه يوجد فقط في جدول العملية الحالية.

فحص الفصل الفصل 10 تنفيذ الأنابيب قبل حوزتنا لنظام ملفات. تصل الدالة `sys_pipe` ذلك التنفيذ بنظام الملفات عبر توفير طريقة لإنشاء زوج أنابيب. معاملها مؤشر لمساحة لعددتين صحيحين، حيث ستسجل واصفي الملفات الجديدين. ثم تخصص الأنبوب وتثبت واصفي الملفات.

11.12 العالم الحقيقي

المخزن المؤقت للكتل في نظام تشغيل في العالم الحقيقي أكثر تعقيداً بدرجة كبيرة عن `xv6` لكنه يخدم نفس الغرضين: التخزين المؤقت وتزامن الوصول للقرص. يستخدم المخزن المؤقت لـ `xv6` سياسة إخلاء الأقل استخداماً مؤخراً (LRU)؛ توجد سياسات أكثر تعقيداً يمكن تنفيذها. مخزن مؤقت لـ LRU أكثر كفاءة سيلغي القائمة الترابطية مستخدماً بدلاً منها جدول تجزئة للأبحاث وكومة لإخلاءات LRU. المخازن المؤقتة الحديثة تُدمج عادة بنظام الذاكرة الافتراضية لدعم الملفات المربوطة بالذاكرة.

تستخدم `xv6` تخطيط قرص للعقد الفهرسية والدلائل يشبه بـ UNIX المبكر؛ هذا المخطط استمر بدرجة ملحوظة عبر السنين. UFS/FFS بـ BSD و ext2/ext3 بـ Linux تستخدم جوهرياً نفس هياكل البيانات. الجزء الأكثر عدم كفاءة في تخطيط نظام الملفات هو المجلد، والذي يتطلب مسحاً خطياً عبر كافة كتل القرص أثناء كل بحث. هذا معقول عندما تكون المجلدات مجرد بضع كتل قرصية، ولكنه مكلف للمجلدات المحتفظة بملفات كثيرة. NTFS بـ Microsoft Windows و HFS بـ macOS و ZFS بـ Solaris تنفذ المجلد كشجرة متوازنة من الكتل على القرص. هذا معقد ولكن يضمن أبحاث مجلدات في وقت لوغاريتمي.

تتطلب `xv6` ملائمة نظام الملفات على جهاز قرص واحد وعدم تغير الحجم. مع قيادة قواعد البيانات العملاقة وملفات الوسائط المتعددة لمتطلبات التخزين أعلى من أي وقت مضى، تطور نظم التشغيل طرقاتاً لإلغاء اختناق «قرص واحد لكل نظام ملفات». النهج الأساسي هو دمج أقراص متعددة في قرص منطقي واحد. الحلول العادية مثل RAID لا تزال الأكثر شعبية، لكن الاتجاه الحالي يتحرك نحو تنفيذ أكبر قدر ممكن من هذا المنطق في البرمجيات. تلك التنفيذات البرمجية تتيح وظيفية غنية مثل تنمية أو تقليص الجهاز المنطقي عبر إضافة أو إزالة أقراص في الهواء. بالطبع طبقة تخزين يمكنها التنمية أو التقليص في الهواء تتطلب نظام ملفات يمكنه فعل نفس الشيء؛ مصفوفة كتل العقد الفهرسية ثابتة الحجم المستخدمة بـ `xv6` لن تعمل جيداً في مثل تلك البيئات. فصل إدارة القرص عن نظام الملفات قد يكون التصميم الأنسب، لكن الواجهة المعقدة بينهما قادت بعض النظم مثل ZFS لدعم دمجها.

يفتقر نظام ملفات `xv6` للعديد من الميزات الأخرى في نظم الملفات الحديثة؛ على سبيل المثال يفتقر للدعم لـ snapshots والنسخ الاحتياطي التزايدية.

تسمح نظم Unix الحديثة بالوصول لأنواع عديدة من المورد باستدعاءات النظام نفسها كالتخزين على القرص: الأنابيب المسماة، اتصالات الشبكة، نظم الملفات الشبكية عن بعد، وواجهات المراقبة والتحكم مثل `proc`. بدلاً من عبارات `if` بـ `xv6` في `filewrite` و `fileread` تعطي هذه النظم كل ملف مفتوح جدولاً من مؤشرات الدوال وتدعو مؤشر الدالة لاستدعاء تنفيذ تلك العقدة الفهرسية للاستدعاء.

11.13 تمارين

1. لماذا الهلع `panic` في `ballocc`؟ هل يمكن لـ `xv6` التعافي؟
2. لماذا الهلع `panic` في `ialloc`؟ هل يمكن لـ `xv6` التعافي؟
3. لماذا لا تنفذ `filealloc` الهلع `panic` عندما تنفذ منها الملفات؟ لماذا هذا أكثر شيوعاً ويستحق المعالجة؟
4. افترض أن الملف المناظر لـ `ip` ألغى ربطه بواسطة عملية أخرى بين استدعاءات `sys_link` لـ `iunlock(ip)` و `dirlink`. هل سينشأ الرابط بأسلوب صحيح؟ لماذا أو لماذا لا؟
5. تنفذ `create` أربعة استدعاءات دوال (واحد لـ `ialloc` وثلاثة لـ `dirlink`) تتطلب النجاح. إذا لم ينجح أي منها تدعو `create` الهلع `panic`. لماذا يُعد هذا مقبولاً؟ لماذا لا يمكن لأي من الاستدعاءات الأربعة الفشل؟
6. تدعو `sys_chdir` الدالة `iunlock(ip)` قبل `iput(cp->cwd)` والتي قد تحاول قفل `cp->cwd` ورغم ذلك فإن تأجيل `iunlock(ip)` حتى بعد `iput` لن يتسبب في انسداد متبادل. لماذا لا؟
7. نفذ استدعاء النظام `lseek`. دعم `lseek` سيتطلب أيضاً تعديل `filewrite` لملء الثقوب في الملف بأصفار إذا ضبطت `lseek` الإزاحة `off` بعد `ip->size`.
8. أضف `O_APPEND` و `O_TRUNC` لـ `open` لكي تعمل المعاملات `>` و `>>` في الشاشة.

9. عدل نظام الملفات لدعم الروابط الرمزية (Symbolic links).
10. عدل نظام الملفات لدعم الأنابيب المسماة (Named pipes).
11. عدل نظام الملفات ونظام الذاكرة الافتراضية لدعم الملفات المربوطة بالذاكرة (mmap).

12 تسجيل المعاملات

واحدة من أكثر المشاكل إثارة في تصميم نظام الملفات هي التعافي من الانهيارات المفاجئة. تنشأ المشكلة لأن العديد من عمليات نظام الملفات تتطلب كتابات متعددة للقرص، والانهيار بعد طقم فرعي من الكتابات قد يترك نظام الملفات على القرص في حالة غير متسقة. على سبيل المثال، افترض وقوع انهيار أثناء قطع ملف. يتضمن القطع كتابة (على الأقل) كتلتي قرص، واحدة تحوي العقدة الفهرسية والأخرى تحوي جزءاً من خريطة البتات الحرة: يجب ضبط طول العقدة الفهرسية إلى الصفر وتصغير أرقام الكتل في العقدة الفهرسية، ويجب ضبط بتات الخريطة البتية لكتل الملف للصفر (معلمة إياها كحرة). الانهيار بعد واحدة من الكتابات، وقبل أن تحوز النواة فرصة لتنفيذ الكتابة الثانية، سيترك نظام الملفات على القرص في حالة غير صحيحة. إذا وقعت كتابة الخريطة البتية فقط، فإن النتيجة بعد الانهيار وإعادة الإقلاع ستكون عقدة فهرسية بطول غير صفري تشير لكتل معلمة كحرة؛ وإذا وقعت كتابة العقدة الفهرسية فقط، فإن النتيجة ستكون كتلاً ليست مستخدمة بواسطة أي ملف ولكنها أيضاً ليست معلمة كحرة.

الأخيرة حميدة نسبياً، لكن العقدة الفهرسية التي تشير لكتلة محررة يُرجح أن تتسبب في مشاكل خطيرة بعد إعادة الإقلاع. بعد إعادة الإقلاع قد تخصص النواة تلك الكتلة لملف آخر، والآن نحوز ملفين مختلفين يشيران غير قصد لنفس الكتلة. لو كانت xv6 تدعم مستخدمين متعددين لكانت هذه الحالة مشكلة أمنية، نظراً لأن مالك الملف القديم سيكون قادراً على قراءة وكتابة كتل في الملف الجديد المملوك لمستخدم مختلف.

تحل xv6 مشكلة الانهيارات أثناء عمليات نظام الملفات بصيغة بسيطة من التسجيل. استدعاء نظام xv6 لا يكتب مباشرة هياكل بيانات نظام الملفات على القرص. بدلاً من ذلك، يضع وصفاً لكافة كتابات القرص التي يرغب بإجرائها في سجل (Log) على القرص. بمجرد أن يسجل استدعاء النظام كافة كتاباته، يكتب سجل تثبيت (Commit) خاص للقرص يشير إلى أن السجل يحوي عملية كاملة. عند تلك النقطة ينسخ استدعاء النظام الكتابات لهياكل بيانات نظام الملفات على القرص. بعد اكتمال تلك الكتابات، يسمح استدعاء النظام السجل على القرص.

إذا انهار النظام وأعاد الإقلاع، فإن شفرة نظام الملفات تتعافى من الانهيار كما يلي، قبل تشغيل أية عمليات. إذا كان السجل معلماً بكونه يحوي عملية كاملة، فإن شفرة التعافي تنسخ الكتابات إلى حيث تنتمي في نظام الملفات على القرص. إذا لم يكن السجل معلماً بكونه يحوي عملية كاملة، تتجاهل شفرة التعافي السجل. تنهي شفرة التعافي عملها بمسح السجل.

لماذا يحل سجل xv6 مشكلة الانهيارات أثناء عمليات نظام الملفات؟ إذا وقع الانهيار قبل تثبيت العملية، فلن يعلم السجل على القرص ك مكتمل، وستتجاهله شفرة التعافي، وستكون حالة القرص كما لو أن العملية لم تبدأ مطلقاً. إذا وقع الانهيار بعد تثبيت العملية، فإن التعافي سيعيد تشغيل كافة كتابات العملية، ربما مكرراً إياها إذا كانت العملية قد بدأت كتابتها لهيكل البيانات على القرص. بأية حال يجعل السجل العمليات ذرية فيما يتعلق بالانهيارات: بعد التعافي إما تشير كافة كتابات العملية على القرص، أو لا يظهر أي منها.

12.1 تصميم السجل

يستقر السجل في موقع ثابت معروف (انظر الشكل شكل 14)، محدد في الكتلة الفائقة. يتكون من كتلة رأس متبوعة بمتتالية من نسخ الكتل المحدثة («الكتل المسجلة»). تحتوي كتلة الرأس على مصفوفة من أرقام الكتل، واحدة لكل من الكتل المسجلة، وعداد كتل السجل. العداد في كتلة الرأس على القرص إما صفر، مما يشير إلى عدم وجود معاملة في السجل، أو غير صفري، مما يشير إلى أن السجل يحوي معاملة مثبتة كاملة بعدد الكتل المسجلة المحدد. تكتب xv6 كتلة الرأس عندما تثبت معاملة ما، وليس قبل ذلك، وتضبط العداد للصفر بعد نسخ الكتل المسجلة لنظام الملفات. وهكذا فإن انهياراً في منتصف معاملة سينتج عنه عداد صفري في كتلة رأس السجل؛ والانهيار بعد التثبيت سينتج عنه عداد غير صفري.

تشير شفرة كل استدعاء نظام لبداءية ونهاية متتالية الكتابات التي يجب أن تكون ذرية فيما يتعلق بالانهيارات. للسماح بالتنفيذ المتزامن لعمليات نظام الملفات بواسطة عمليات مختلفة، يمكن لنظام التسجيل تجميع كتابات استدعاءات

نظام متعددة في معاملة واحدة. وهكذا فإن تثبيتاً واحداً قد يتضمن كتابات استدعاءات نظام مكتملة متعددة. لتجنب تقسيم استدعاء نظام عبر المعاملات، فإن نظام التسجيل يثبت فقط عندما لا تكون هناك استدعاءات نظام لنظام الملفات جارية.

فكرة تثبيت عدة معاملات معاً تُعرف بـ التثبيت الجماعي (Group commit). يقلل التثبيت الجماعي عدد عمليات القرص لأنه يوزع الكلفة الثابتة للتثبيت عبر عمليات متعددة. كما يسلم التثبيت الجماعي لنظام القرص كتابات متزامنة أكثر في الوقت نفسه، ربما متيحاً للقرص كتابتها جميعاً أثناء دوران قرصي واحد. برنامج تشغيل virtio بـ xv6 لا يدعم هذا النوع من التجميع الدفعي، لكن تصميم نظام ملفات xv6 يتيح ذلك.

أثر جانبي دقيق للتثبيت الجماعي هو أنه يجعل آثار استدعاء نظام الكتابة غير متزامنة: أي أنه حتى بعد رجوع استدعاء نظام الكتابة للتطبيق، فليس مضموناً أن الكتل التي عدلها أصبحت على القرص. على سبيل المثال عند انتهاء معاملة كتابة، قد تكون معاملة أخرى قد بدأت وستثبت معاملة الكتابة مع المعاملة الثانية كمجموعة واحدة، مؤجلة آثار الكتابة على القرص حتى تثبت المعاملة الثانية. إذا أراد تطبيق الانتظار حتى تصبح تعديلاته دائمة (أي على القرص)، يمكنه استدعاء استدعاء النظام sync بعد استدعاء نظام الكتابة؛ تنتظر sync حتى يكتمل تثبيت جماعي قيد التنفيذ.

تخصص xv6 مقداراً ثابتاً من المساحة على القرص لاحتجاز السجل. العدد الإجمالي للكتل المكتوبة بواسطة استدعاءات النظام في معاملة يجب أن يتسع في تلك المساحة. هذا يحوز أثرين. لا يُسمح لأي استدعاء نظام بمفرده كتابة كتلة متميزة أكثر مما توجد مساحة في السجل. هذه ليست مشكلة لمعظم استدعاءات النظام، لكن اثنين منها يمكنهما كتابة كتل كثيرة: write و unlink. كتابة ملف ضخم قد تكتب كتل بيانات كثيرة وكتل خريطة بنية كثيرة بالإضافة لعقدة فهرسية؛ وإلغاء ربط ملف ضخم قد يكتب كتل خريطة بنية كثيرة وعقدة فهرسية. يقسم استدعاء نظام الكتابة بـ xv6 الكتابات الضخمة لكتابات أصغر متعددة تتسع في السجل، و unlink لا تتسبب في مشاكل لأن نظام ملفات xv6 في الممارسة يستخدم كتلة خريطة بنية واحدة فقط. الأثر الآخر لمساحة السجل المحدودة هو أن نظام التسجيل لا يمكنه السماح لاستدعاء نظام بالبدا ما لم يكن متأكداً من أن كتابات استدعاء النظام ستستوعب في المساحة المتبقية في السجل.

12.2 الكود البرمجي: التسجيل

يبدو الاستخدام المعهود للسجل عند استدعاء النظام كالتالي:

```
begin_op();
...
bp = bread(...);
data = [...];
log_write(bp);
...
end_op();
```

تنتظر begin_op حتى يكون نظام التسجيل غير مثبت حالياً، وحتى توجد مساحة سجل غير محجوزة كافية لاحتجاز الكتابات من هذا الاستدعاء. تحسب log.outstanding عدد استدعاءات النظام التي حجزت مساحة سجل؛ المساحة المحجوزة الإجمالية هي log.outstanding مضروبة في MAXOPBLOCKS. زيادة log.outstanding تحجز المساحة وتمنع وقوع تثبيت أثناء استدعاء النظام هذا. تفترض الشفرة بأسلوب محافظ أن كل استدعاء نظام قد يكتب ما يصل إلى MAXOPBLOCKS كتلة متميزة.

تعمل log_write كوكيل لـ bwrite. تسجل رقم الكتلة للكتلة في الذاكرة، حازرة لها خانة في السجل على القرص، وتثبت الحاجز في المخزن المؤقت للكتل لمنع المخزن المؤقت من إخلائه. يجب أن تظل الكتلة في المخزن المؤقت حتى تثبت؛ حتى ذلك الحين فإن النسخة المخزنة مؤقتاً هي المسجل الوحيد للتعديل؛ لا يمكن كتابتها لمكانها على القرص إلا بعد التثبيت؛ والقراءات الأخرى في نفس المعاملة يجب أن تشاهد التعديلات. تلاحظ log_write عندما تُكتب كتلة ما مرات متعددة أثناء معاملة واحدة، وتخصص لتلك الكتلة نفس الخانة في السجل. هذا التحسين يُسمى غالباً الامتصاص (Absorption). من الشائع، على سبيل المثال، أن كتلة القرص الحاوية لعقد فهرسية لعدة ملفات تُكتب مرات متعددة في معاملة. عبر امتصاص كتابات قرص متعددة في واحدة، يمكن لنظام الملفات توفير مساحة السجل وتحقيق أداء أفضل لأن نسخة واحدة فقط من كتلة القرص يجب أن تُكتب للقرص.

تخفض end_op أولاً عداد استدعاءات النظام الجارية. إذا كان العداد صفراً الآن، تثبت المعاملة الحالية عبر دعوة commit(). توجد أربع مراحل في هذه العملية. تنسخ write_log() كل كتلة معدلة في المعاملة من المخزن المؤقت للكتل لخانتها في السجل على القرص. تكتب write_head() كتلة الرأس للقرص؛ هذه هي نقطة التثبيت، والانتهاء بعد الكتابة سينتج عنه تعاف يعيد تشغيل كتابات المعاملة من السجل. تقرأ install_trans كل كتلة من السجل وتكتبها للمكان المناسب في نظام الملفات. أخيراً تكتب end_op رأس السجل مع عداد صفراً؛ هذا يجب أن يحدث قبل أن تبدأ المعاملة التالية كتابة الكتل المسجلة، لكي لا ينتج عن انهيار استخدام التعافي لرأس معاملة مع الكتل المسجلة للمعاملة التالية.

فرضية رئيسية في تنفيذ end_op هي أن write_head تحدد كتلة الرأس ذرياً: بالكامل أو لا شيء على الإطلاق. تذكر من الفصل الفصل 11 أن الكتلة بـ xv6 هي قطاعاً قرص، لكن رأس السجل يتسع في القطاع الأول، وبذلك لا توجد مشكلة حتى وإن كان القرص يضمن فقط أن التحديث لقطاع واحد ذري.

تُدعى recover_from_log من initlog والتي تُدعى من fsinit أثناء الإقلاع قبل تنفيذ أول عملية مستخدم. تقرأ رأس السجل، وتحاكي أفعال end_op إذا كان الرأس يشير إلى أن السجل يحوي معاملة مثبتة.

مثال على استخدام السجل يقع في fwrite. تبدو المعاملة كالتالي:

```
begin_op();
ilock(f->ip);
r = writei(f->ip, ...);
iunlock(f->ip);
end_op();
```

هذه الشفرة محاطة في حلقة تكرارية تقسم الكتابات الضخمة لمعاملات فردية من بضع كتل فقط في الوقت الواحد، لتجنب إفاضة السجل. استدعاء writei يكتب كتل كثيرة كجزء من هذه المعاملة: العقدة الفهرسية للملف، كتلة أو أكثر من كتل الخريطة البتية، وبعض كتل البيانات.

12.3 العالم الحقيقي

نظام التسجيل بـ xv6 غير كفء. لا يمكن لوقوع تثبيت أن يتزامن مع استدعاءات نظام لنظام الملفات. يسجل النظام كتل كاملة، حتى وإن جرى تغيير بضع بايتات فقط في كتلة. ينفذ كتابات سجل متزامنة، كتلة في الوقت الواحد، كل منها يرجح أن تتطلب كامل زمن دوران القرص. نظم التسجيل الحقيقية تعالج كافة هذه المشاكل.

التسجيل ليس الطريقة الوحيدة لتوفير التعافي من الانهيارات. استخدمت نظم الملفات المبكرة منطفاً أثناء إعادة الإقلاع (على سبيل المثال، برنامج fsck بـ UNIX) لفحص كل ملف ومجلد وقوائم الكتل والعقد الفهرسية الحرة، بحثاً عن ودعماً لحل عدم الاتساق. المنظف قد يستغرق ساعات لنظم الملفات الضخمة، وتوجد حالات لا يكون من الممكن فيها حل عدم الاتساق بطريقة تجعل استدعاءات النظام الأصلية ذرية. التعافي من سجل أسرع بكثير ويتسبب في جعل استدعاءات النظام ذرية في مواجهة الانهيارات.

12.4 تمارين

1.

لفهم كيفية استخدام السجل، أدخل السطر البرمجي التالي قبل عبارة log_write في الدالة writei في kernel/

```
fs.c: printf("log write %d
```

;d %d %d\n", addr, bp->blockno, off, n% نفذ make clean ثم make qemu، وفي موجه أوامر xv6 نفذ

```
الامر التالي: $ f > hi echo ينبغي لك مشاهدة مخرجات شبيهة بهذا: log write 47
```

```
:log write 936 16 368 47
```

```
:log write 936 2 0 936
```

1 2 936 تشير المخرجات إلى أن log_write دُعيت ثلاث مرات لـ أمر موجه الأوامر هذا. لماذا دُعيت log_write 3 مرات؟ ماذا تكتب كل log_write؟ هل الكتابة الأولى أعلاه تناظر تحديث المجلد الرئيسي والكتابتان الأخريان تناظران كتابة الملف f؟ قد يكون الشكل شكل 14 نافعاً ولا تتردد في إضافة عبارات printf أكثر لمساعدتك في الإجابة عن السؤال.

13 العودة لموضوع التزامن

تُعد المحافظة في الوقت نفسه على الأداء المتوازي الجيد، والصحة على الرغم من التزامن، والشفرة سهلة الفهم، تحدياً كبيراً في تصميم النواة. إن الاستخدام المباشر للأقفال هو أفضل طريق للصحة، ولكنه ليس ممكناً دائماً. يسلط هذا الفصل الضوء على أمثلة يُجبر فيها xv6 على استخدام الأقفال بأسلوب معقد، وأمثلة يستخدم فيها xv6 تقنيات شبيهة بالأقفال ولكنها ليست أقفالاً صريحة.

13.1 أنماط القفل

غالباً ما تشكل العناصر المخزنة مؤقتاً تحدياً في القفل. على سبيل المثال، يخزن المخزن المؤقت لكتل نظام الملفات نسخاً لما يصل إلى NBUF من كتل القرص. من الجوهر أن تحظى كتلة قرصية محددة بنسخة واحدة فقط في المخزن المؤقت؛ وإلا فقد تحدث عمليات مختلفة تغييرات متضاربة على نسخ مختلفة لما ينبغي أن يكون نفس الكتلة. تُخزن كل كتلة مخزنة مؤقتاً في هيكل struct buf. يحتوي هيكل struct buf على حقل قفل يساعد في ضمان أن عملية واحدة فقط تستخدم كتلة قرصية محددة في الوقت الواحد. ومع ذلك، فإن ذلك القفل ليس كافياً؛ ماذا لو لم تكن الكتلة موجودة في المخزن المؤقت مطلقاً، وأرادت عمليتان استخدامها في الوقت نفسه؟ لا يوجد هيكل struct buf (نظراً لأن الكتلة ليست مخزنة مؤقتاً بعد)، وبالتالي لا يوجد شيء ليُقفّل. يتعامل xv6 مع هذه الحالة عبر ربط قفل إضافي (bcache.lock) مع طقم هويات الكتل المخزنة مؤقتاً. الشفرة التي تحتاج لفحص ما إذا كانت كتلة ما مخزنة مؤقتاً (مثل bget)، أو تغيير طقم الكتل المخزنة مؤقتاً، يجب أن تستحوذ على bcache.lock؛ وبعد أن تعثر تلك الشفرة على الكتلة وهيكل struct buf الذي تحتاجه، يمكنها تحرير bcache.lock وقفل الكتلة المحددة فقط. هذا نمط شائع: قفل واحد لطقم العناصر، بالإضافة لقفل واحد لكل عنصر.

عادةً ما تقوم نفس الدالة التي تستحوذ على قفل بتحريره. ولكن الطريقة الأكثر دقة للنظر إلى الأمور هي أن القفل يُستحوذ عليه في بداية تسلسل يجب أن يبدو ذرياً، ويُحرر عندما ينتهي ذلك التسلسل. إذا بدأ التسلسل وانتهى في دوال مختلفة، أو خيوط مختلفة، أو على معالجات مختلفة، فإن الاستحواذ على القفل وتحريره يجب أن يفعل نفس الشيء. وظيفة القفل هي إجبار الاستخدامات الأخرى على الانتظار، وليس تثبيت قطعة بيانات إلى وكيل محدد. أحد الأمثلة على ذلك هو الاستحواذ acquire في yield والذي يُحرر في خيط المجدول بدلاً من العملية المستحوذة. مثال آخر هو acquire_sleep في ilock؛ تنام هذه الشفرة غالباً أثناء قراءة القرص؛ وقد تستيقظ على معالج مختلف، مما يعني أن القفل قد يُستحوذ عليه ويُحرر على معالجات مختلفة.

إن تحرير كائن محمي بقفل مدمج في الكائن أمر دقيق، لأن امتلاك القفل ليس كافياً لضمان أن التحرير سيكون صحيحاً. تنشأ حالة المشكلة عندما يكون خيط آخر ينتظر في acquire لاستخدام الكائن؛ تحرير الكائن يحرر ضمناً القفل المدمج، مما يتسبب في تعطل الخيط المنتظر. أحد الحلول هو تتبع عدد المراجع للوجود في الكائن، بحيث لا يُحرر إلا عندما يختفي آخر مرجع. انظر pipeclose كمثال؛ حيث يتتبع pi->readopen و pi->writeopen ما إذا كان للأنبوب واصفات ملفات تشير إليه.

عادةً ما نرى أقفالاً حول تسلسلات القراءة والكتابة لأطقم من العناصر المرتبطة؛ تضمن الأقفال أن الخيوط الأخرى تشاهد فقط التسلسلات المكتملة للتحديثات (طالما أنها تقفل هي الأخرى). ماذا عن الحالات التي يكون فيها التحديث كتابة بسيطة لمتغير مشترك واحد؟ على سبيل المثال، تدعو setkilled و killed الأقفال حول استخدامهما البسيط لـ p->killed. إذا لم يكن هناك قفل، فيمكن لخيط كتابة p->killed في الوقت نفسه الذي يقرؤه فيه خيط آخر. هذه حالة تنافس (Race)، وتنص مواصفة لغة C على أن التنافس يعطي سلوكاً غير محدد (Undefined behavior)، مما يعني أن البرنامج قد ينهار أو يعطي نتائج غير صحيحة. تمنع الأقفال التنافس وتتجنب السلوك غير المحدد.

أحد الأسباب التي تجعل حالات التنافس تكسر البرامج هو أنه إذا لم تكن هناك أقفال أو تركيبات مكافئة، فقد ينشئ المترجم شفرة آلة تقرأ وتكتب الذاكرة بطرق مختلفة تماماً عن شفرة C الأصلية. على سبيل المثال، شفرة الآلة لخيط يدعو killed قد تنسخ p->killed لسجل وتقرأ فقط تلك القيمة المخزنة مؤقتاً؛ سيكون معنى هذا أن الخيط قد لا يشاهد مطلقاً أية كتابات لـ p->killed. تمنع الأقفال مثل هذا التخزين المؤقت.

13.2 أنماط شبيهة بالأقفال

في العديد من الأماكن يستعين xv6 بعدد مراجع أو علامة بأسلوب يشبه بالقفل للإشارة إلى أن كائناً ما مخصص ولا ينبغي تحريره أو إعادة استخدامه. يعمل p->state الخاص بالعملية بهذه الطريقة، كما تفعل عدادات المراجع

في هياكل file و inode و buf. بينما يحمي قفل في كل حالة العلامة أو عداد المراجع، فإن الأخير هو الذي يمنع التحرير المبكر للكائن.

يستخدم نظام الملفات عدادات مراجع struct inode كنوع من القفل المشترك الذي يمكن احتجازه بواسطة عمليات متعددة، من أجل تجنب الانسدادات المتبادلة التي كانت ستحدث لو استخدمت الشفرة أقفاً عادية. على سبيل المثال، الحلقة التكرارية في namex تقفل المجلد المسمى بواسطة كل مكون في اسم المسار بالدور. ومع ذلك، يجب على namex تحرير كل قفل في نهاية الحلقة التكرارية، لأنه لو احتجزت أقفاً متعددة فستقع في انسداد متبادل مع نفسها إذا تضمن اسم المسار نقطة (مثل a./b). قد تقع أيضاً في انسداد متبادل مع بحث متزامن يتضمن المجلد و ... كما يشرح الفصل الفصل 11، الحل هو أن تحمل الحلقة التكرارية العقدة الفهرسية للمجلد للتكرار التالي مع زيادة عداد مراجعها، دون قفلها.

بعض عناصر البيانات محمية بآليات مختلفة في أوقات مختلفة، وقد تكون في بعض الأوقات محمية من الوصول المتزامن ضمناً بواسطة هيكل شجرة xv6 بدلاً من الأقفال الصريحة. على سبيل المثال، عندما تكون صفحة فيزيائية حرة، تكون محمية بـ kmem.lock. إذا خصصت الصفحة بعد ذلك كأنبوب، تكون محمية بقفل مختلف (pi->lock) (الدمج). إذا أُعيد تخصيص الصفحة لذاكرة مستخدم لعملية جديدة، فإنها لا تكون محمية بقفل على الإطلاق. بدلاً من ذلك، فإن حقيقة أن المخصص لن يعطي تلك الصفحة لأية عملية أخرى (حتى تُحرر) تحميها من الوصول المتزامن. ملكية ذاكرة العملية الجديدة معقدة: أولاً يخصصها الأب ويتعامل معها في fork، ثم يستخدمها الابن، و(بعد خروج الابن) يمتلك الأب الذاكرة مجدداً ويمررها لـ kfree. هناك درسان هنا: كائن البيانات قد يكون محمياً من التزامن بطرق مختلفة في نقاط مختلفة من دورة حياته، والحماية قد تأخذ شكل هيكل ضمني بدلاً من الأقفال الصريحة.

مثال آخر يشبه بالقفل هو الحاجة لتعطيل المقاطعات حول الاستدعاءات لـ mycpu(). يتسبب تعطيل المقاطعات في جعل الشفرة المستدعية ذرية فيما يتعلق بمقاطعات الميقاتي التي قد تجبر على تبديل السياق، وبالتالي نقل العملية لمعالج مختلف.

13.3 بدون أقفال على الإطلاق

توجد بضعة أماكن يتشارك فيها xv6 بيانات قابلة للتغيير بدون أقفال على الإطلاق. أحدها في تنفيذ أقفال الدوران، على الرغم من أنه يمكن المرء النظر لتعليمات RISC-V الذرية على أنها تعتمد على أقفال منفذة في العتاد. مكان آخر هو المتغير started في main.c المستخدم لمنع المعالجات الأخرى من التنفيذ حتى ينهي المعالج صفر تهيئة xv6؛ يضمن volatile أن المترجم ينشئ بالفعل تعليمات التحميل والتخزين.

يحتوي xv6 على حالات يكتب فيها معالج أو خيط بعض البيانات، ويقرأ معالج أو خيط آخر البيانات، ولكن لا يوجد قفل محدد مخصص لحماية تلك البيانات. على سبيل المثال، في fork يكتب الأب صفحات ذاكرة المستخدم للابن، ويقرأ الابن (خيط مختلف، ربما على معالج مختلف) تلك الصفحات؛ لا يوجد قفل يحمي صراحة تلك الصفحات. هذه ليست مشكلة قفل بالمعنى الدقيق، لأن الابن لا يبدأ التنفيذ إلا بعد أن ينهي الأب الكتابة. إنها مشكلة ترتيب ذاكرة محتملة (انظر الفصل الفصل 8)، لأنه بدون حازر ذاكرة لا يوجد سبب لتوقع أن يشاهد معالج كتابات معالج آخر. ومع ذلك، ونظراً لأن الأب يحرر أقفاً، والابن يستحوذ على أقفال أثناء بدئه، فإن حواجز الذاكرة في acquire و release تضمن أن معالج الابن يشاهد كتابات الأب.

13.4 التوازي

القفل يتعلق أساساً بقمع التوازي لمصلحة الصحة. ونظراً لأن الأداء مهم أيضاً، فإن مصممي النواة يتعين عليهم غالباً التفكير في كيفية استخدام الأقفال بأسلوب يحقق الصحة ويسمح بالتوازي في الوقت نفسه. بينما xv6 ليس مصمماً بأسلوب منهجي للأداء العالي، إلا أنه لا يزال من المفيد النظر في أية عمليات بـ xv6 يمكنها التنفيذ بالتوازي، وأنها قد يتضارب على الأقفال.

الأنابيب بـ xv6 مثال على توازي جيد إلى حد ما. لكل أنبوب قفله الخاص، بحيث يمكن لعمليات مختلفة قراءة وكتابة أنابيب مختلفة بالتوازي على معالجات مختلفة. بالنسبة لأنبوب محدد، يتعين على الكاتب والقارئ الانتظار حتى يحرر الآخر القفل؛ لا يمكنهما قراءة/كتابة نفس الأنبوب في الوقت نفسه. كما أن القراءة من أنبوب فارغ (أو الكتابة في أنبوب ممتلئ) يجب أن تحظر، لكن هذا ليس بسبب مخطط القفل.

تبديل السياق مثال أكثر تعقيداً. خيطان في النواة، كل منهما ينفذ على معالجه الخاص، يمكنهما دعوة `yield` و `sched` و `swtch` في الوقت نفسه، وستنفذ الاستدعاءات بالتوازي. يمتلك كل خيط قفلاً، لكنهما قفلان مختلفان، لذا لا يتعين عليهما الانتظار. بمجرد التواجد في `scheduler`، قد يتضارب المعالجان على الأقفال أثناء البحث في جدول العمليات عن عملية في حالة `RUNNABLE`. أي أنه من المرجح أن يحصل `xv6` على فائدة أداء من المعالجات المتعددة أثناء تبديل السياق، ولكن ربما ليس بقدر ما يمكنه.

مثال آخر هو الاستدعاءات المتزامنة لـ `fork` من عمليات مختلفة على معالجات مختلفة. قد يتعين على الاستدعاءات الانتظار لبعضها البعض من أجل `pid_lock` و `kmem.lock` والأقفال لكل عملية المطلوبة للبحث في جدول العمليات عن عملية في حالة `UNUSED`. ومن جهة أخرى، يمكن للعمليات المنشئين نسخ صفحات ذاكرة المستخدم وتنسيق صفحات جدول الصفحات بالتوازي الكامل.

مخطط القفل في كل من الأمثلة أعلاه يصحى بالأداء المتوازي في حالات معينة. في كل حالة يمكن الحصول على توازي أكثر باستخدام تصميم أكثر تفصيلاً. ما إذا كان ذلك مستحقاً يعتمد على التفاصيل: كم مرة تُدعى العمليات المعنية، كم من الوقت تقضيه الشفرة مع احتجاز قفل متنافس عليه، كم معالجات قد ينفذ عمليات متضاربة في الوقت نفسه، ما إذا كانت أجزاء أخرى من الشفرة تشكل اختناقات أكثر تقييداً. قد يكون من الصعب التكهن بما إذا كان مخطط قفل محدد قد يتسبب في مشاكل أداء، أو ما إذا كان تصميم جديد أفضل بدرجة كبيرة، لذا يُعد القياس على أحمال عمل واقعية أمراً مطلوباً غالباً.

13.5 تمارين

1. عدل تنفيذ الأنابيب بـ `xv6` للسماح للقراءة والكتابة لنفس الأنابيب بالمتابعة بالتوازي على معالجات مختلفة.
2. عدل `scheduler()` بـ `xv6` لتقليل التنافس على الأقفال عندما تبحث معالجات مختلفة عن عمليات قابلة للتشغيل في الوقت نفسه.
3. أُلغ بعض التسلسل في `fork()` بـ `xv6`.

14 الخاتمة

قدّم هذا الكتاب المفاهيم والأفكار الأساسية في نظم التشغيل من خلال دراسة تطبيقية لنظام تشغيل واحد، وهو `xv6`، سطرًا بسطر. وتجسد بعض أسطر الشفرة المصدرية الجوهر الحقيقي للأفكار الرئيسية (مثل تبديل السياق، والحد الفاصل بين مساحة المستخدم والنواة، والأقفال، وغيرها)، ويمثل كل سطر منها أهمية جوهرية؛ في حين تقدم أسطر شفرة أخرى توضيحاً لكيفية تنفيذ فكرة معينة في نظم التشغيل وكان بالإمكان تحقيقها بطرق مختلفة (على سبيل المثال: تطوير خوارزمية أفضل للجدولة، أو بناء هياكل بيانات كفؤة على القرص لتمثيل الملفات، أو تحسين آليات التسجيل للسماح بالمعاملات المتزامنة). وقد وُضحت كافة هذه الأفكار في إطار واجهة استدعاءات نظام واحدة حققت نجاحاً هائلاً، وهي واجهة `Unix`، إلا أن هذه المفاهيم والأسس التجريدية تمتد وتنتقل لتشمل تصميم مختلف نظم التشغيل الأخرى.

15 ملحق الإعراب النحوي والتحليل اللغوي

يُقدم هذا الملحق تحليلاً لغوياً وإعراباً نحوياً تفصيلياً لأهم الجمل المفتاحية، والمصطلحات التقنية، والتركيب العلمية المستعملة في كتاب `xv6 RISC-V`، وذلك وفق القواعد النحوية المعتمدة في التراث اللغوي والأكاديمي.

15.1 أولاً: إعراب الجمل المفتاحية في واجهات ونظام التشغيل

15.1.1 1. جملة: «تَصْنَعُ النواة عَمَلِيَّةً جَدِيدَةً»

- **تَصْنَعُ**: فعل مضارع مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **النواة**: فاعل مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **عَمَلِيَّةً**: مفعول به منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.
- **جَدِيدَةً**: نعت (صفة) منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.

• **الجملة الفعلية:** جملة ابتدائية لا محل لها من الإعراب.

15.1.2.2. جملة: «تُنَقِّذُ العملية إِسْتِذْعَاءَ النَّطَامِ»

- **تُنَقِّذُ:** فعل مضارع مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **العملية:** فاعل مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **إِسْتِذْعَاءَ:** مفعول به منصوب، وعلامة نصبه الفتحة الظاهرة على آخره، وهو مضاف.
- **النَّطَامِ:** مضاف إليه مجرور، وعلامة جره الكسرة الظاهرة تحت آخره.

15.1.3.3. جملة: «يُحَقِّقُ القفل الاستِيعَادَ المُتَبَادِلَ»

- **يُحَقِّقُ:** فعل مضارع مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **القفل:** فاعل مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **الاستِيعَادَ:** مفعول به منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.
- **المُتَبَادِلَ:** نعت (صفة) منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.

15.1.4.4. جملة: «تُخَرِّنُ النواة العُنْوَانَ الإِفْتِرَاضِيَّ فِي جَدُول الصفحات»

- **تُخَرِّنُ:** فعل مضارع مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **النواة:** فاعل مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.
- **العُنْوَانَ:** مفعول به منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.
- **الإِفْتِرَاضِيَّ:** نعت منصوب، وعلامة نصبه الفتحة الظاهرة على آخره.
- **فِي:** حرف جر مبني على السكون لا محل له من الإعراب.
- **جَدُول:** اسم مجرور بـ (في) وعلامة جره الكسرة الظاهرة تحت آخره، وهو مضاف.
- **الصفحات:** مضاف إليه مجرور، وعلامة جره الكسرة الظاهرة تحت آخره (جمع مؤنث سالم).

15.2 ثانياً: إعراب المصطلحات الفنية الموحدة (TermBase)

15.2.1.1. مصطلح: «مخزن الترجمة المؤقت» (Translation Lookaside Buffer - TLB)

- **مَخَرَّنُ:** مبتدأ (أو فاعل بحسب موقعه في الجملة) مرفوع، وعلامة رفعه الضمة الظاهرة على آخره، وهو مضاف.
- **التَّرْجَمَةُ:** مضاف إليه مجرور، وعلامة جره الكسرة الظاهرة تحت آخره.
- **المُؤَقَّتُ:** نعت (صفة) لـ (مخزن) مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.

15.2.2.2. مصطلح: «العقدة الفهرسية» (Inode)

- **العُقْدَةُ:** اسم مرفوع (أو بحسب الإيقاع النحوي بالجملة) وعلامة رفعه الضمة الظاهرة.
- **الفَهْرَسِيَّةُ:** نعت (صفة) مرفوع، وعلامة رفعه الضمة الظاهرة على آخره.

15.2.3.3. مصطلح: «واصف الملف» (File Descriptor)

- **وَاصِفُ:** اسم مرفوع بالضمة، وهو مضاف.
- **المَلَفُ:** مضاف إليه مجرور وعلامة جره الكسرة الظاهرة.

15.3 ثالثاً: قواعد ضبط أواخر الكلمات والجماليات النحوية في الترجمة

1. **الضبط مع الأفعال والأسماء الإنجليزية المدمجة:** عند إدخال اسم دالة مثل fork () أو exec ()، يُعامل الاسم من الناحية النحوية كـ اسم علم أجنبي ممنوع من الصرف، ولا تظهر عليه علامات الإعراب، بل تُقدر العوامل النحوية تقديرًا.
 - مثال: «تَسْتَدْعِي العملية fork -> fork()»: مفعول به مبني في محل نصب.

2. **المثنى والجمع في المصطلحات الفنية:**

- **المفرد:** مَخَرَّنُ ترجمة مؤقت.
- **المثنى:** مَخَرَّتَا ترجمة مؤقتان (رفع الألف وحذف النون للإضافة).
- **الجمع:** مَخَارِنُ الترجمة المؤقتة.